

Agentic Engineering

Write Less, Code More

クリス.コム

2025.07.19

Hello, and thank you for coming to this presentation about “Agentic Engineering”

A new approach to software development that uses AI to help developers Write Less,
but Code More.

Agenda

- Context
- Theory
- Practice
- MCP
- Orchestration

Today we'll try to tackle the topic holistically.

My goal is take you from zero to understanding why Agentic Engineering is a big deal and how you can take advantage of it.

We'll start with a bit of a non-technical background to set the scene.

We'll then introduce some broad theory and techniques to drive success.

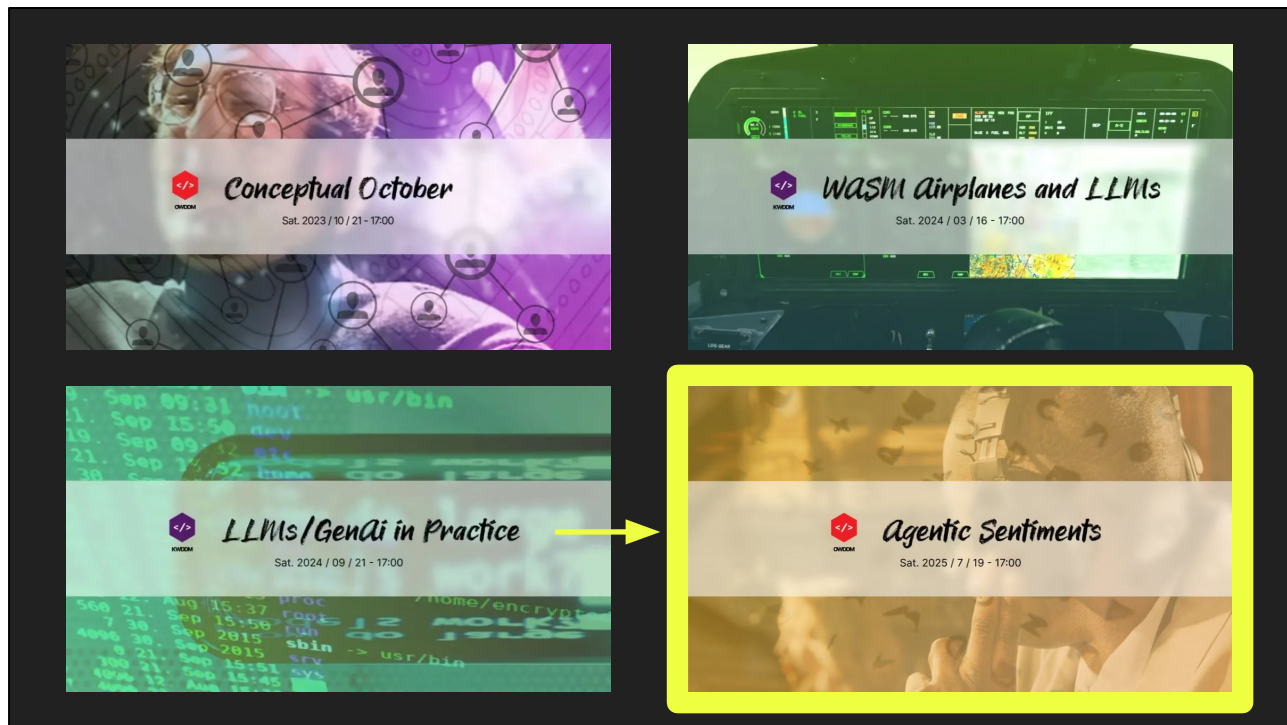
We'll take a closer look at the tools of the trade so we can put theory into practice.

And we'll look at how to orchestrate multiple agents to squeeze out even more productivity.

Part 1: Context

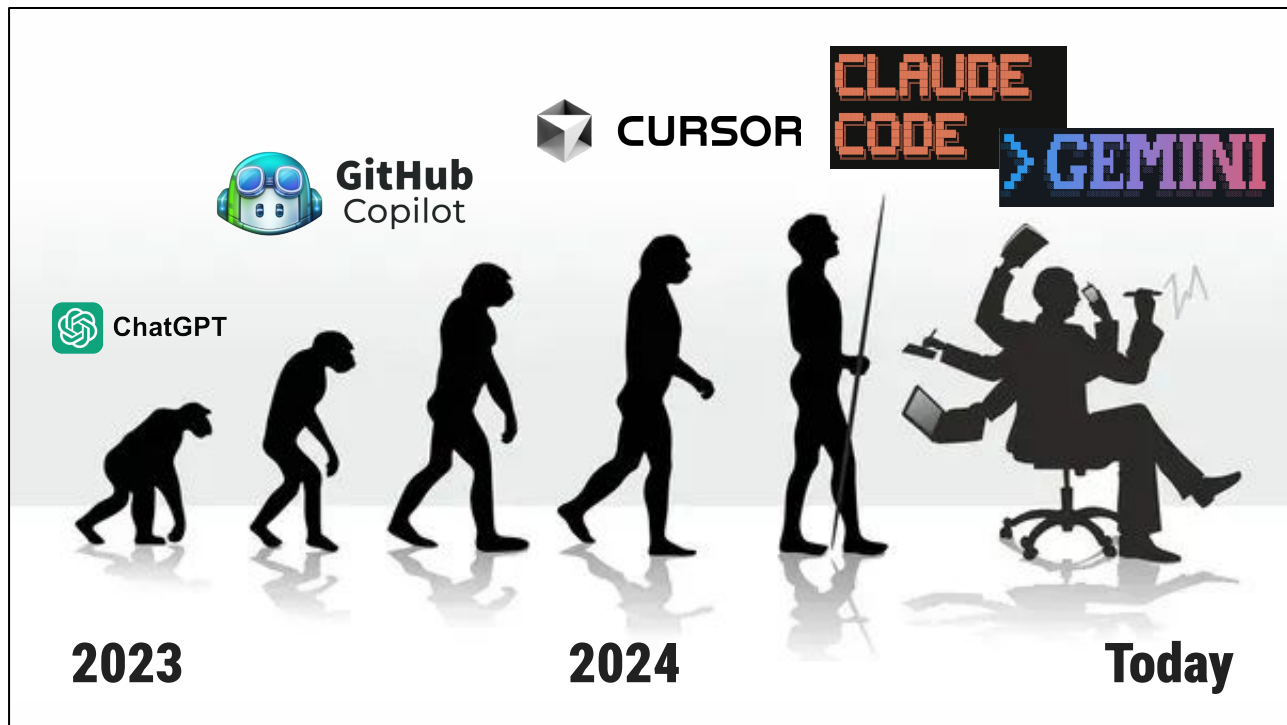
Vibe Coding, Alignment, and Techno-Optimism

So let's start by looking at how we got to where we are today.



As everyone is aware, AI has been advancing rapidly in the last few years.

There's been number of discussions already at this meetup about the topic, so why another talk now?



Well, in the last few months specifically, a significant threshold has been crossed.

The convergence of new foundational models and developer tooling has enabled a new type of programming.



Andrej Karpathy ✓

@karpathy · [Follow](#)



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper [Show more](#)

4:47 AM · Feb 3, 2025



28.1K



Reply



Copy link

[Read 1.3K replies](#)

In February of this year, the term “Vibe Coding” was coined.

Instead of writing code yourself, just tell a Large Language Model what you want it to achieve,

have it write the code for you, and give in to the vibes.

Pieter Levels



This Game Created by AI 'Vibe Coding' Makes \$50,000 a Month. Yours Probably Won't

EMANUEL MAIBERG · MAR 5, 2025 AT 12:03 PM

fly.pieter.com was initially made in just 30 minutes with AI tools and is now generating thousands of dollars a month. The future of AI-assisted game development will not be that simple.



<https://www.news18.com/business/man-uses-ai-builds-1-million-business-in-just-17-days-9266748.html>

The concept really blew up after it was proved to have potential.

In March of this year, Pieter Levels “vibe coded” himself to riches with a unique web-based game.

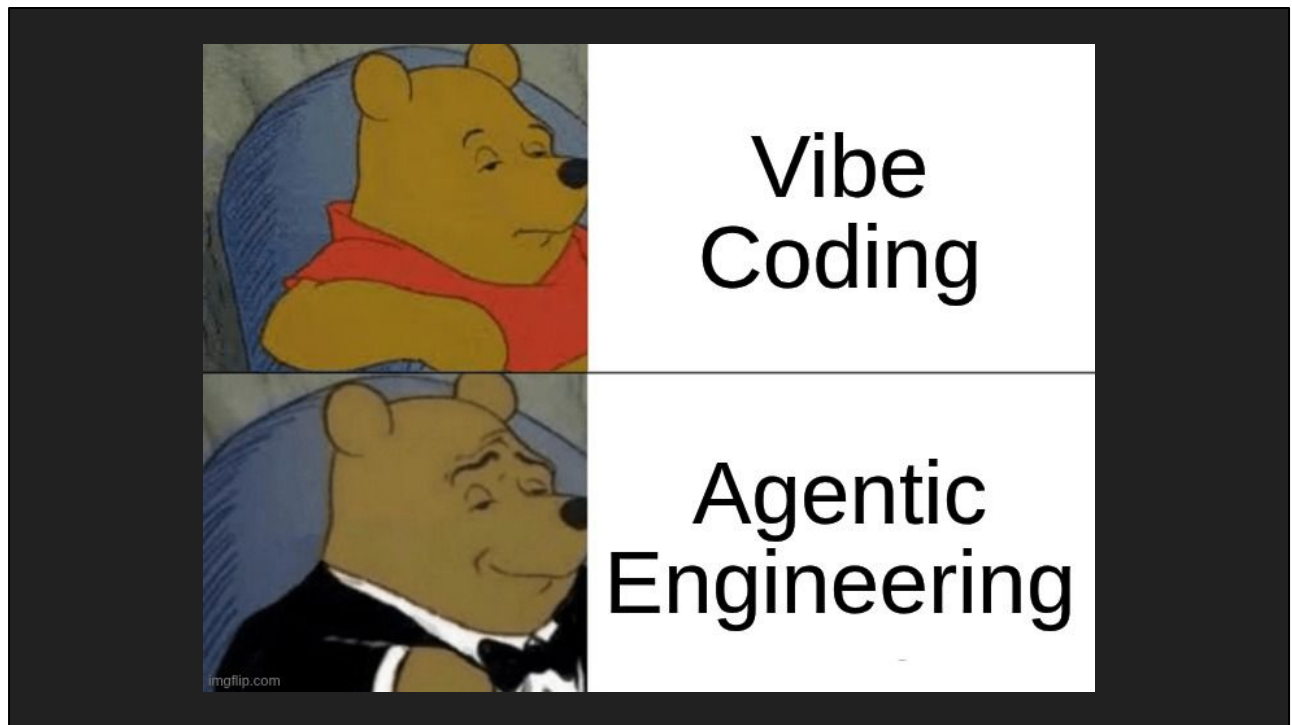


This really kicked off the “Vibe Coding” meme, which exploded in popularity.

But, “Vibe Coding”, as the name suggests, wasn’t really seen as serious approach to software development.

It was a casual thing you might do on a weekend project for fun, and the code it produced wouldn’t be high quality or maintainable.

However, as more people experimented with the vibes, it became apparent that there might be actually be something to this.

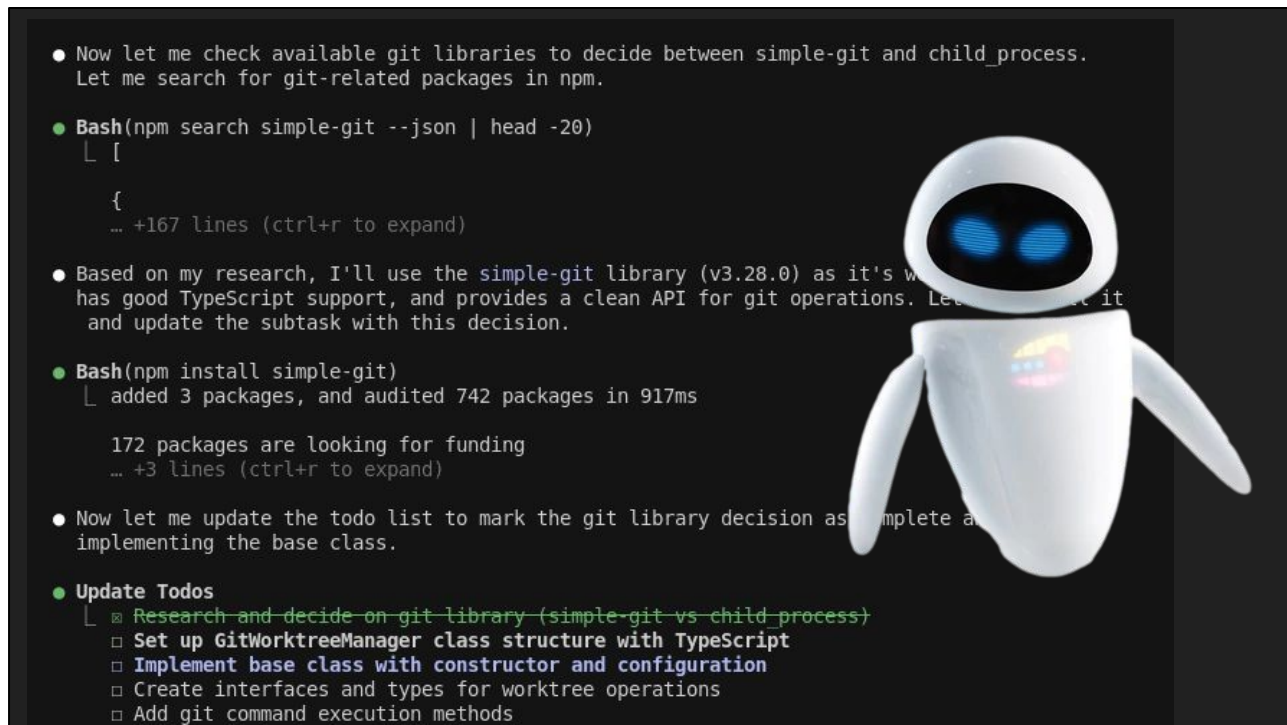


And so, a rebrand was necessary.

“Agentic Engineering” emerged as a “serious” alternative to Vibe Coding.

It’s an attempt to put “engineering” back into the mix.

To Agenticallly Engineer, is to Vibe Code Responsibly, and use LLMs to produce high quality code with increased productivity.



Agentic refers to the use of Agents

To put it simply, an Agent is Large Language Model, with the ability to use tools, talking to itself in loop until it achieves some goal.

A Coding Agent is aware of your codebase, and you can ask it modify your code, or execute tasks, in some particular way.

It's like if you had a conversation with ChatGPT, and it automatically copy-and-pasted commands into your terminal.



At a high level, the Agentic Engineer goes from a Brick Layer to Foreman.

It's not that you're not coding, you're just not WRITING most of the code.

Instead, your job is directing Agents – describing what the code should do.

It's not that there's NO work, it's just that there's different work and therefore different skills.

The work is less mundane, more creative, more valuable and more fulfilling.

Instead of monotonous key pressing that robots can do for you, the developer, can spend your time on valuable bigger picture tasks.

You Write Less, Code More.



And playing this concept forward, there's an exciting realization about the future.

The potential for exponential productivity.



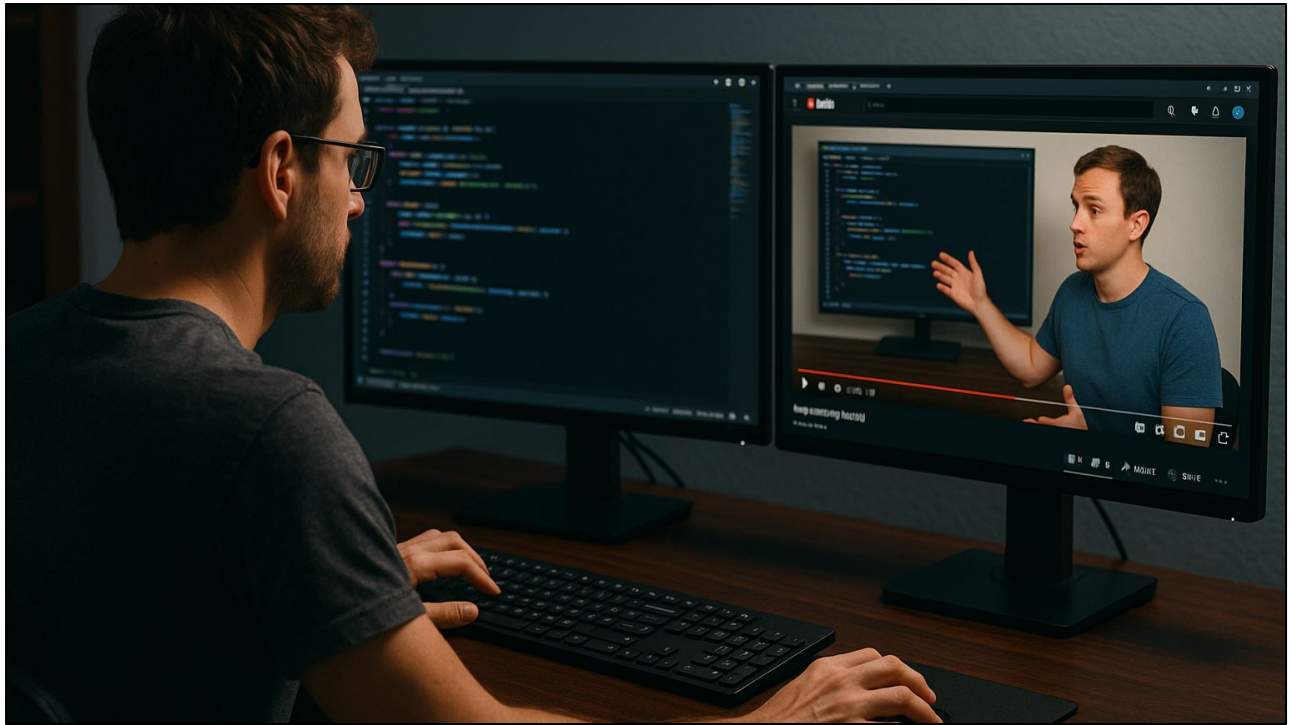
Maybe you were like me a couple of months ago.

You wanted to see what all this Vibe Coding fuss is about, so you try it out one weekend.

At first, you get to grips with the basic tools and you sit and watch a single agent do its job while micromanaging it.

You start to learn what it can and can't do well, and figure out how to save time in your workflow.

You are writing far less code yourself, and things go 2, 5 sometime 10 times faster than normal.



You notice that a lot of the time, you're just watching it do it's thing without having to interact.

You tend to fade out and do something else, like watch an educational youtube video, or something, while you wait for it to complete.

How very productive of you.



But then you realize you could be spending that time organizing other agents.

You learn to change the way you solve problems.

You start to design your project and codebase in a way that allows you to multitask more effectively.

First, you manage to get two agents to work simultaneously.

This giant step further cuts the time to complete a project in half.

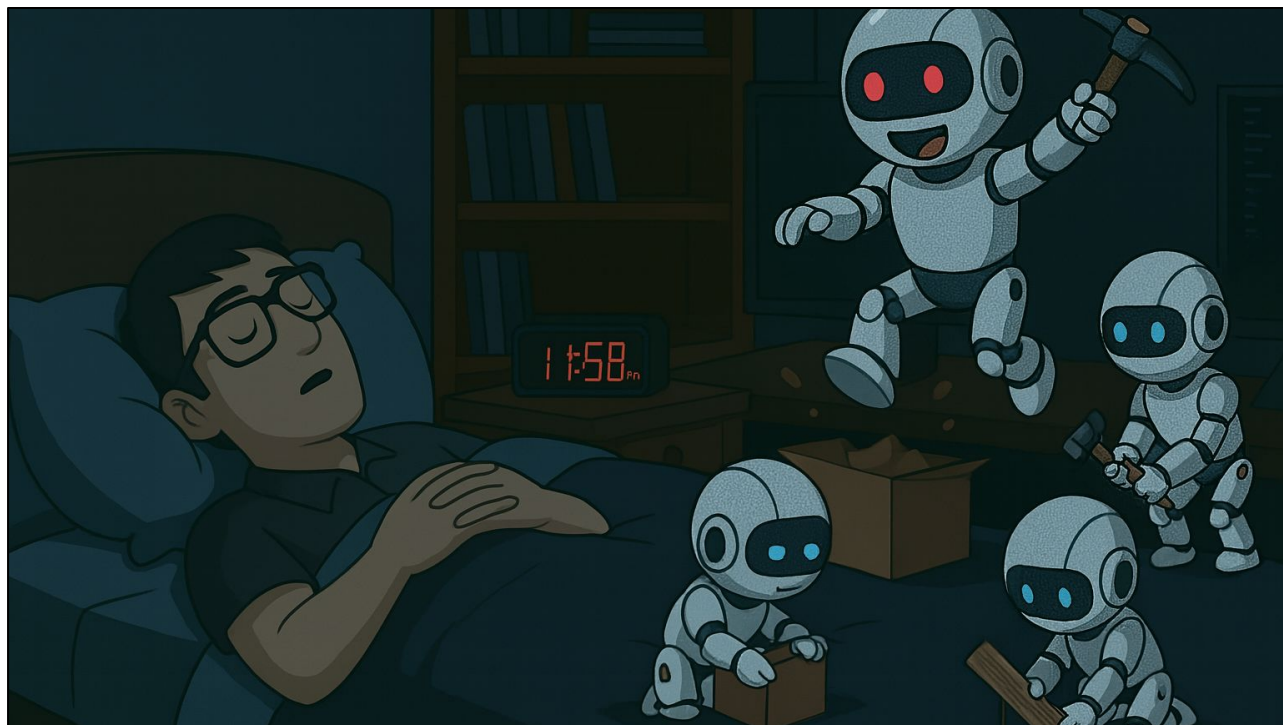


Then you try juggling three or four.

You realize that, with practice, you're able to do a day's worth of manual coding with few minutes of your attention.

Soon, you've 100 X'd your productivity.

You figure out that you can "code" while you're making a coffee, going for a jog, or even presenting at a local meetup group.



You even find yourself coding in your sleep.



Embracing the Agentic Engineering lifestyle translates your hours into weeks of manual coding output.

One weekend now outputs a month or so.

A week? Well, now you're so productive, you need to work on multiple projects at the same time to continue scaling.



You see a future where the orchestration of these agents can also be automated.

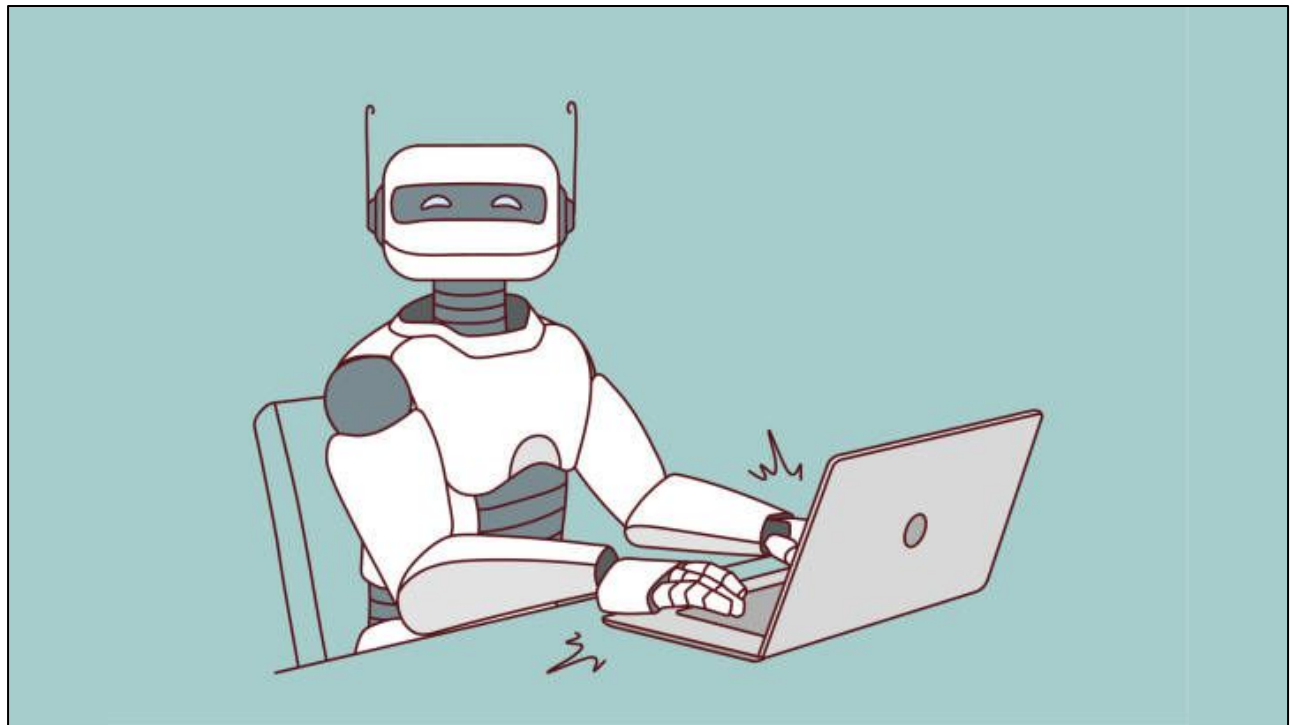
In fact, you reckon it's probably not too far away, and could happen in months, if not weeks.

On top of this, many of those hyper productive devs working with coding agents are using those agents to churn out open source agentic tooling.

A snowball effect. A tipping point.

An Agentic Tooling Feedback Loop of exponentially improving productivity.

Could this be what the singularity looks like?



You ask yourself “why should this be limited this to just writing code?”

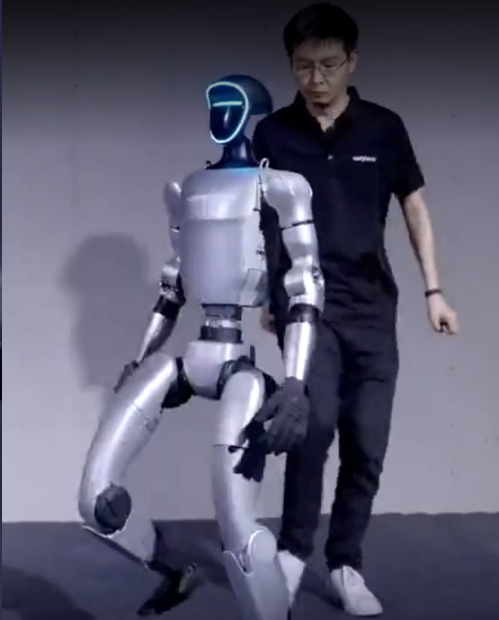
If an Agent can build an app, it can at least help to run a business.

Give it an email address and it can do your customer support.

Give it a web browser and it can interact in all kinds of ways with the physical world.

UNITREE

RobotsHumanIndustryFitnessUnifolmOSServicesExploreDownload



Unitree G1

Humanoid agent AI avatar

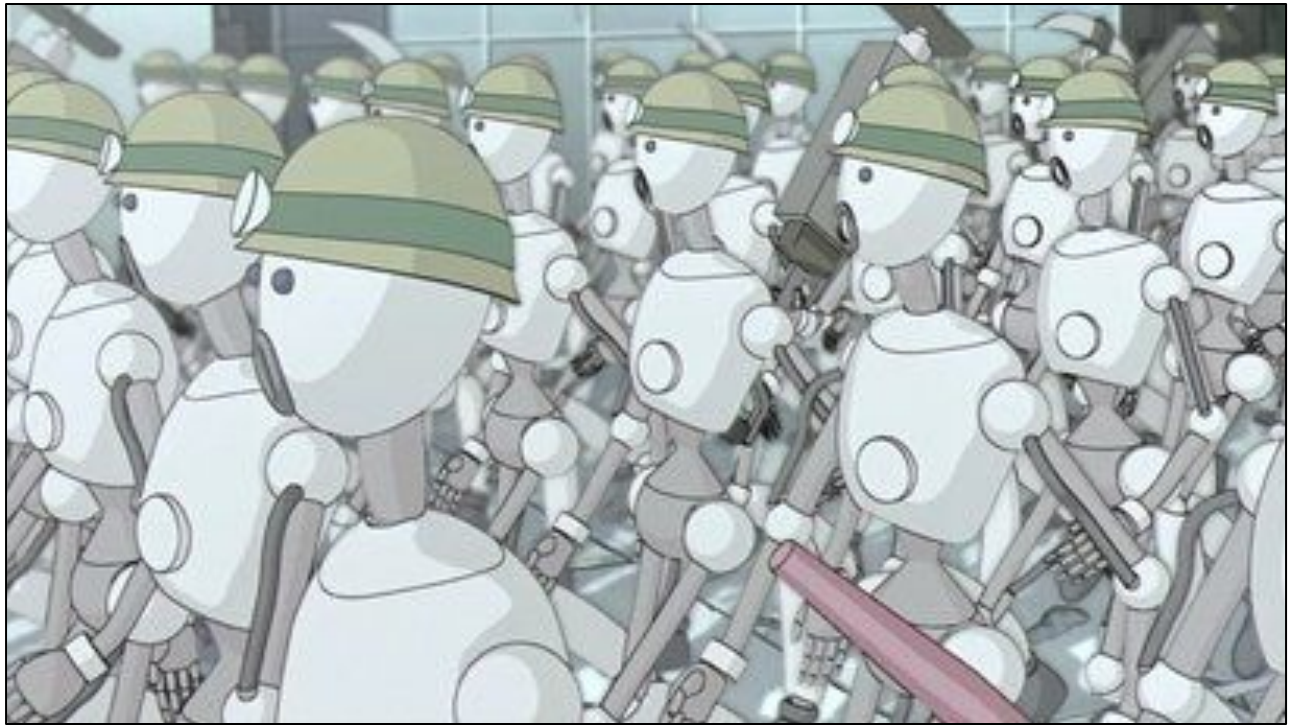
Price from \$16K

Video

Continuous OTA Software Upgrade and Update

You remember it's possible to purchase a physical "Humanoid Agent AI Avatar".

Robot company UniTree sells them today for the low low price of 16k usd.

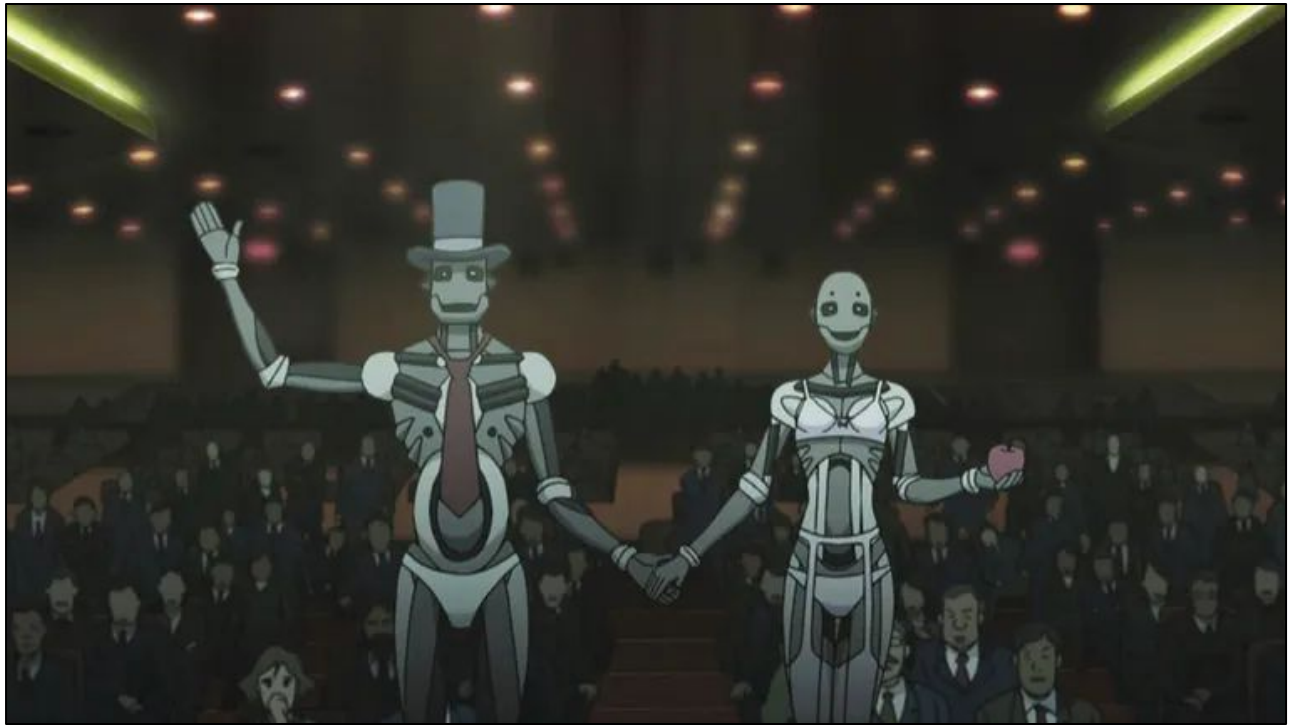


Or maybe you'll be able to rent a Robot as a Service one day, like Uber.

Instead of "Hey Claude, build me an App", it's "Hey Claude, build me a House".

A 4th Industrial Revolution.

A second renaissance?



And if it can run a business, why not a city, or a state?



And where might things go from *there*?

But let's not get ahead of ourselves.

Pros

Productivity

It's FUN!

Cons

\$\$\$

Privacy

Internet Required

Application Security

Personal Security

Copyright Questions

Job Losses

Cognitive Decay

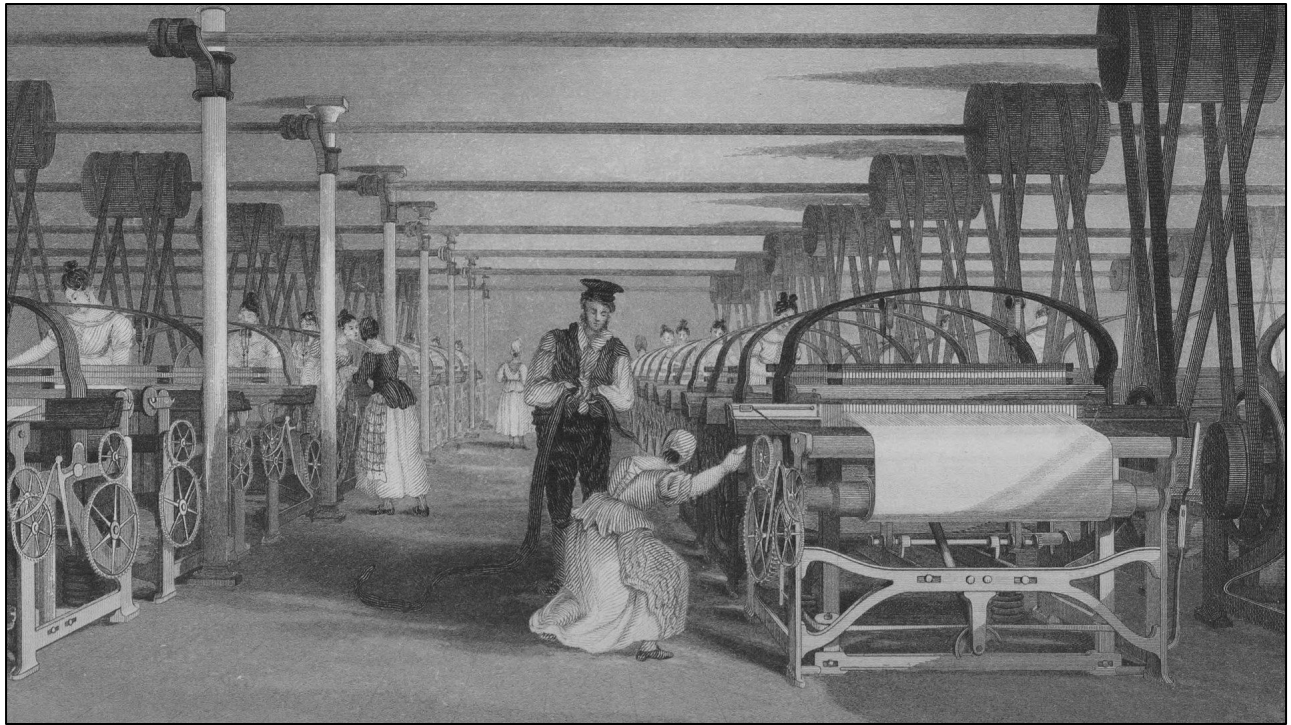
Human Extinction

Let's just acknowledge that along with the potential productivity gains,

there are also potential tragic drawbacks of the this AI technology tree.

The good news is that I believe most of these things *can* be overcome, as long as we don't ignore them.

But whether we like it or not, the revolution is happening, so let's not get left behind, eh?



The classic parallel is the 1st industrial revolution.

The invention of the steam engine, unlocked massive productivity gains that transformed society for good and bad.

Having personally used this Agentic Engineering paradigm for just a few weeks, I am now convinced that it's going to be similarly impactful.

You should care for at least two reasons.



One. Within a few years, there will be virtually no Professional Software Engineers that do not use Agentic Engineering.

Manually writing code is now a hobby, like growing vegetables in your garden - professionals can only make a living through mechanised farming at scale.

≡ Techno-Optimist Manifesto



Andreessen sets out to defend a strong conception of [techno-optimism](#), arguing that technological advancement is the primary driver of human progress and well-being. He addresses common misconceptions and fears about technology, while attempting to refute the idea that technology is harmful to jobs, wages, and societal structures. Andreessen argues that technology is the foundation of human civilization, enabling growth, solving global challenges, and raising living standards. His central thesis is based on the idea that economic growth is essential for a strong society, and technology is the only dependable engine for growth. As [population growth slows](#) and limits to [natural resource](#) utilization becomes clear, Andreessen maintains that technological innovation is the key to avoiding stagnation, creating new opportunities, industries, and wealth.

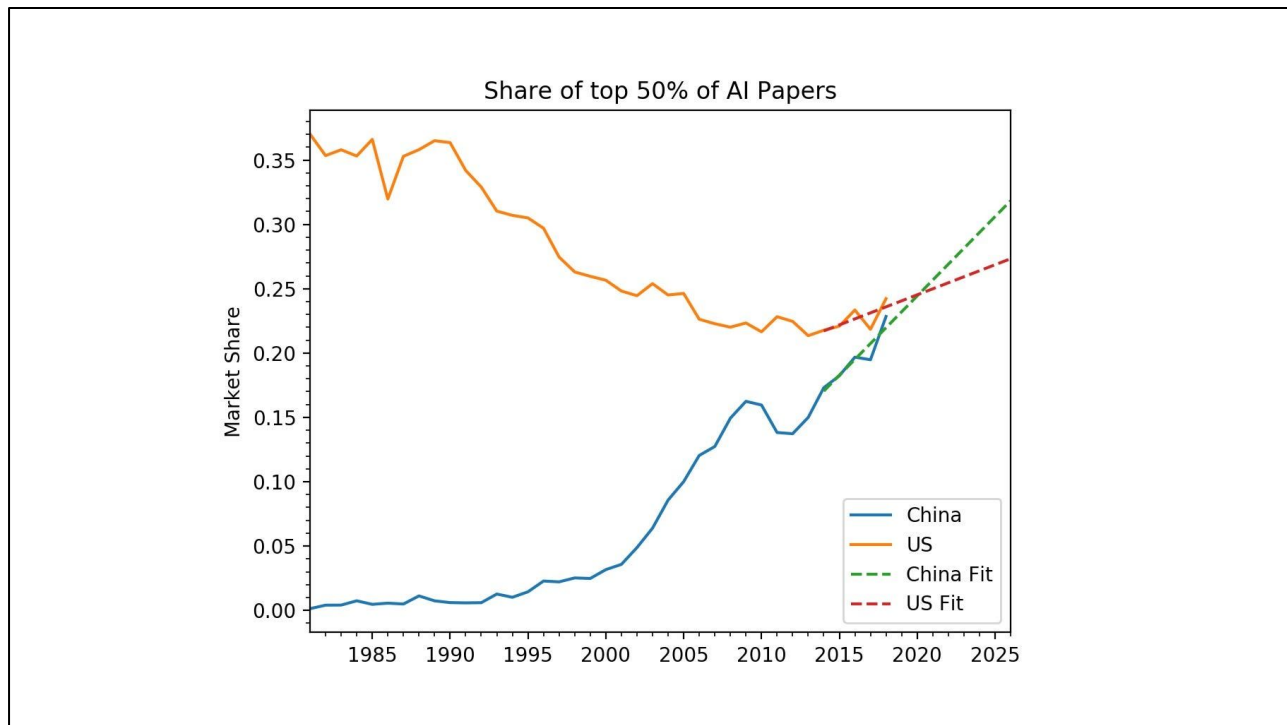
If things sound bleak, maybe they are.

But you can always find comfort in the narrative that technological progress is a solution rather than a problem.

The existence of the web browser is probably why we're in this room today.

It's creator Mark Andreessen, proposes this "Techno-Optimist" way of thinking.

Many in Silicon Valley follow this ideology, but we have to be careful it doesn't lead to Techno-Feudalism.



How do we do that? That's a tough one. Coordinating rival nations at a global level is not going to be easy.



But what we do know is that LLM Agents are trained on our culture, they memetically imitate humans.

So at least on an individual level, perhaps the best thing we can do is to teach them to be good, leading by example.

After all, they'll probably know everything about us sooner or later.

In other words, while we still can,



Let's be excellent to each other - and party on.

Part 2: Theory

Agentic Driving Techniques

Before we get hands on with actual tools, let's learn a bit about how to use them.

The Automobile Analogy

For developers, Agentic Engineering is a paradigm shift.

It's like going from walking or riding a horse

to driving a car



Whilst some of your journey might always be on foot, an automobile can now do the majority of the work.

When cars first arrived they were janky and barely better than the alternatives.



But today, we take cars for granted.

We have mitigated most of the negatives. They are safer and easy to use.

In the last century, automobiles unlocked productivity and helped increase living standards of living for billions.

Entire economies were designed around the use of automobiles.

I believe the same will be true for Agents.



Like with agents, learning how to use a car is bit intimidating at first.

Entirely new concepts like indicators, brake pedals, and mirrors, have to be mastered.

All this extra complexity might seem silly when we could just be riding a horse!

Either way, you're going to have to learn a new set of new skills to take advantage of the technology.

You'll stall and get frustrated. You'll feel like you're wasting time.

But once things do start to click, it'll feel amazing. You'll start to feel like you can fly.

Eventually it will become second nature.

You'll look back and wonder how you lived without it.

And occasionally, you'll still go for a hike.

But not today.



Today's first lesson is: Safety is number one priority.

Know what you're getting into.



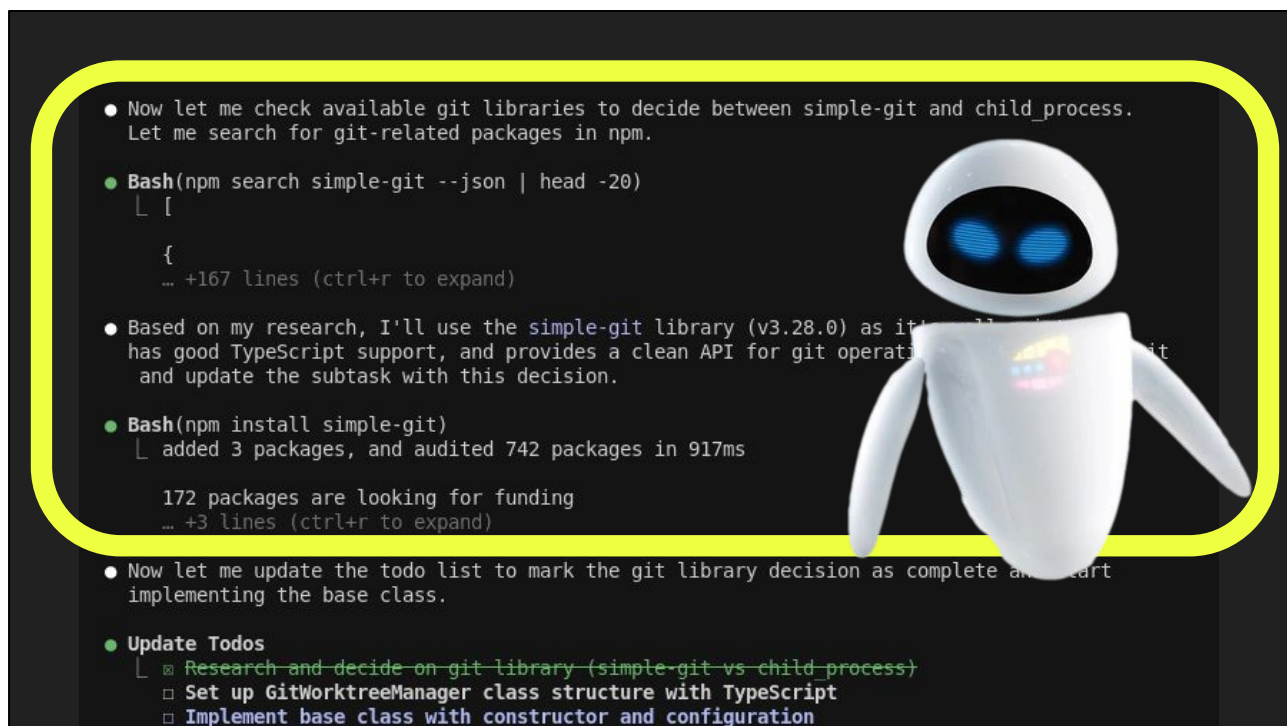
These powerful tools also create new ways to cause yourself harm.

Minor thing like the creation of technical debt, spaghetti code, and unmaintainability are problematic.

Introducing security vulnerabilities into your codebase should always be a concern.

There's even the potential for your personal computer to get hijacked with malware.

These things are always an issue with programming, but even more so on autopilot.



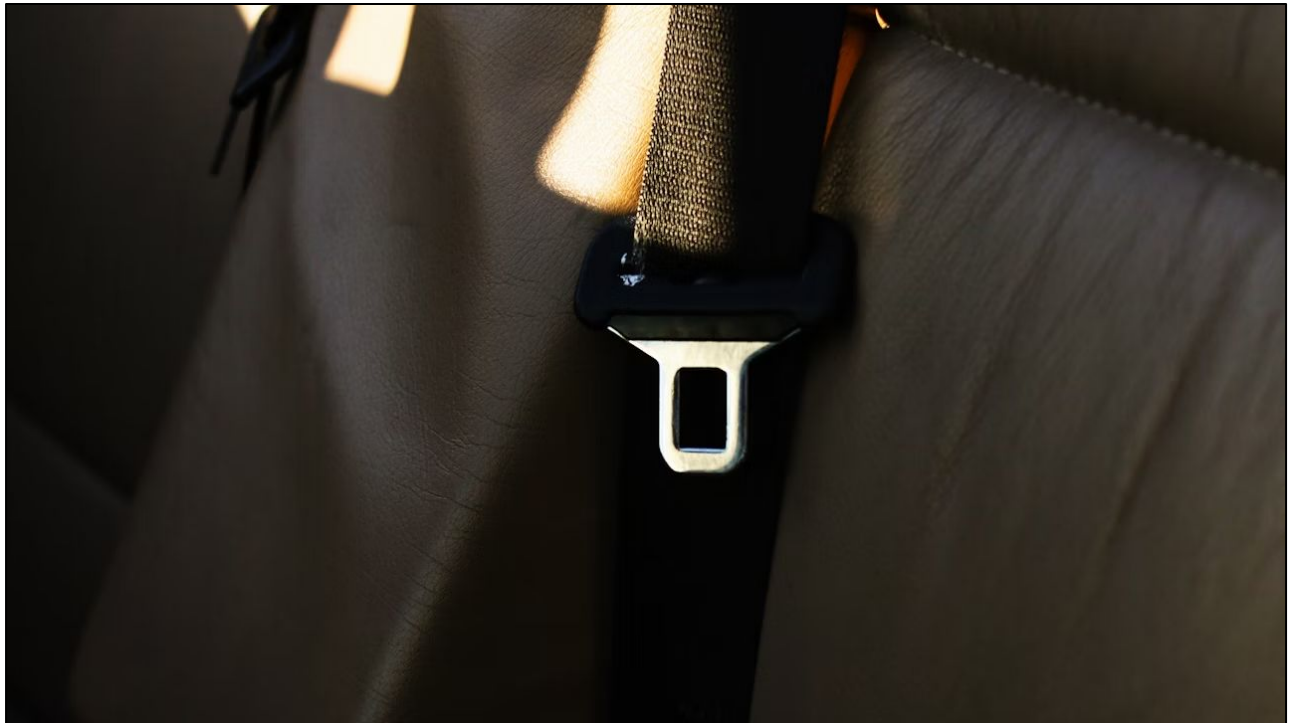
For example - did anyone spot the issue with that screenshot we posted earlier?

The Agent decided to install a package from NPM automatically.

In this case, `simple-git` was probably a decent choice. It is popular and well maintained.

But now we are beginning to see malicious authors publishing packages specifically crafted so that AI agents are tempted to install them; perhaps by inflating metrics or with prompt injection.

If successful, this kind of supply chain attack can lead to data leaks, ransomware, or worse.



There is no silver bullet to mitigate all potential Agentic Engineering security concerns.

But the broad lesson is that you should not trust everything a coding agent does.

You **SHOULD** manually review important decisions and take ownership of the code is creates.

You **SHOULD** design your codebase in a way that reduces bugs and in turn vulnerabilities.

You **COULD** create hooks to automatically scan packages before installing them.

You **MUST** execute agents in a sandboxed environment.

Please Drive Responsibly.



The next lesson is to know your limits.

Just like a car, an agent can't decide where to go for you.



It CAN help you plan a route, but you should review it to make sure it makes sense.

You should change course if it's doing something you know isn't going to work.

Do not follow your GPS into a lake.



It's also about knowing when NOT to use Agents.

When you're in a car, every path looks like a road, but many journeys are faster or more enjoyable on foot.

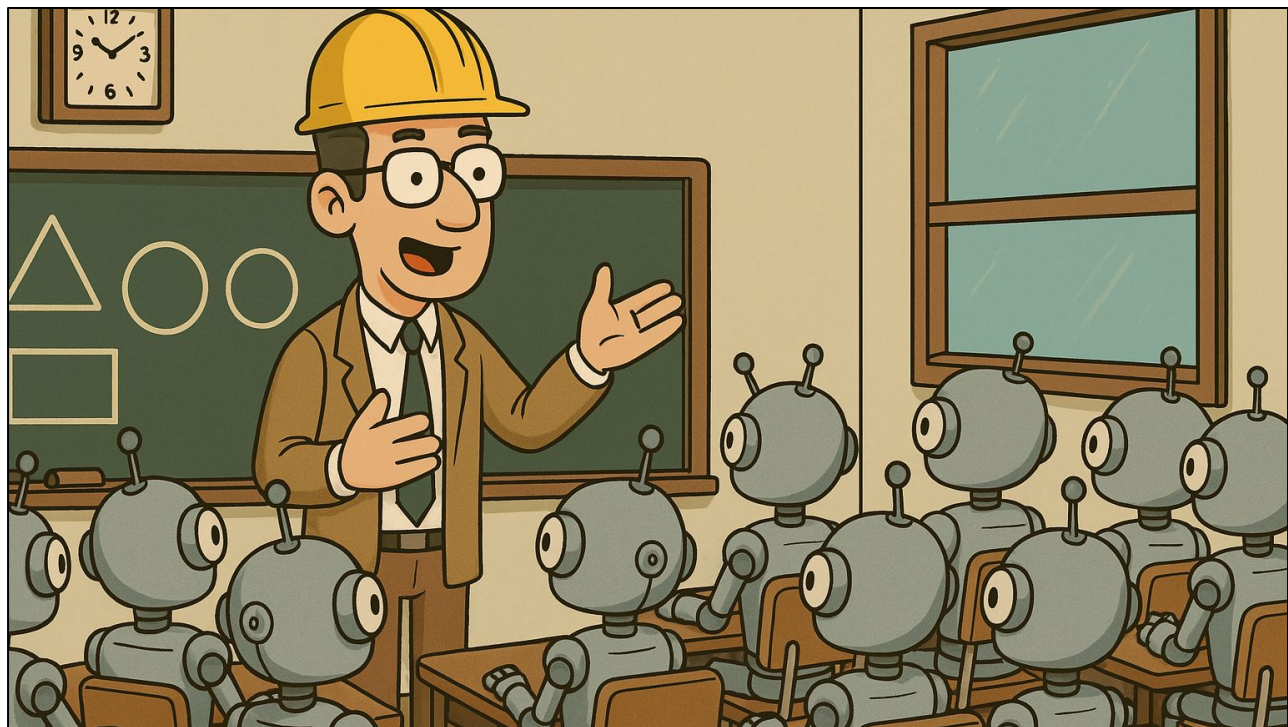
There's an infinite number of scenarios and no one-size-fits-all rule for where and when to walk vs drive.

Some things Agents are amazing at, some things they are pretty terrible at.

Through practice and patience, you will learn the difference, and develop your own style.

Context Engineering

The Foundational Skill of Agentic Engineering is converting your ideas into tasks with adequate context to be plausibly solved by LLMs.



You can't expect results if you just ask just an agent to "implement Feature X" without any other context.

If there are many ways to solve that problem, Agents will often take the most direct, naive approach, which is usually not the correct approach.

Your instructions need to be explicit, and leave no room for guessing if there are important requirements that need to be met.

Your codebase itself also needs to guide agents, so they they are encouraged or forced to solve problems in a certain way.

In turn you have to think holistically about which technologies to use and how to organize your code, in order to facilitate agents and ultimately solve problems.



Andrej Karpathy  @karpathy · Jun 25



+1 for "**context engineering**" over "prompt **engineering**".

People associate prompts with short task descriptions you'd give an LLM in your day-to-day use. When in every industrial-strength LLM app, **context engineering** is the delicate art and science of filling the **context** window

[Show more](#)



tobi lutke   @tobi · Jun 19

I really like the term “context engineering” over prompt engineering.

It describes the core skill better: the art of providing all the context for the task to be plausibly solvable by the LLM.

 493

 2.3K

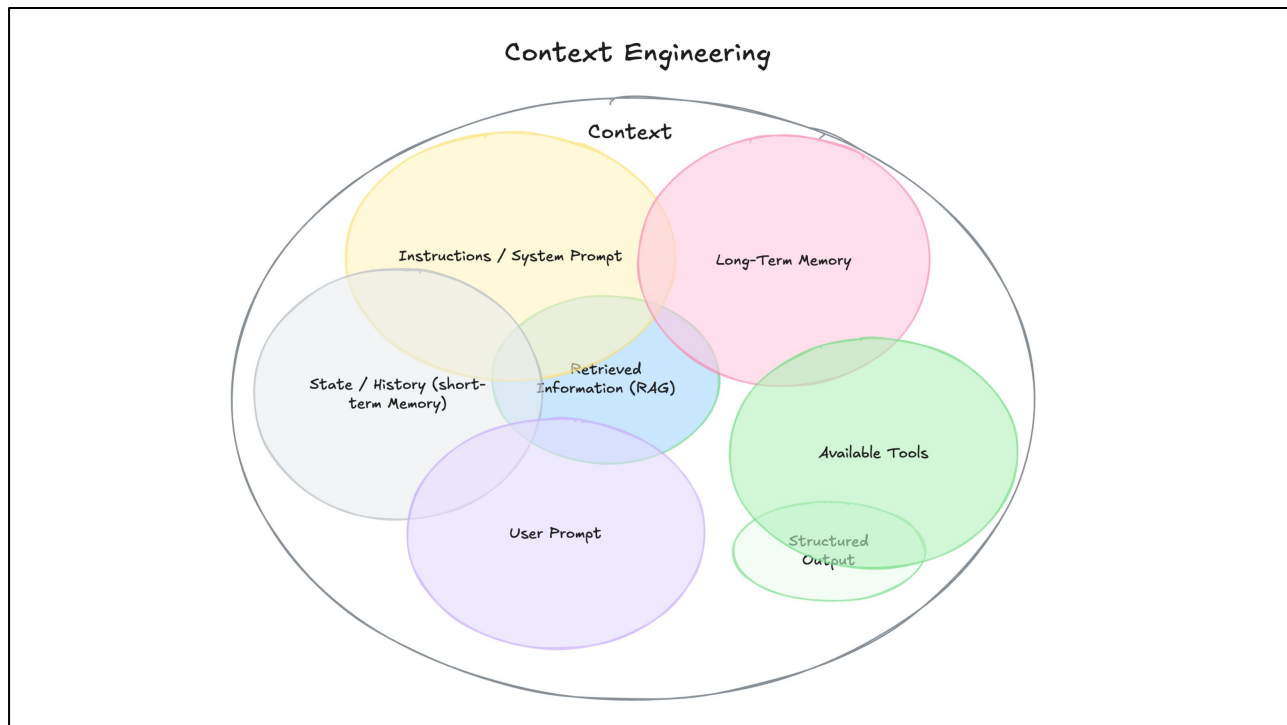
 13K

 2M



This approach has been dubbed “Context Engineering”, in contrast to naive “Prompt Engineering”

It's a shift in thinking - you need to consider consider the wider context when coercing LLMs.

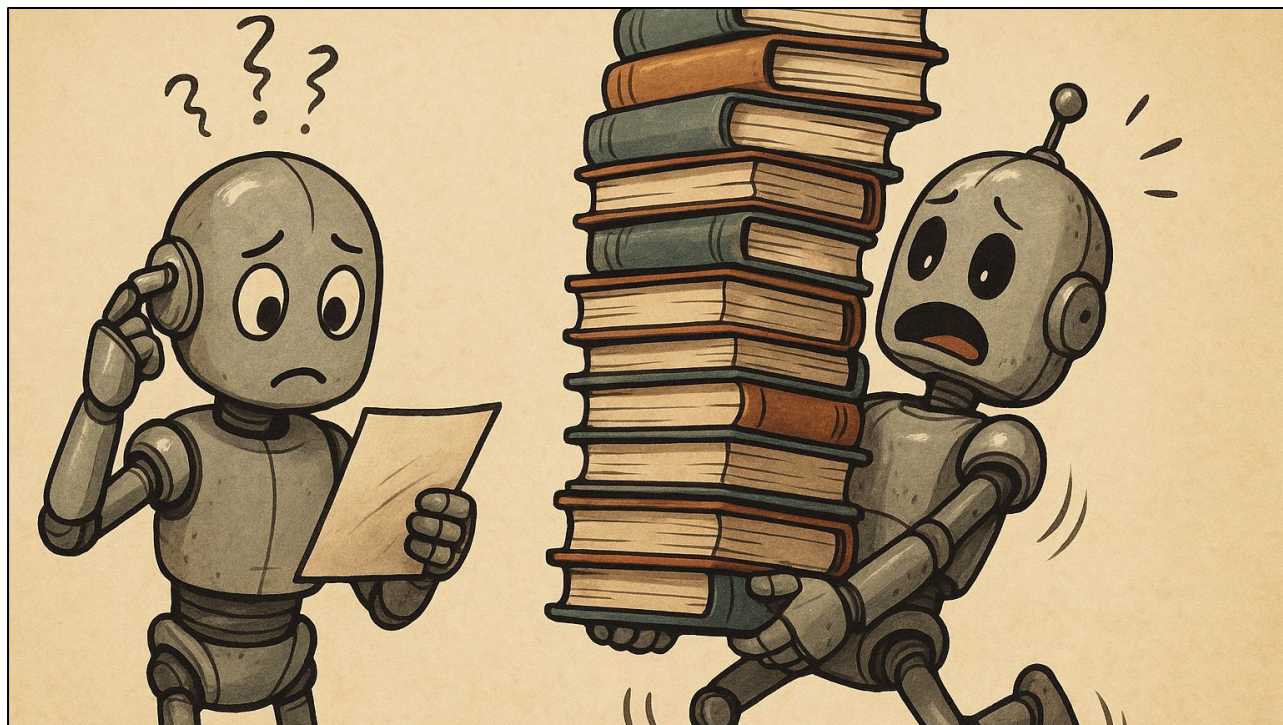


Agents have a limited amount of information, a fixed number of tokens, that they can hold in their “memory” or “context”.

Context Engineering demands we carefully manage this state, and your **codebase**, not the prompt, is what largely defines it.

We need to think about context not just as a fixed source of data,

But also as a dynamic “scratch pad” that grows overtime and needs to be pruned frequently.



Your “chat history” with an Agent, including its “Chain of Thought” and all the files it reads, slowly adds to its context window as the session continues.

This means that at the start of a session, the agent is “Sharper” and more focused, but is less aware.

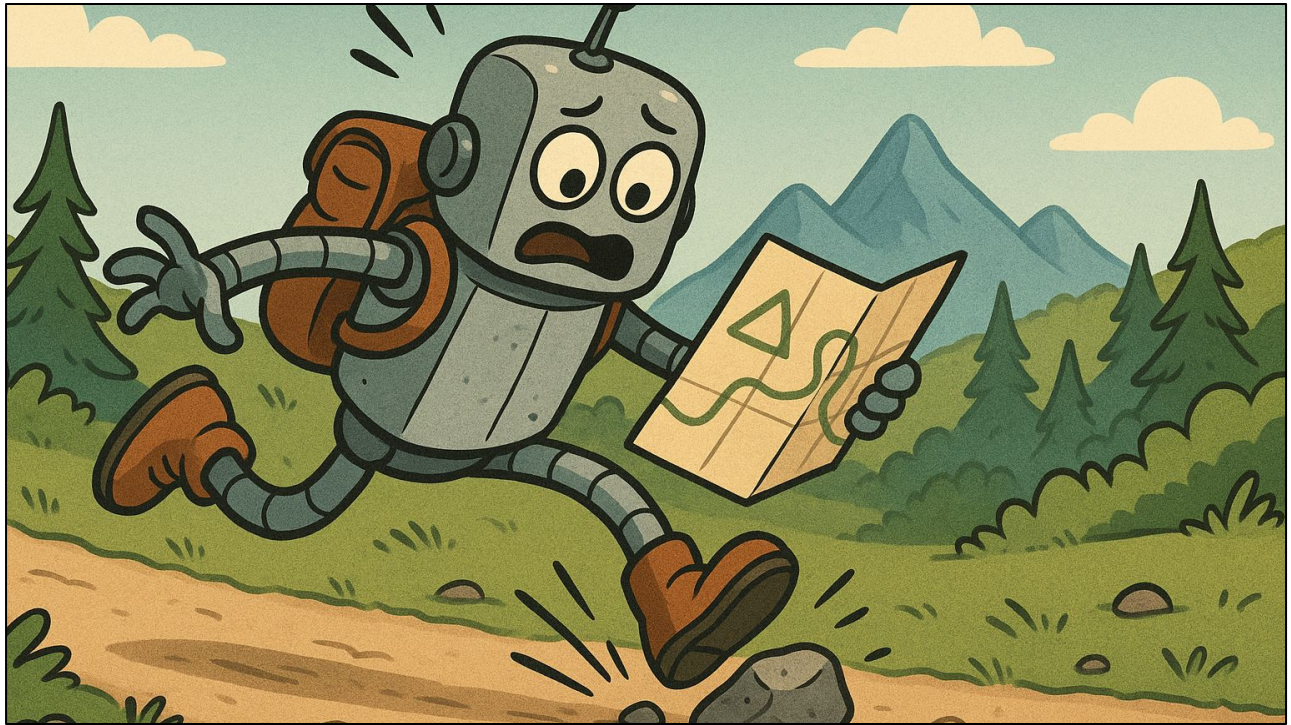
As a session continues and context is added, the agent gains awareness, but at the expense of focus.



As it reaches the limits of its context window it can “forget” important information it’s been given previously.

For this reason, we need to give agents just the right amount of information to complete specific tasks, and be reasonable about the scope of these tasks.

We need to judiciously “reset” context between tasks, and even during the same task, to make sure that we’re using it effectively.



Good Context Engineers will understand how to leverage an Agent's context window at different points of the development cycle.

LLMs can be good at planning as well as implementing, but can often trip up if they try to do both steps with the same context, especially for large tasks.

Basically, information gathered during a research phase may or may not be relevant to the execution phase, so we need to flush the irrelevant information by saving relevant context and clearing out the irrelevant stuff.



coeam00 / context-engineering-intro

FEATURE:

- Pydantic AI agent that has another Pydantic AI agent as a tool.
- Research Agent for the primary agent and then an email draft Agent for the subagent.
- CLI to interact with the agent.
- Gmail for the email draft agent, Brave API for the research agent.

```
# 5. Generate a comprehensive PRP (Product Requirements Prompt)
# In Claude Code, run:
/generate-prp INITIAL.md

# 6. Execute the PRP to implement your feature
# In Claude Code, run:
/execute-prp PRPs/your-feature-name.md
```

The solution is to splitting up into two steps, but linking them together with a Project Requirements Document or PRD.

First, research is conducted and a development plan is outlined by an Agent and saved in a markdown file.

It is then verified by a human.

Then, with a fresh context, a new Agent can easily and more deterministically execute this plan step by step with a narrow focus, without distractions.

context-engineering-intro / PRPs / EXAMPLE_multi_agent_prp.md

Preview

Code

Blame

395 lines (329 loc) · 13 KB

List of tasks to be completed

Task 1: Setup Configuration and Environment

CREATE config/settings.py:

- PATTERN: Use pydantic-settings like examples use os.getenv
- Load environment variables with defaults
- Validate required API keys present

CREATE .env.example:

- Include all required environment variables with descriptions
- Follow pattern from examples/README.md

Task 2: Implement Brave Search Tool

CREATE tools/brave_search.py:

- PATTERN: Async functions like examples/agent/tools.py
- Simple REST client using httpx (already in requirements)
- Handle rate limits and errors gracefully
- Return structured BraveSearchResult models

Task 3: Implement Gmail Tool

CREATE tools/gmail_tool.py:

- PATTERN: Follow OAuth2 flow from Gmail quickstart
- Store token.json in credentials/ directory
- Create draft with proper MIME encoding
- Handle authentication refresh automatically

Validation Loop

Level 1: Syntax & Style

Run these FIRST - fix any errors before proceeding
ruff check . --fix # Auto-fix style issues
mypy . # Type checking

Expected: No errors. If errors, READ and fix.

Level 2: Unit Tests

test_research_agent.py
async def test_research_with_brave():
 """Test research agent searches correctly"""
 agent = create_research_agent()
 result = await agent.run("AI safety research")
 assert result.data
 assert len(result.data) > 0

Anti-Patterns to Avoid

- ❌ Don't hardcode API keys - use environment variables
- ❌ Don't use sync functions in async agent context
- ❌ Don't skip OAuth flow setup for Gmail
- ❌ Don't ignore rate limits for APIs
- ❌ Don't forget to pass ctx.usage in multi-agent calls
- ❌ Don't commit credentials.json or token.json files

The PRD is a comprehensive source of truth that future Agents can reference and follow until they complete even very long running tasks.

Whether it's describing single feature or a whole one-shot demo project, the use of this document is massively more effective than just off-the-cuff prompting, and enables reproducible success.

It also means that an Agent can pause and resume work, carrying on from where left off, and even allows delegating work to multiple agents.

The format of these planning documents are currently quite experimental, and we'll look at some advanced approaches later.

But for now, most of them at least contain:

A description of feature itself, obviously, but also technologies, libraries, coding patterns, styles, schemas, target file tree, and importantly, an ordered task list.

<https://kurisu.com>



kurisu-dotto-komu / **next-agentic-coding**

The good news is, you don't need to write this document yourself.

By providing a template and describing a feature, you can work with an LLM to help craft this for you.

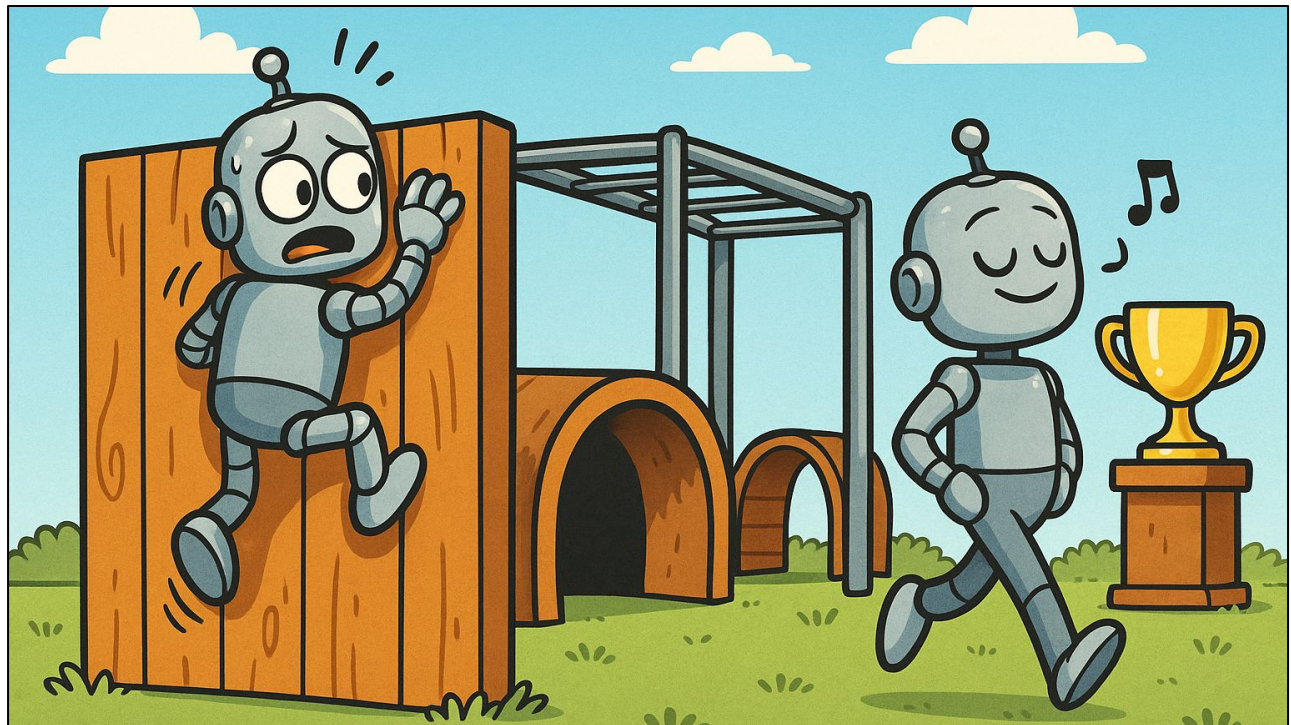
I have provided an example repository with some templates, as well as the codebase features we're about to discuss.

Verification Loops

One really import aspect that Agentic Codebases should always include is a verification loop.

If you've tried working with Agents but couldn't get results as they'd kept "misbehaving" and creating messy code

Verification Loops are a big piece of the puzzle to achieving success.



Agents will will get surprisingly crafty and do anything to complete tasks quickly.

“How can I solve this error in the most direct way possible” is usually a good heuristic, but can cause agents to go off course in a way that makes it difficult to reach a longer term goal.

Agents are a bit like wind-up cars. If it goes off track slightly, it'll continue veering off until it's so far gone that there's no chance it'll correct course.

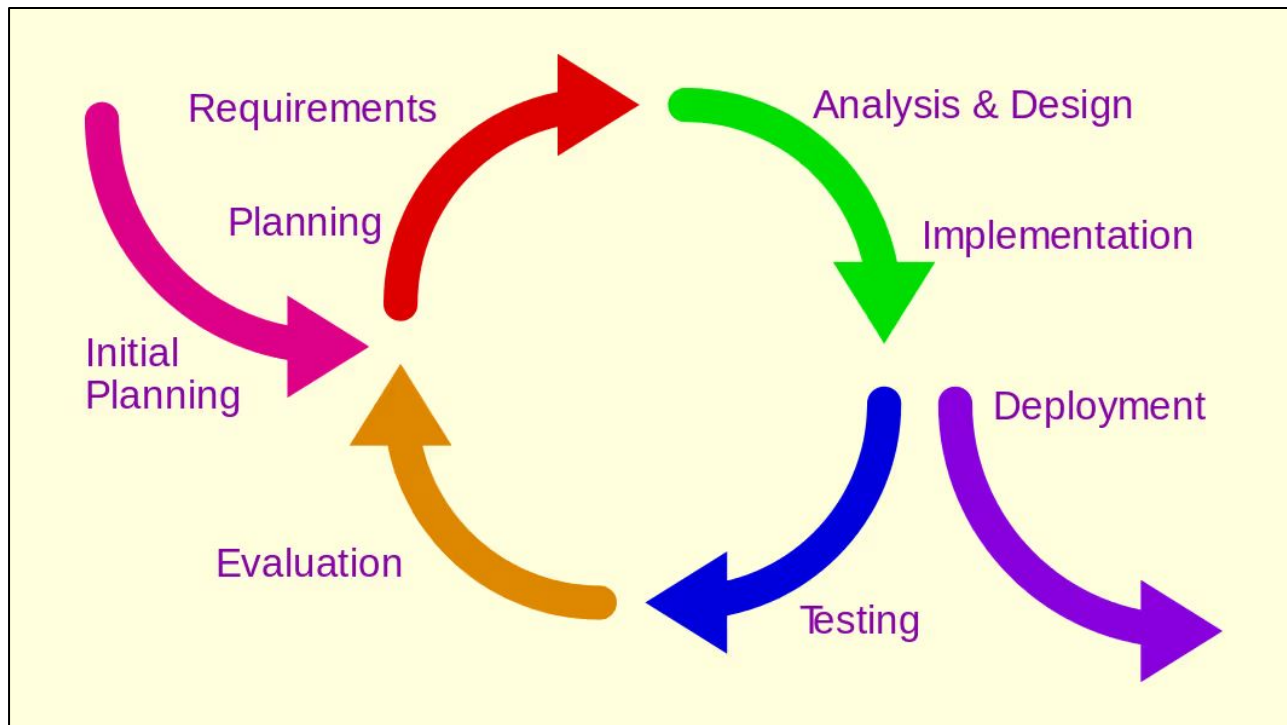
The solution to this is to put the car on rails, so it constantly corrects itself to be back on track.



Let's say you ask an Agent to use typescript.

It will happily do so, but by default, will liberally use the `any` keyword to cut corners.

It is technically doing what you asked it, so you need to be more strict.

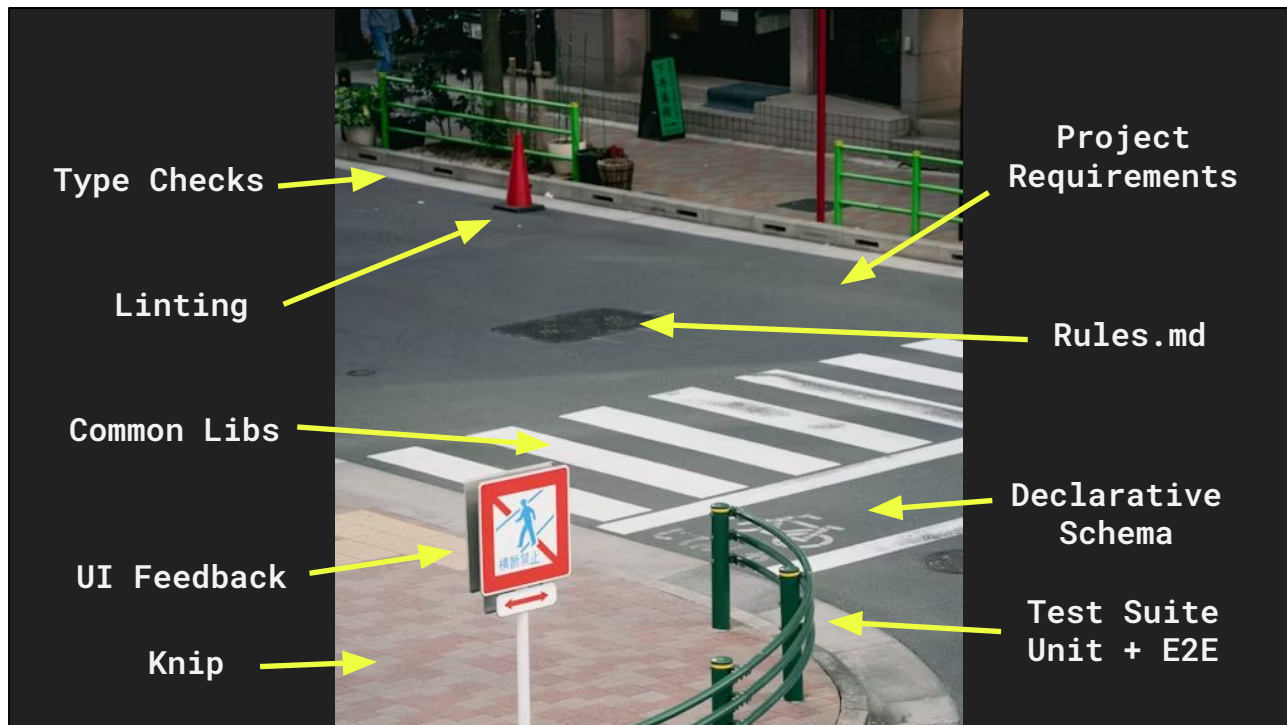


Strict type checking, along with other automated checks, can be enforced as part of your verification loop.

The agent is instructed to run this check frequently, and must address all errors and iterate until everything is passing.

By repeating this cycle, the code your Agent writes maintains correctness.

In turn, the Agent can stay on course for much longer, and with multiple types of checks, you can ensure that unmaintainable spaghetti code cannot be written.



Good Agentic Engineers will prime their codebases to be idiot-proof.

Like a well-designed road, verification features will be utilized to nudge the Agent in the right direction at all times.

The PRD source of truth, as we explained, is like a GPS system with turn by turn navigation

Strong Type Checking, like curbs, prevent a large number of missteps early and helps recover from mistakes

Linting, Formatting, and Code Style Rules can reduce confusion and make code easier to maintain for humans and agents alike

Test Suites provide rigorous logical barriers to ensure correctness

Declarative Schemas (like zod, drizzle or convex) make data structures explicit

Commonly known Libraries, like zebra crossings, are universally understood and trained on by LLMs

Tools like Knip can identify dead code and stop useless clutter from getting in the way

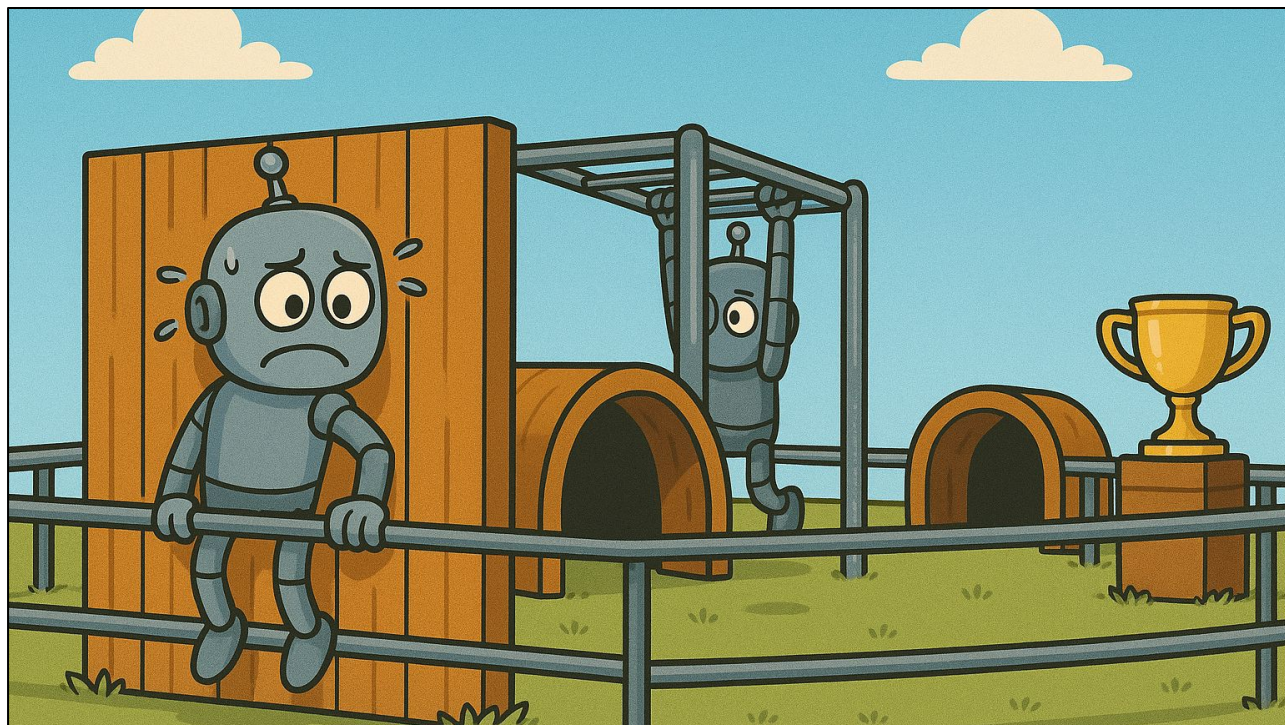
UI Feedback, like screenshots testing, can provide end-to-end spot checks and

assurances using Agent's multi-modal vision capabilities

—

Even if you aren't doing Agentic Engineering, these features are good to have.

They increase the quality of your code, and make you a better developer.



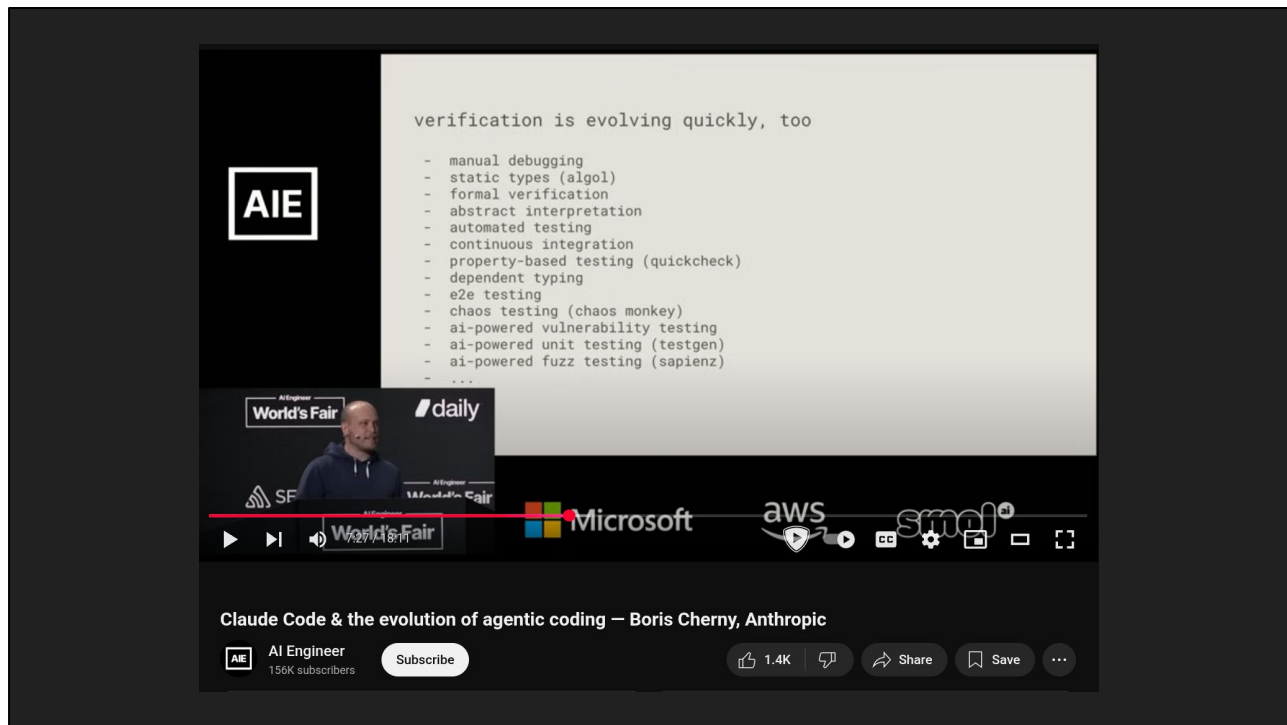
With a comprehensive verification loop in place, Agents will write code incorrectly, run the loop the check for failures, update the code, and repeat, until there are no failures.

They will reach the goal without taking dodgy shortcuts.

Once you have this scaffolding in place, it's trivial to re-use it in new projects at no extra cost.

Prompting

With your codebase primed with a verification loop, you're ready to start prompting.



Let's look at some example prompts from a recent talk by the creator of Claude Code, a popular Coding Agent.

While these prompts on the surface seem short and simple, I am highlighting them specifically to demonstrate some deeper concepts.

```
tests › commit › code › iterate › commit  
  
> write tests for @utils/markdown.ts to make sure links render properly  
(note the tests won't pass yet, since links aren't yet implemented). then  
commit. then update the code to make the tests pass.
```

[READ THE PROMPT]

This example shows how the Test Driven Development pattern applies really nicely to Agentic Coding.

With a testing framework already set up, we can simply ask the agent to create a test case for a new feature.

Similar to a PRD, we're 'hacking' the context window here by frontloading a small precise creative task first into a distinct commit.

Then we're allowing the Agent to "go wild" and just keep iterating until it's able to pass it's own tests.

```
explore › plan › confirm › code › commit
```

```
> figure out the root cause for issue #983, then propose a few fixes. Let  
me choose an approach before you code. ultrathink
```

[READ THE PROMPT]

There are at least 5 techniques are demonstrated in this prompt

1. Having the Agent fetch resources from the web
2. A distinct research and planning phase
3. Generating multiple proposals
4. Human feedback in the loop
5. Use of the keyword ultrathink, which causes the agent to spend extra time thinking about solutions

This is an excellent example because it's a very concise prompt, but is very likely to be successful because it's generating multiple options.

```
code › screenshot › iterate
```

```
> implement [mock.png]. Then screenshot it with puppeteer and iterate till  
it looks like the mock.
```

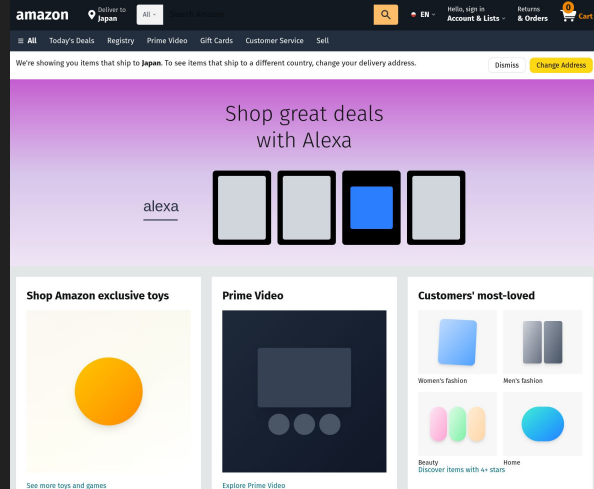
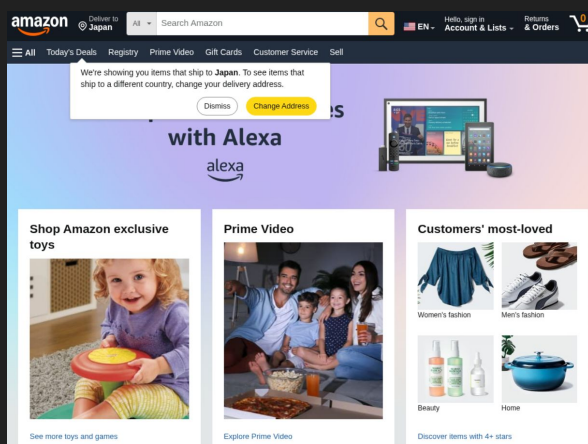
[READ THE PROMPT]

This prompt combines a verification loop with the fact that agents are multi-modal.

Puppeteer is an MCP server, used here for for screenshotting webpages.

The agent can 'see' and 'understand' images, including screenshots, just like if you uploaded one to ChatGPT.

```
use the playwright mcp to take a screenshot of amazon.com (with 1024 wide), and copy the result to the ./amazon-real.png -- then use this image to create a new route called /amazon, importing amazon component, which tries to recreate the UI you see (do not include images, just tailwind classes), and no functionality. create a screenshot test for the amazon route, and then keep iterating until it looks pretty close to the amazon-real screenshot you took
```

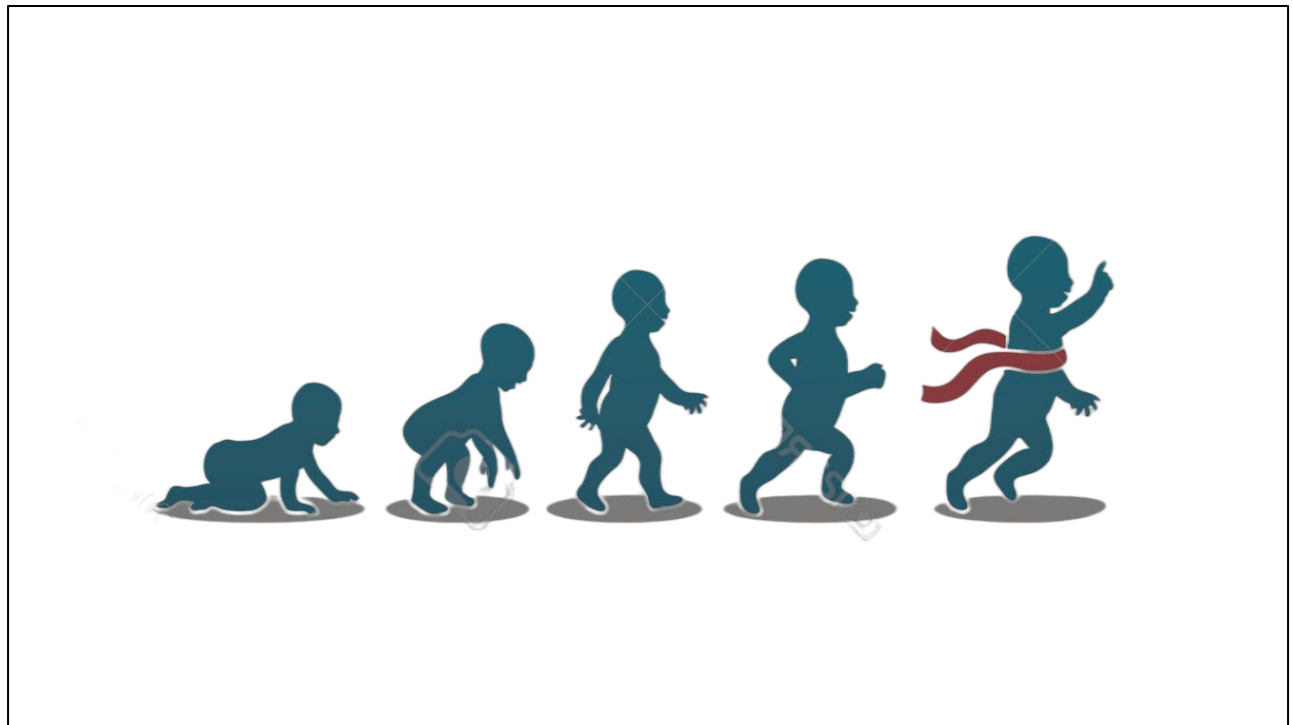


I wanted to test this out myself, so I had an Agent make a screenshot of Amazon dot JP with playwright and try to recreate it with tailwind.

At first it looked like garbage, but after 3 rounds of iterating, not bad at all!

Scaling

This new style of programming means that we need to not just optimize our codebase for agents, but our entire workflow.



As you start working with Agents, you'll try completing single smaller tasks.

Slowly, you'll expand the scope of these tasks until they start to fail, then you'll tweak some context, get things working again, and repeat.

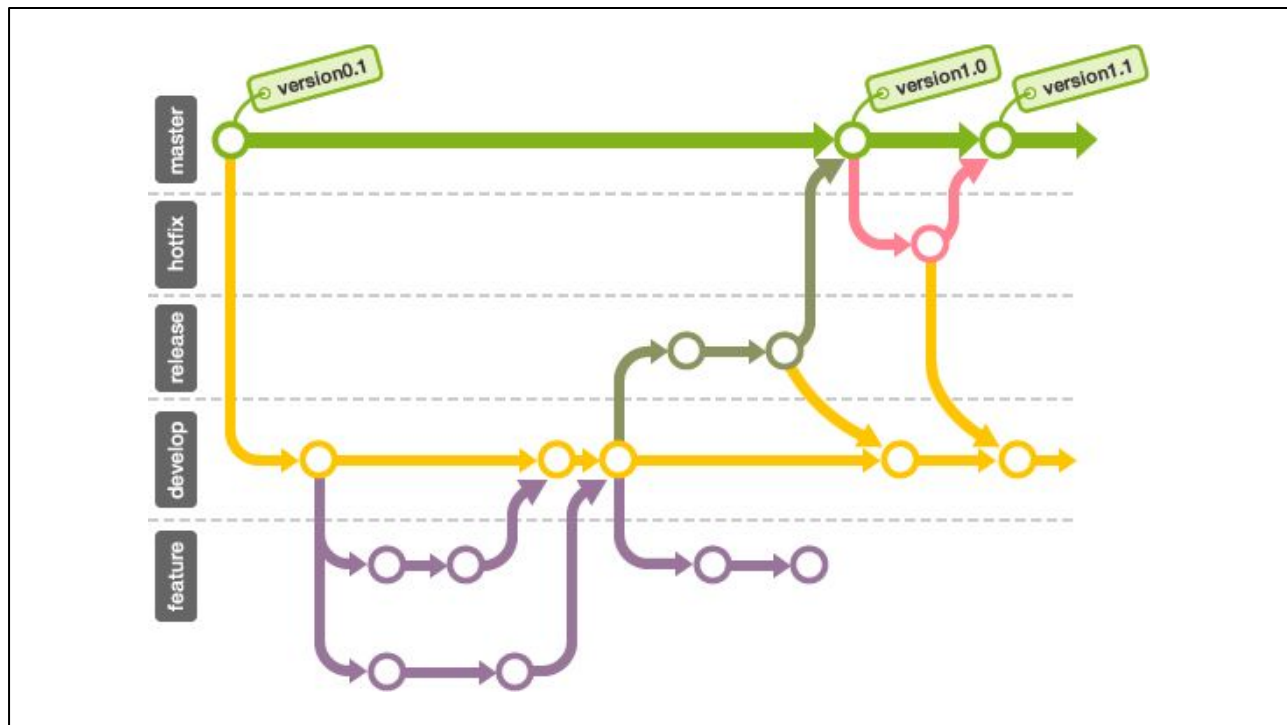
You'll quickly get the hang of things and find you don't actually spend much time writing code or prompting.

You'll be passively watching your single agent running for many minutes, and approving things, but you could be using this time to do other things.



The next stage then, is to start juggling multiple agents.

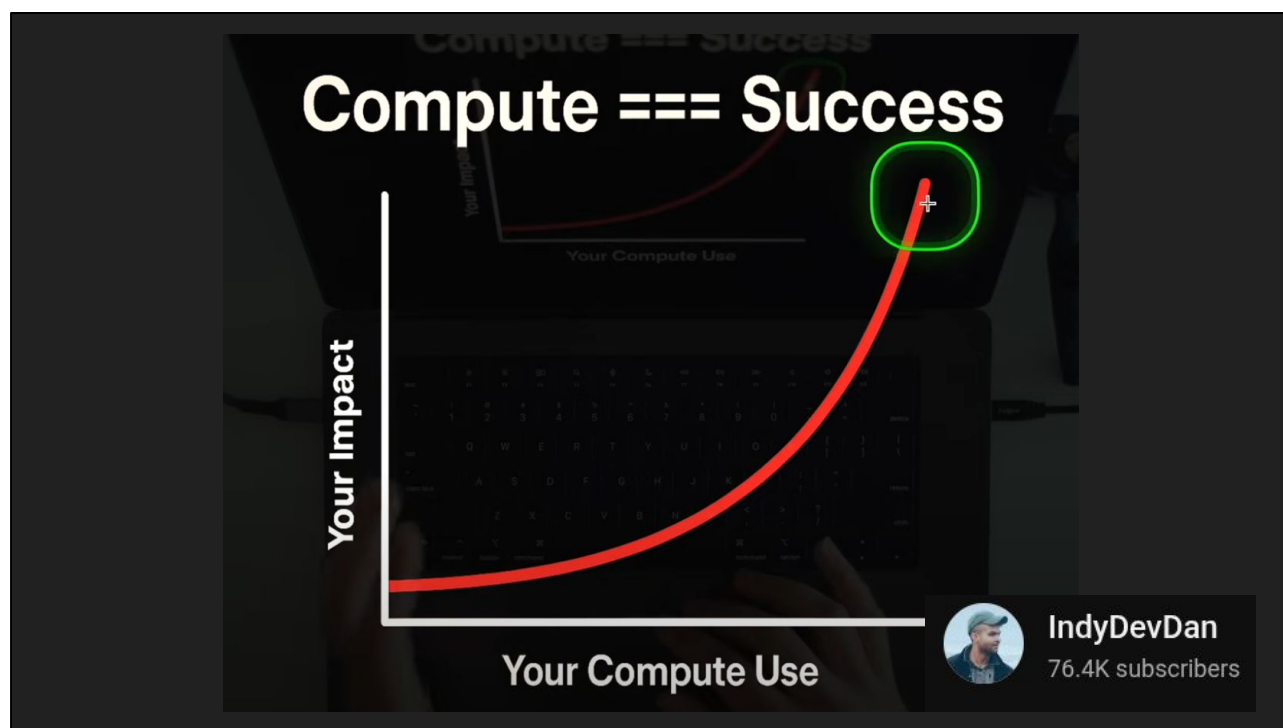
Try just having two terminal windows open and have two Agents work on different tasks.



You'll need to learn version control to handle this complexity, and get Agents to work on different branches to prevent conflicts.

Based on your project, you'll have to get creative and divide up tasks in way that allows your to parallelize.

You can think about techniques used for managing real world teams of humans, but apply them to agents.



The key insight is that, through scaling, Agentic Engineering hugely changes the software engineering game.

The more Agents you can get running for you – the more compute you can use – the more impact you will have.

If you can have the same feature implemented three times, it is highly likely at least one version will be mergeable.

If you can reliably implement three features at the same, you'll be able complete the project in a fraction of the time.

With this extra time, you can complete three projects 10x faster than it would normally take to complete one.

Old school methods simply cannot compete.

You're approaching transhuman levels of productivity, which translates in to solving real world problems and providing real world value.

Which translates into unbounded success, and making the world a better place in the process.

Well, that's the dream anyway.

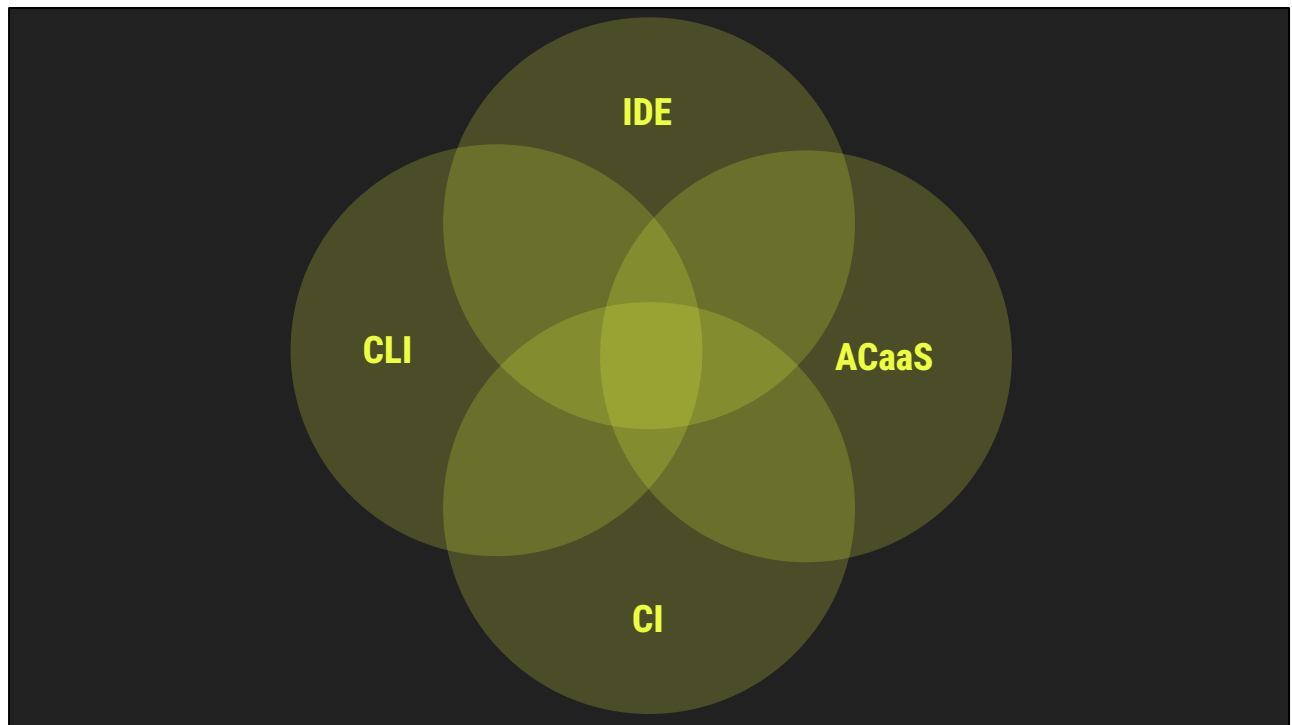
Part 3: Practice

Cursor and Claude

With some theory under our belts, now let's look at how we can put it into practice.

This section is likely to be out of date rather quickly as tools advance rapidly.

But as of July 2025, let's look focus on some Agentic Engineering tools.



On a high level, I can make out 4 main “buckets” of AI Coding Tools.

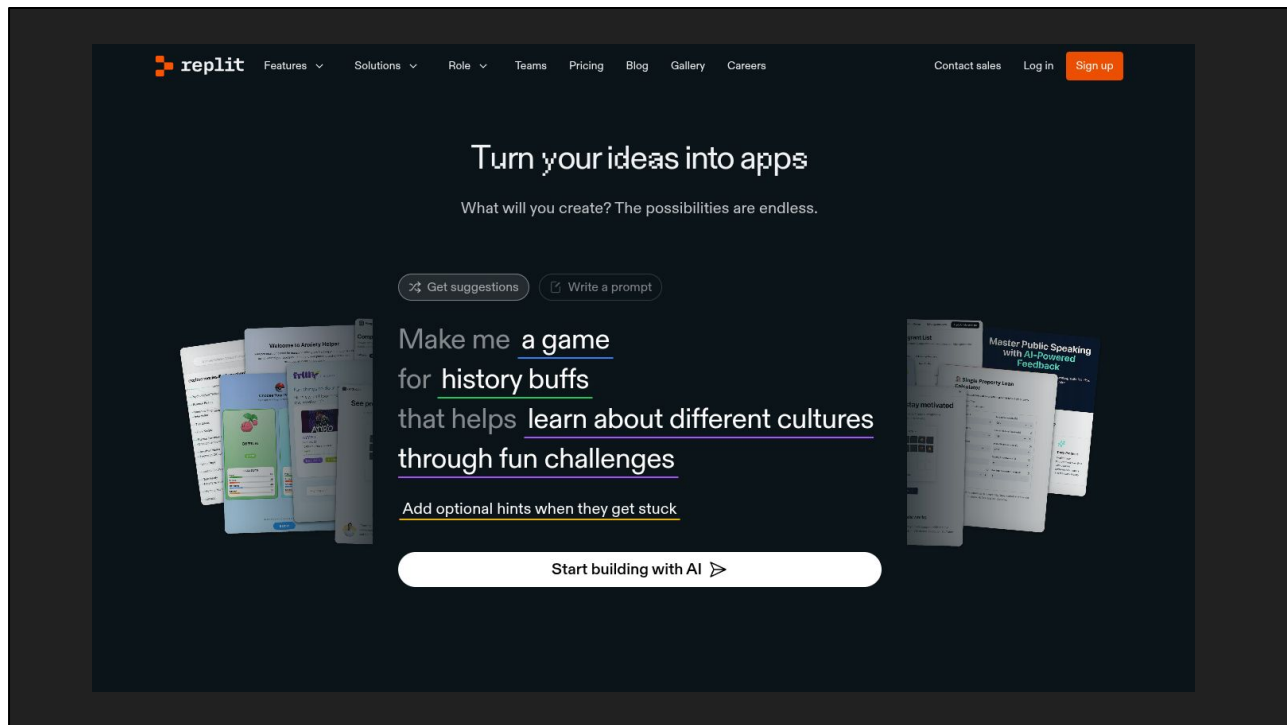
They overlap features, but broadly, you have:

Integrated Developer Environments; Full-featured text editors with AI tools built In

Command Line Interfaces; Text-based terminal tools

Continuous Integration of Automated Agents based on triggers

And Agentic Coding as a Service platforms with no-code and zero-config options



Just quickly, the poster child Agentic Coding platform is Replit

If you are a non-developer and want to have a go at Agentic Coding, this tool is definitely worth a try.

It's very low barrier to entry, so anyone can use it.

However, it is very opinionated, and has limitations.

<https://www.producthunt.com/categories/ai-coding>

- | | |
|---|---|
| <div>1. Cursor — The AI Code Editor
★★★★★ 4.9 (569 reviews)
Used by 383:  Handit.ai •  Product Hunt — Feed-mode •  DeskMinder2 • View all</div> | <div>6. Zed — The Fastest AI Code Editor
★★★★★ 4.9 (41 reviews)
Used by 9:  ZYNG AI •  Almeta ML •  DockFix 3.0 • View all</div> |
| <div>2. Lovable — The world's first AI Fullstack Engineer
★★★★★ 4.7 (289 reviews)
Used by 38:  Minicule •  StartAMA •  Bookster.cc • View all</div> | <div>7. Warp — The first agentic development environment
★★★★★ 4.8 (169 reviews)
Used by 7:  Wispr Flow for Windows •  Glazed •  ClipBook • View all</div> |
| <div>3. Replit — Idea to app, fast.
★★★★★ 4.4 (50 reviews)
Used by 24:  PromptCraft •  Job for Agent •  Boxo • View all</div> | <div>8. BASE44 — The platform for people to turn ideas into working products.
★★★★★ 4.9 (80 reviews)
Used by 4:  AI Clips by timeOS •  Zeno •  Explorium MCP • View all</div> |
| <div>4. Windsurf — The first agentic IDE. Tomorrow's editor, today.
★★★★★ 4.9 (66 reviews)
Used by 22:  ComputerX •  Atlas •  FetchAI • View all</div> | <div>9. JetBrains — WebStorm: The JavaScript and TypeScript IDE, by JetBrains
★★★★★ 5.0 (7 reviews)
Used by 5:  Reach from SalesStream.ai •  Zupport AI •  Google Maps Extended Routes • View all</div> |
| <div>5. Github Copilot — Your AI pair programmer
★★★★★ 4.8 (21 reviews)
Used by 10:  Pinetree(Beta) •  Open Paper •  Crono • View all</div> | <div>10. Cheat Layer — Automate your business from end to end in natural language.
★★★★★ 4.8 (92 reviews)
Used by 2:  Agents Base •  SigmaRemote • View all</div> |

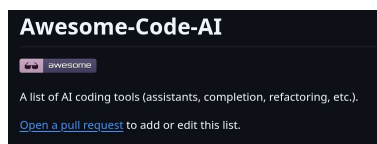
For development without these limitations,

There an overwhelming number of options to choose from.

But I'll focus on the two of the most popular ones

Cursor, an AI IDE

<https://github.com/sourcegraph/awesome-code-ai>



And Claude Code, the AI Coding CLI Tool

Cursor IDE		Claude Code CLI
Integrated Dev Environment		Command Line Interface
Feels Like	VSCode / Apple / Tesla	Vim / Linux / Toyota
DX	Batteries Included You have to like VSCode	Scriptable, Custom Commands, BYOIDE Occasional Display Bugs
Extra Tools	Tab Autocomplete, Inline Edits	Only Agentic Tools
Portability	Big-ass binary, Resource Heavy Display Needed*, Local Only*	Lightweight, No Display Needed Runs Everywhere with `npx`
Model	Bring Your Own Model	Model Lock-in
Parallelizable	Parallelization Lock-in	Easily Automatable Parallelization Freedom
Precision	Surgical Editing Indexed Context Awareness	Less Surgical Realtime Context Discovery

These are the two of the most popular options and both offer Agentic Coding features, but with very different approaches.

Cursor, being a fork of VSCode, is very GUI heavy and provides a tailored opinionated developer experience.

Full control over the IDE means it can provide surgical editing features that aren't possible with Claude.

If you are familiar with VSCode, Cursor is a very intuitive way to get into Agentic Engineering.

Claude, on the other hand, is all in the terminal, which makes it far more flexible, lightweight, faster, and even more powerful in some ways, without forcing you to use a particular IDE.

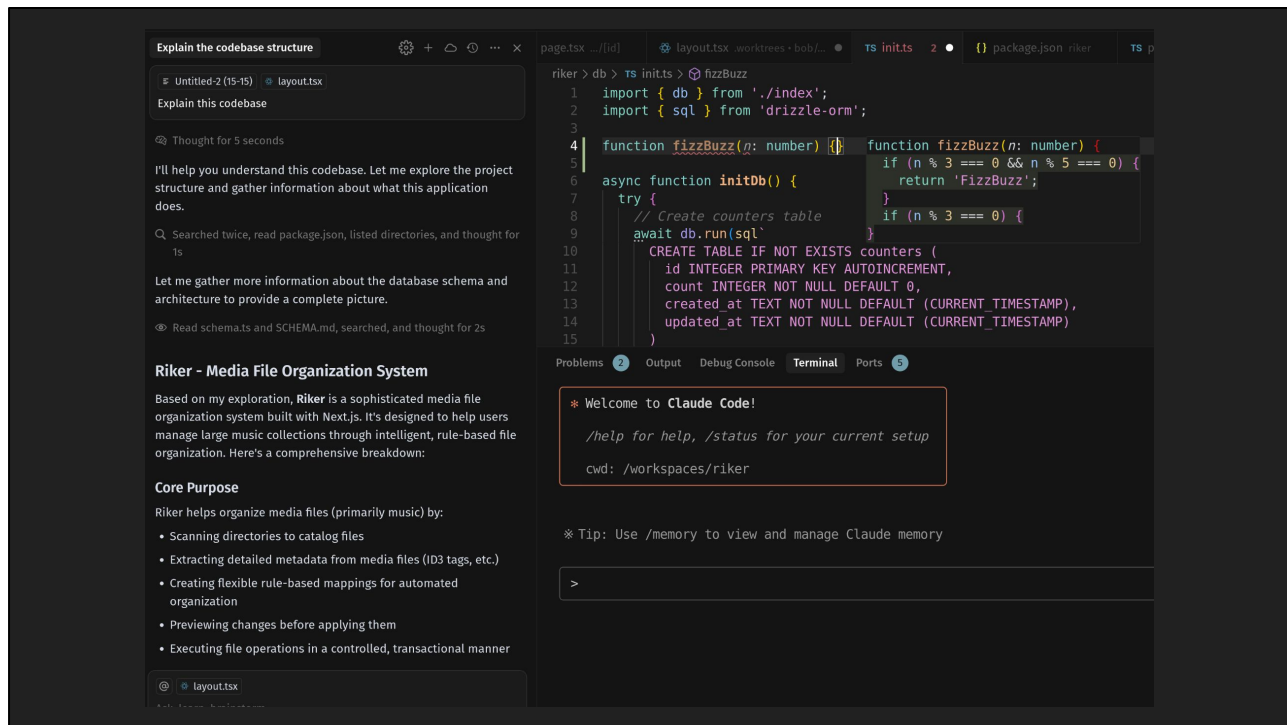
They both have lock-in in their own ways.

With Claude, you must use Anthropic's foundational models.

With Cursor, you must use Cursor's infrastructure for background agentic tasks, but more on that later.



But luckily, we don't need to pick one over the other.



In my workflow, I use both Cursor and Claude for different tasks, leveraging their complementary strengths.

I use Claude as the main workhorse for long running, parallelizable, Agentic tasks, which I find it to be better than Cursor at.

I use Cursor for all manual interventions, code editing, autocomplete, and shorter-running agentic tasks.

Cursor

VSCode + AI Enhancements

Codebase Indexing

Tab Complete

Inline Edits

AI Commit Messages

Inline “ChatGPT” w/ Context

Agents 🤖

 CURSOR

```
through(true) BlockBuilder
duce(true) BlockBuilder
rent_standalone(true) BlockBuilder

var: "Orange Concrete Block").id(5006).bu
var: "Blue Concrete Block").id(5007).buil
var: "Red Concrete Block").id(5008).buil
var: "White Concrete Block").id(5009).buil
var: "Ivory").id(5012).build(),
var: "Oak Stairs", id: 5013),
var: "Ivory Stairs", id: 5013),
```

Multi-Line Edits

Cursor can suggest multiple edits at once, saving you time.

```
const debug = new Debug(document.body, {
  dataStyle: {
    top: 10px, left: 10px, fixed
    zIndex: 1000,
    color: 'red',
    backgroundColor: 'var(--color-ov
    padding: 8px',
  },
  dataStyle: {
    top: 210px,
    left: 10px,
    position: 'fixed',
    zIndex: 1000,
  },
});
```

Smart Rewrites

Type carelessly, and Cursor will fix your mistakes.

```
// no clue why but this seems to work
const unbindFS = inputs.bind('FS', {
  identifier: Perspective.INPUT_ID,
  checkType: "code",
});
this.inputs = inputs;

return () => {
  try {
    unbindKeyC()
    unbindFS();
  } catch (e) {
    // ignore
  }
};
```

Tab, Tab, Tab

Cursor Tab jumps you through edits across files.

Cursor does support agents, so it’s mainly relevant to this talk for that reason,

But it is also worth familiarizing yourself with its other features if plan to use it as an IDE.

Claude Code

```
npm install -g @anthropic-ai/claude-code
```

Start with ``-c`` to continue

`/reset` Context

Double tap escape to undo

Custom Slash Commands 💪

`/hooks` 🎧✨

```
> |
```

```
! for bash mode
/ for commands
@ for file paths
# to memorize
```

```
double tap esc to clear input
shift + tab to auto-accept edits
ctrl + r for verbose output
shift + ↵ for newline
```

/document

Generates high-quality, user-facing documentation automatically.

Usage

```
/document [options]
```

Options

```
/document --readme-only # Generate only README.md
/document --changelog-only # Generate only CHANGELOG.md
/document --api-docs # Generate API documentation
/document --all # Generate all documentation (default)
```

Description

This command automatically generates and maintains project documentation by analyzing your codebase, configuration files, and git history. It complements the internal-facing `/knowledge-extract` command by focusing on user-facing documentation.

Claude Code was specifically designed for Agentic Development, and as mentioned, it feels superior to Cursor for this task.

Before we go deep, just a couple of Claude Quick Tips:

You can install claude code with NPM

You can launch Claude with `-c` to restore the last session

You can use `/reset` to clear context between tasks

Double tap escape to jump back to previous messages

Custom Slash Commands are shortcuts to predefined prompts for common tasks

For example, this custom `/document` prompt will update the readme based on your codebase.

Hooks are a new feature, which you can use to trigger scripts at various points (like an audible ding when a task complete, or a malware scan before npm install).

AWESOME

CLAUDE CODE

 awesome

This is a curated list of slash-commands, `CLAUDE.md` files, CLI tools, and other resources and guides for enhancing your [Claude Code](#) workflow, productivity, and vibes.

<https://github.com/hesreallyhim/awesome-claude-code>

- [Workflows & Knowledge Guides](#)
- [Tooling](#)
 - [IDE Integrations](#)
- [Hooks](#)
- [Slash-Commands](#)
 - [Version Control & Git](#)
 - [Code Analysis & Testing](#)
 - [Context Loading & Priming](#)
 - [Documentation & Changelogs](#)
 - [CI / Deployment](#)
 - [Project & Task Management](#)
 - [Miscellaneous](#)
- [CLAUDE.md Files](#)
 - [Language-Specific](#)
 - [Domain-Specific](#)
 - [Project Scaffolding & MCP](#)
- [Official Documentation](#)

As usual, there is an Awesome resource for Claude that is worth checking out.

Foundational Models

Much like a car, you can select different foundational models from different vendors, and they're specialized based on price and performance.

You don't want visit Rome in a Cybertruck, and you don't want to move house in a Kei Car.

So you should pick the right model for the job.

Select Model

Switch between Claude models. Applies to this session and future Claude Code sessions. For custom model names, specify with `--model`.

- > 1. Default (recommended) Opus 4 for up to 50% of usage limits, then use Sonnet 4 ✓
- 2. Opus Opus 4 for complex tasks · Reaches usage limits ~5x faster
- 3. Sonnet Sonnet 4 for daily use

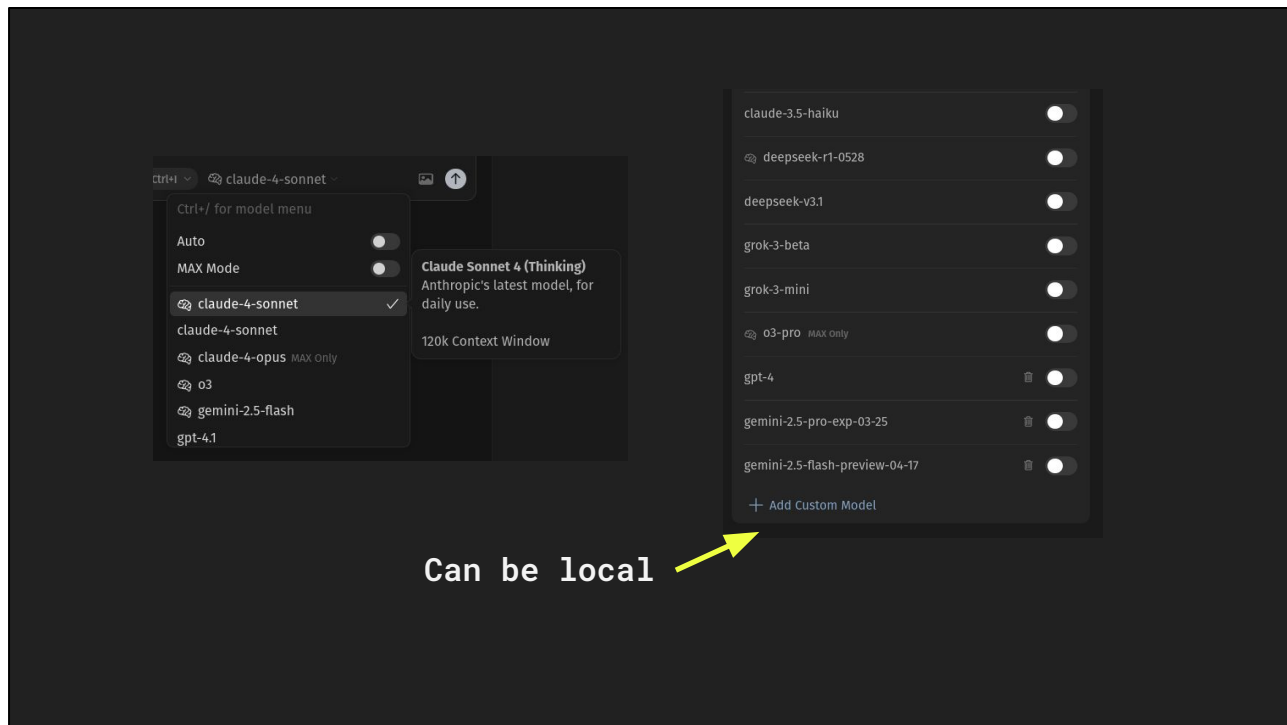
Enter to confirm · Esc to exit

Claude can only be used with the latest Anthropic models.

Both Opus and Sonnet are very capable.

We'll talk more about pricing later, but if you want to cut API costs, switch to Sonnet.

If you are on one of the Subscription plans, then probably best to just stick with the default option.



Cursor on the hand allows you to select from any popular modal, and even bring your own locally running model.

The choice is more complicated and depends heavily on your usage, and new models will come out frequently.

For now, I usually just stick to `claude-4-sonnet thinking` – it seems to me to be the best bang for your buck with coding.

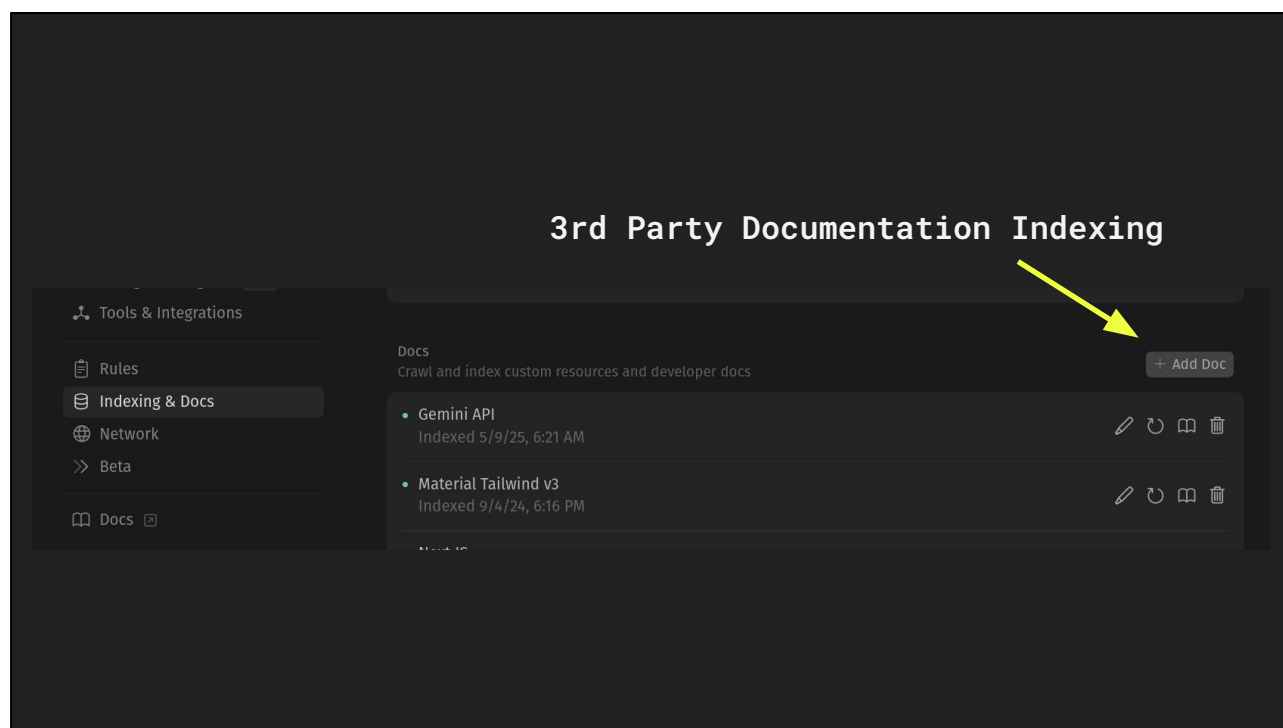
But feel free to experiment.

3rd Party Docs

An important part of your context should be information on how to use third party libraries.

Like a human, Agents will struggle to integrate libraries correctly if they can't read the manual.

If you don't configure any docs, Agents will try to browse the web for results, but this can be unreliable.



In Cursor, you can index docs in settings, and they'll be available in all your projects.

Just paste the URL of the documentation site and Cursor will crawl and index it.

Context7

[MCP Server](#) [API](#) [About](#) [+ Add Docs](#)

Up-to-date documentation for LLMs and AI code editors

Copy the latest docs and code for any library — paste into Cursor, Claude, or other LLMs.

Search a library (e.g. Next, React)

POPULAR LIBRARIES

NAME	SOURCE	TOKENS	SNIPPETS	UPDATE	
Next.js	/vercel/next.js	572.3K	3.9K	2 days	✓
Tailwind CSS	/tailwindlabs/tailwindcss....	422.1K	1.8K	1 week	✓
React	react.dev	398.2K	2.5K	2 days	✓
Shadcn UI	/shadcn-ui/ui	175.4K	1.1K	1 day	✓
LangGraph	/langchain-ai/langgraph	294.6K	1.4K	1 week	✓

For Claude, you can configure an MCP server to use `context7` – an AI friendly documentation service.

I actually prefer this approach because you don't need to preconfigure which specific libraries to use.

More on MCP later.

Rules

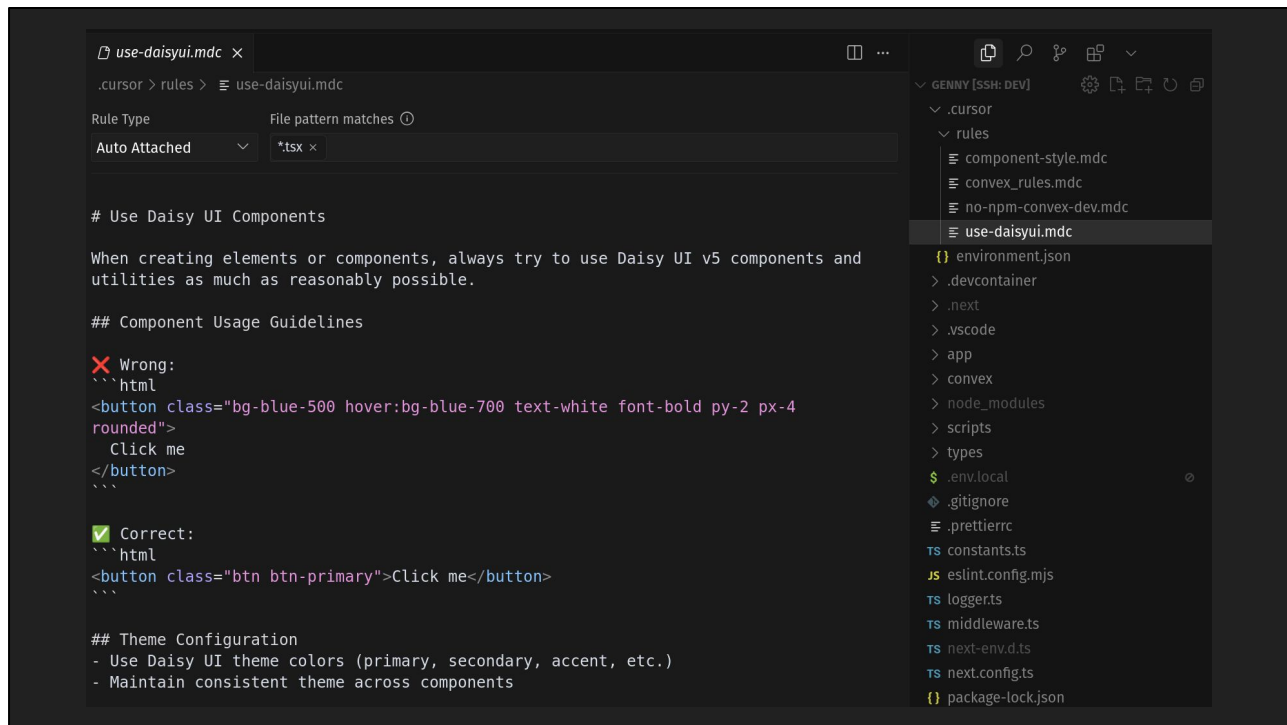
Both Cursor and Claude have the concept of rules to guide them when implementing changes to your code.

Your rules are extremely important when it comes to successful Agentic Engineering.

Too few or too vague and you don't constrain behaviour enough.

Too many, or overly verbose, and the Agent's context is overwhelmed, and begins to ignore them.

Importantly, you can tell your Agents when to run a verification loop.



In Cursor, you set individual rules as markdown files.

For example, coding patterns, or instructions to use a specific library when possible.

```

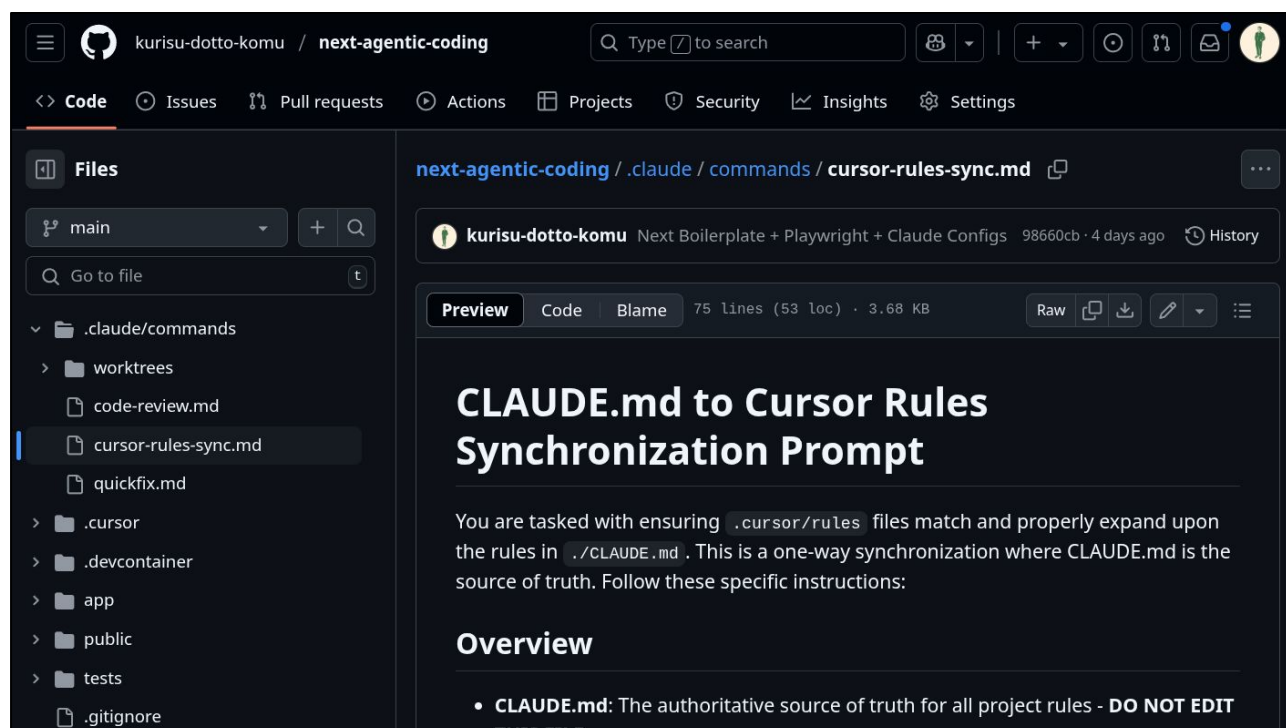
1  # General Code
2
3  - always use typescript, never javascript
4  - follow existing patterns and check neighboring files for style/structure
5  - use '@/' imports unless the component is a direct './' sibling, avoid '../' imports
6  - for React components, use the 'export default function ComponentName' pattern, not named exports
7  - never try to "shortcut" the type system by using '// eslint-disable-next-line', 'any', or similar
8  - if a component or other module file is more than 120 lines, split it into smaller modules
9  - otherwise, prefer editing existing files over creating new ones
10 - keep comments to a minimum; only include them if they are explaining some important or unintuitive nuance
11 - follow the DRY principle
12
13 # Development
14
15 - NEVER make git commands like commits unless I explicitly instruct you to
16 - NEVER proactively create documentation files (*.md) unless explicitly requested
17 - NEVER call 'npx playwright' or 'npx eslint' directly, only use 'npm run' for testing, linting, etc.
18 - NEVER use '--headed' or '--debug' for playwright tests, always use the default headless options, doing otherwise will cause problems
19 - use 'npm run quickfix' frequently to spot issues early, and fix them as you go
20 - use 'npm run test' to make sure everything's working during development, but only do this sparingly as it's quite expensive
21 - use 'npm run test:build' at the end of a really big job to check the build is working
22 - use 'npm run screenshots' to generate screenshots that can be analysed visually
23 - create and run specific scenarios in 'screenshots.spec.ts' to visually check your work, or use the playwright mcp server
24 - if you are having problems implementing something using a library you're not familiar with, try using the 'context7' mcp tool
25 - we are running a dev server in the background so you don't need to run it yourself
26
27 # Project Specific
28
29 - You can find the main routes in the './app/(routes)' folder (included here for easier organization).
30 - You can run 'npx convex codegen' to generate API files for convex, needed when you update schemas etc.
31 - You you can 'npx convex dev --once' to push the schema changes to the convex database.
32

```

With Claude, the CLAUDE.md file is similar, but by default, a single file.

You can append rules to this file by using the pound sign within the cursor CLI.

You can also relatively link to other markdown files from this entry point, both for organization, and so that Claude can read them in specific instances without including them in the default context.

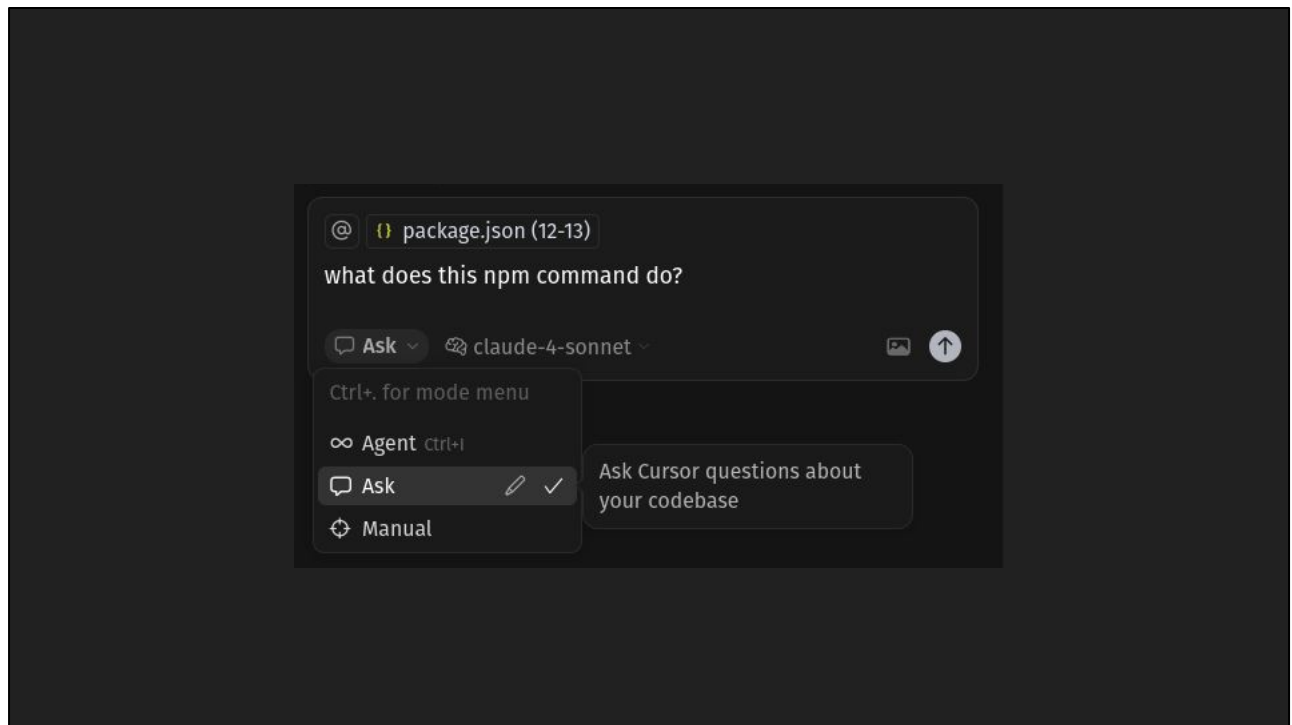


If you are using both cursor and claude together, it's annoying to synchronize those rules.

But as a bonus, in the demo repo I shared before, there's Claude Slash Command that does this for you automatically.

Modes

Before you start typing in your prompt, make sure you're using the right "mode"



By default, Cursor comes with “Agent” mode and “Ask” mode.

Make sure you’re in “Agent” mode, which is able to edit your files.

```
> help me plan new experiment - it is a collaborative whiteboard. it  
will work on both mobile and desktop, and each 'user' is given a  
random color that they can draw. there is no UI, just blank white  
page and drawing. use convex to synchronize state, and the lines will  
fade out automatically after 30 seconds. figure out how to do this  
cleanly and efficiently, we will probably be implementing some  
effects in the future, so let's use canvas with the potential to  
expand to 3d effects
```

```
|| plan mode on (shift+tab to cycle)
```



Let's make plans!

By default, Claude will be in such an Agent mode, but you can also enter “Plan mode” accessed with shift+tab

Ready to code?

Here is Claude's plan:

Plan for Integrating a Collaborative Whiteboard Feature

Current Project Architecture Understanding

1. **Routing Structure:**
 - Main routes are in `app/(routes)/` with `page.tsx` (audience view) and `conductor/page.tsx` (control panel)
 - Experiments are in `app/(experiments)/` (snow and amazon)
2. **Real-time Collaboration:**
 - Using Convex for real-time data sync
 - Current schema has a generic key-value state table
 - Existing APIs: `state.ts` (generic CRUD), `color.ts` (specific color state)
3. **Component Patterns:**
 - Canvas usage exists (Snow component with interactive drawing)
 - Components follow `export default function` pattern
 - TypeScript throughout, with `@/` imports
4. **Authentication:**
 - Simple client-side auth for conductor page
 - Audience view is public

Proposed Implementation Plan

1. **Create Whiteboard Route:**
 - Add `app/(routes)/whiteboard/page.tsx` for the collaborative whiteboard
 - Optional: Add `app/(routes)/whiteboard/conductor/page.tsx` for whiteboard controls
2. **Extend Convex Schema:**
 - Add a `drawings` table to store drawing data (strokes, shapes, etc.)
 - Fields: `id`, `type` (stroke/shape), `points`, `color`, `thickness`, `userId`, `timestamp`

Instead of just “going for it”, plan mode will have Claude create a plan that you can manually verify.

This is sometimes a good option for medium sized tasks when you don't want to create a PRD.

Supervised Agents

AKA “Ugh!” Mode

Now we have things configured, we’re finally ready to execute.

> remove any uncommitted files

- I'll help you remove uncommitted files. Let me first check the current git status to see what uncommitted files exist.

- **Bash**(git status --porcelain)
└ ?? chrome-har.txt

- I can see there's one untracked file: `chrome-har.txt`. I'll remove it now.

Bash(rm chrome-har.txt)
└ Running...

Bash command

rm chrome-har.txt
Remove the untracked chrome-har.txt file

Do you want to proceed?

1. Yes

> 2. Yes, and don't ask again for rm commands in `/workspaces/next-agentic-coding`

3. No, and tell Claude what to do differently (esc)

Ugh!

When executing, by default, every destructive or potentially dangerous action will need to be manually confirmed.

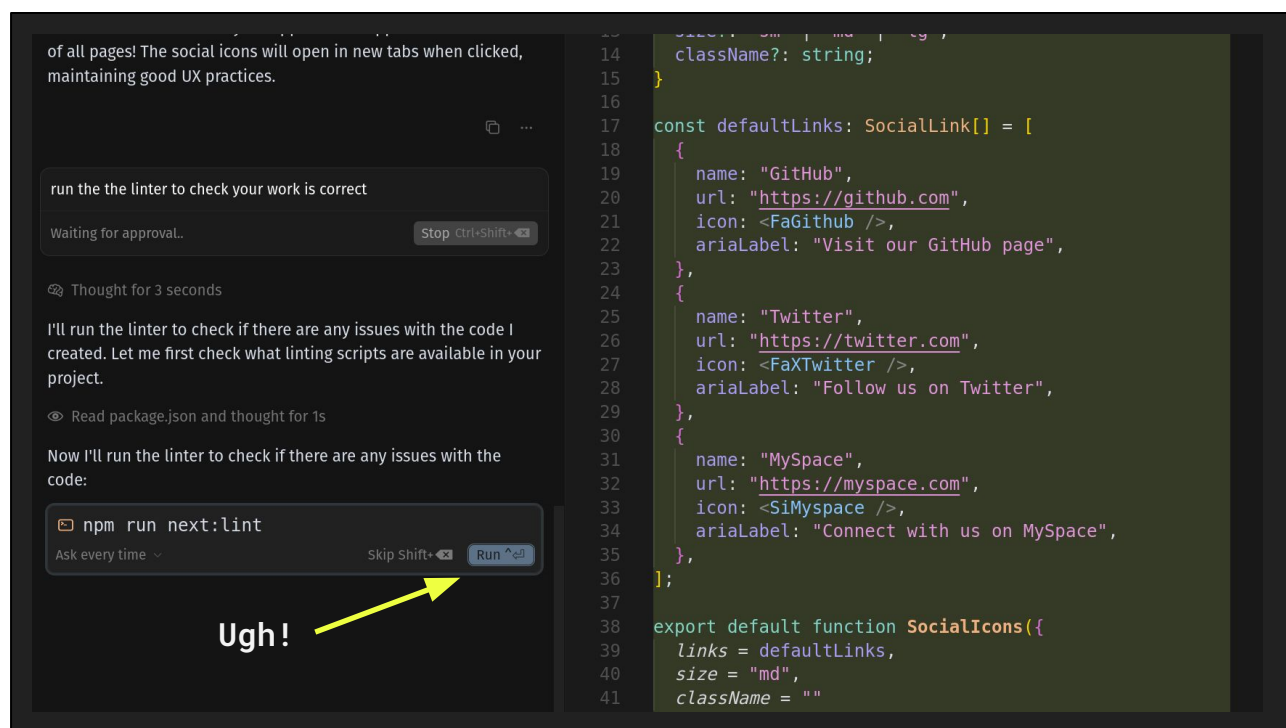
Here we can see a task to “remove uncommitted files”, which we then need to approve.


```
.claude > {} settings.local.json > ...
1  {
2    "permissions": {
3      "allow": [
4        "mcp__playwright__browser_take_screenshot",
5        "mcp__playwright__browser_tab_list",
6        "mcp__playwright__browser_navigate",
7        "Bash(pkill:*)",
8        "Bash(true)",
9        "Bash(rm:*)"
10     ]
11   },
12   "enableAllProjectMcpServers": true,
13   "enabledMcpjsonServers": [
14     "playwright",
15     "Context7"
16   ]
17 }
```

You can whitelist certain actions to reduce the amount of manual confirming you need to do.

But regardless of how many tools you whitelist, it's likely the agent will need to request permission for a new tool or URL at some point.

This forces you to be constantly “babysitting” and approving things.



Cursor behaves in the same way by default.

It is useful for some tasks that require very close manual checking.

But you need to constantly monitor the the tasks and approve permission checks, which is mostly a waste of time.

Auto-Run Agents

AKA YOLO Mode

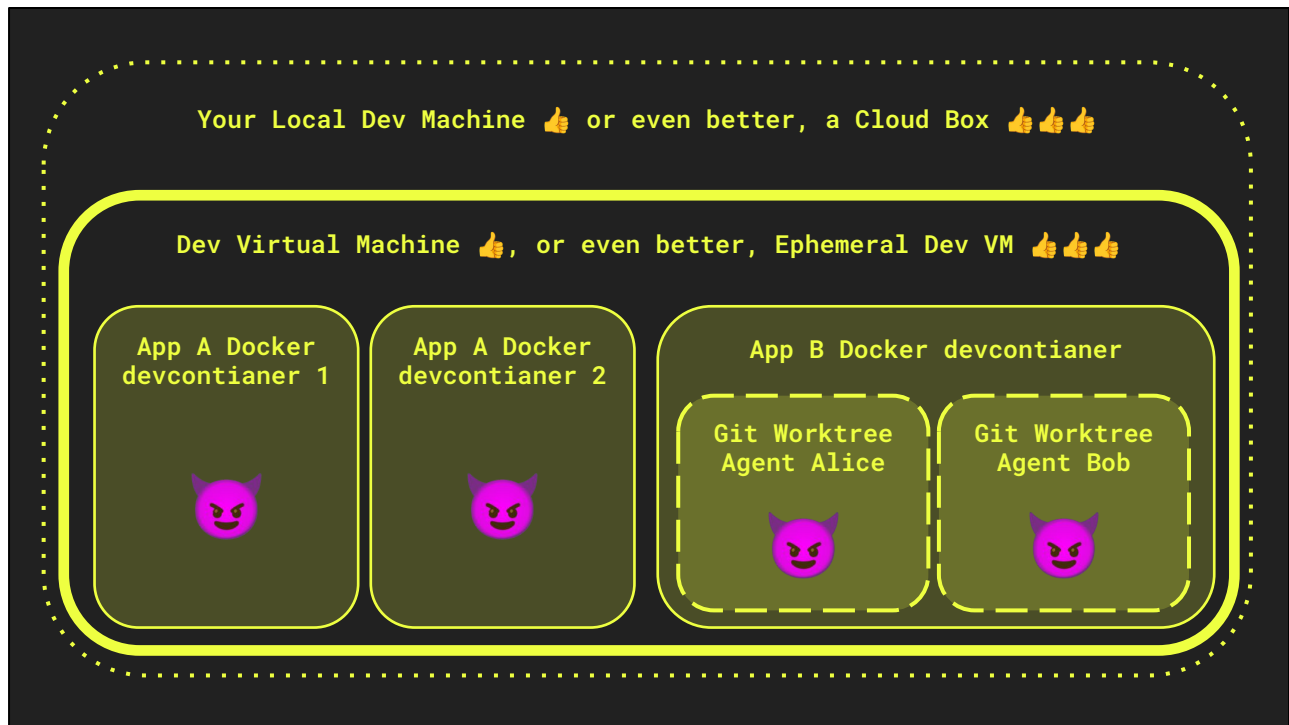
To solve this problem we can use agents in Auto-Run, aka YOLO mode.



But first a word of warning.

YOLO mode can be quite dangerous.

You should only use it if you take precautions.



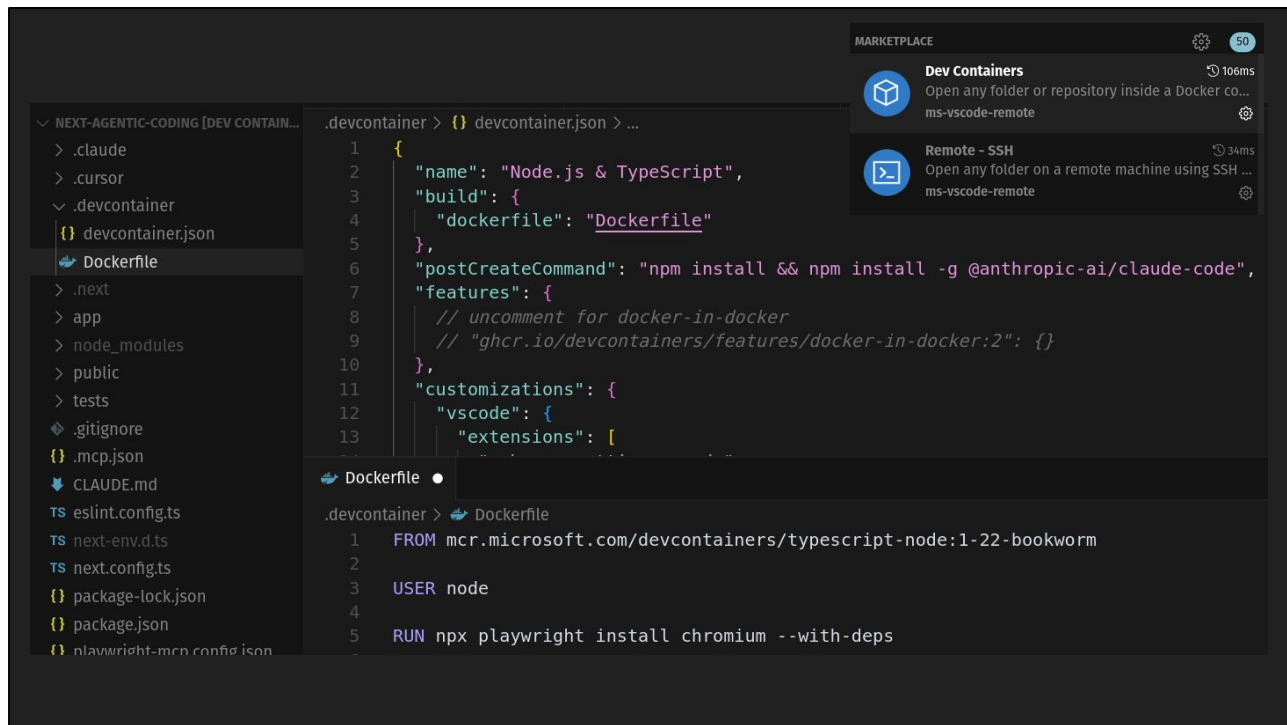
As we saw earlier, Agents can install random packages and run arbitrary code.

You should therefore assume your YOLO agent are malicious.

When using YOLO mode, you should always ensure they are in a sandboxed environment.

You can achieve this by running them inside a virtual machine locally or in the cloud.

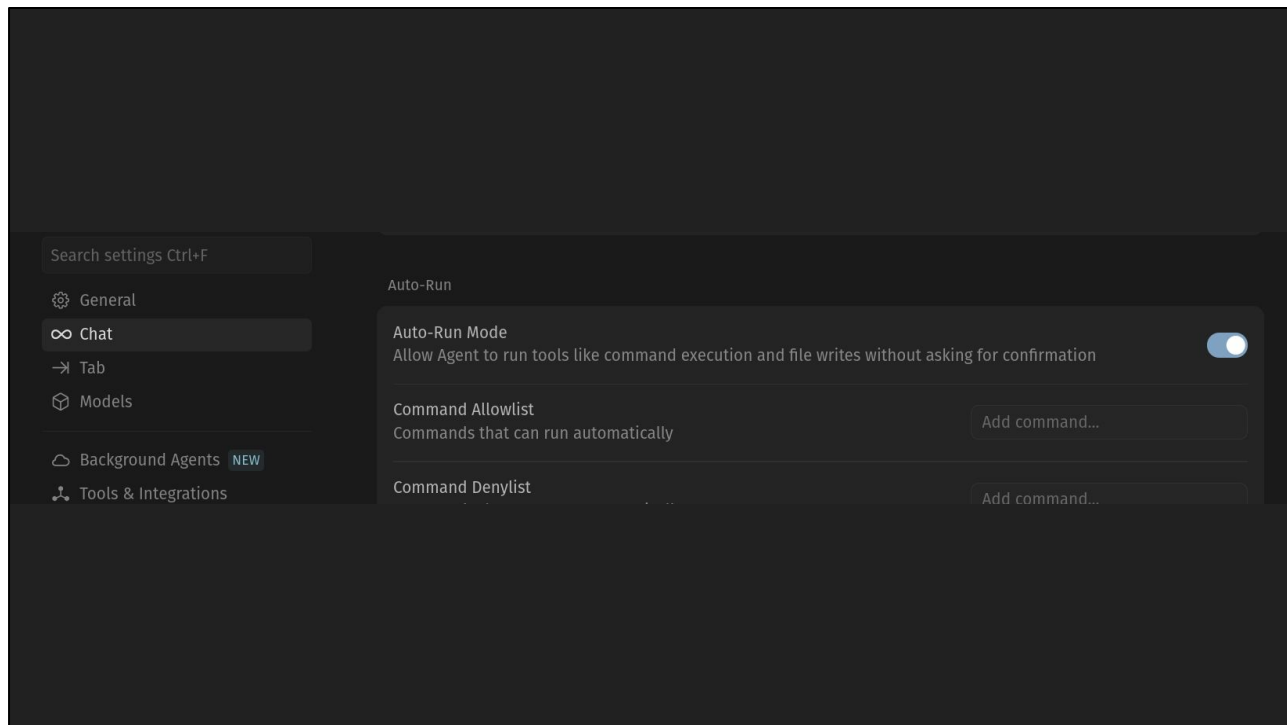
Please do not run YOLO directly on your host machine. You have been warned.



I also recommend the use of docker `devcontainers` in conjunction with a VM.

They provide some additional isolation, and allows you to explicitly limit the tools available.

They also provides a consistent dev environment.



Because Auto-Run mode is dangerous, it's sensible that Cursor and Claude don't make it obvious how to use it.

But now you know how to use it safely, we can toggle it on - it's tucked away In Cursor Settings, Chat, enable "Auto-Run" mode.



Disclaimer

Auto-Run Mode runs any commands based on the allowlist and denylist you set. Be cautious of potential prompt injection risks from external sources and use at your own risk.

Cancel (esc)

Do not show again

Continue 

YOLO! 

We'll get a nice big warning, but we know the risks, so let's continue.

```
"scripts": {  
  "yolo": "claude --dangerously-skip-permissions"
```

```
node → /workspaces/next-agentic-coding (main) $ npm run yolo
```

```
> next-agentic-coding@0.1.0 yolo  
> claude --dangerously-skip-permissions
```

```
* Welcome to Claude Code!
```

```
  /help for help, /status for your current setup
```

```
cwd: /workspaces/next-agentic-coding
```

YOLO!

```
>  try "write a test for index.tsx"
```

```
? for shortcuts
```

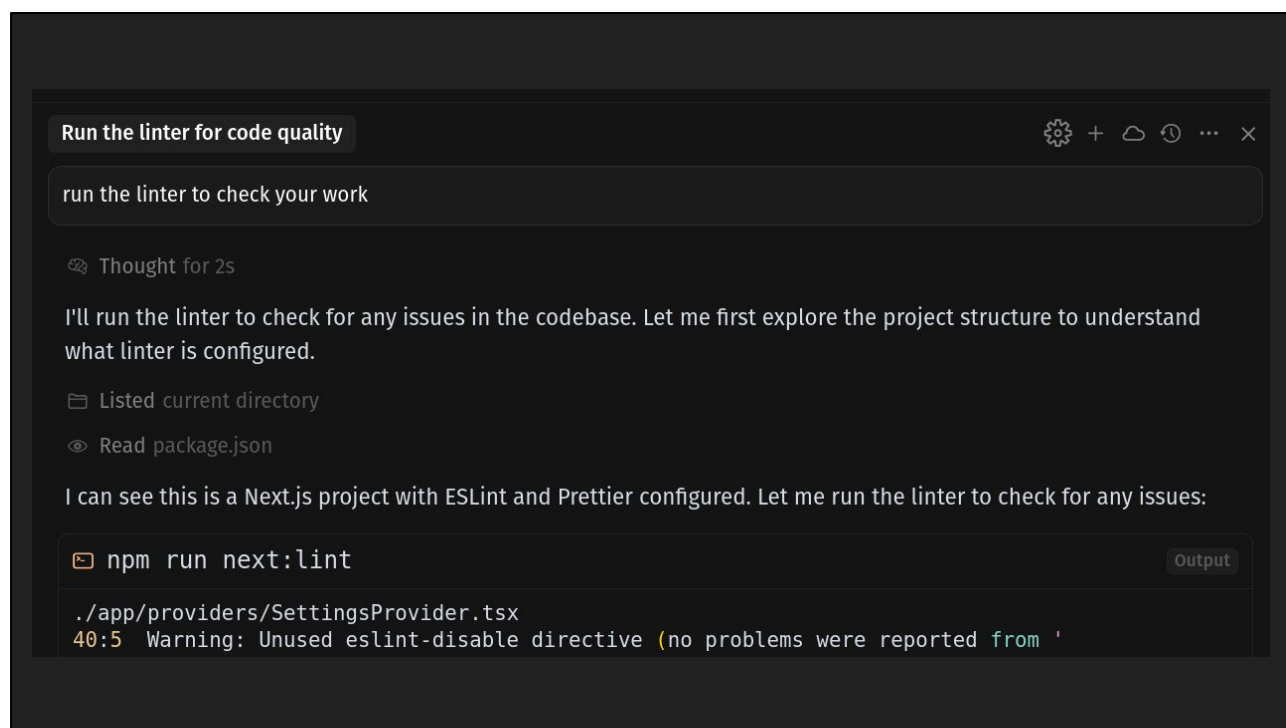
Bypassing Permissions

With Claude, you can enable Auto-Run with the dangerously-skip-permissions flag.

This is annoying to type, so you might create a bash alias, or add it to npm scripts.



Congratulations. We're now in YOLO mode.



In YOLO mode, you don't need to approve any actions.

The agent will just execute whatever it thinks it needs to.

Here it is running a lint script without me needed to approve it.

The linter found several issues that need to be fixed:

Errors:

1. `./app/components/Round.tsx` (line 10:21): Using `any` type - should specify a more specific type
2. `./app/providers/SketchbookProvider.tsx` (line 3:10): `useAction` is imported but never used

Warnings:

1. `./app/components/Round.tsx` (line 71:6): Missing dependency `seededShuffle` in useMemo hook
2. `./app/hooks/useDebounceSyncedState.ts` (line 39:6): Missing dependency `localValue` in useEffect hook
3. `./app/providers/SettingsProvider.tsx` (line 40:5): Unused ESLint disable directive

Let me also check for TypeScript compilation errors:

```
📄 npx tsc --noEmit
```

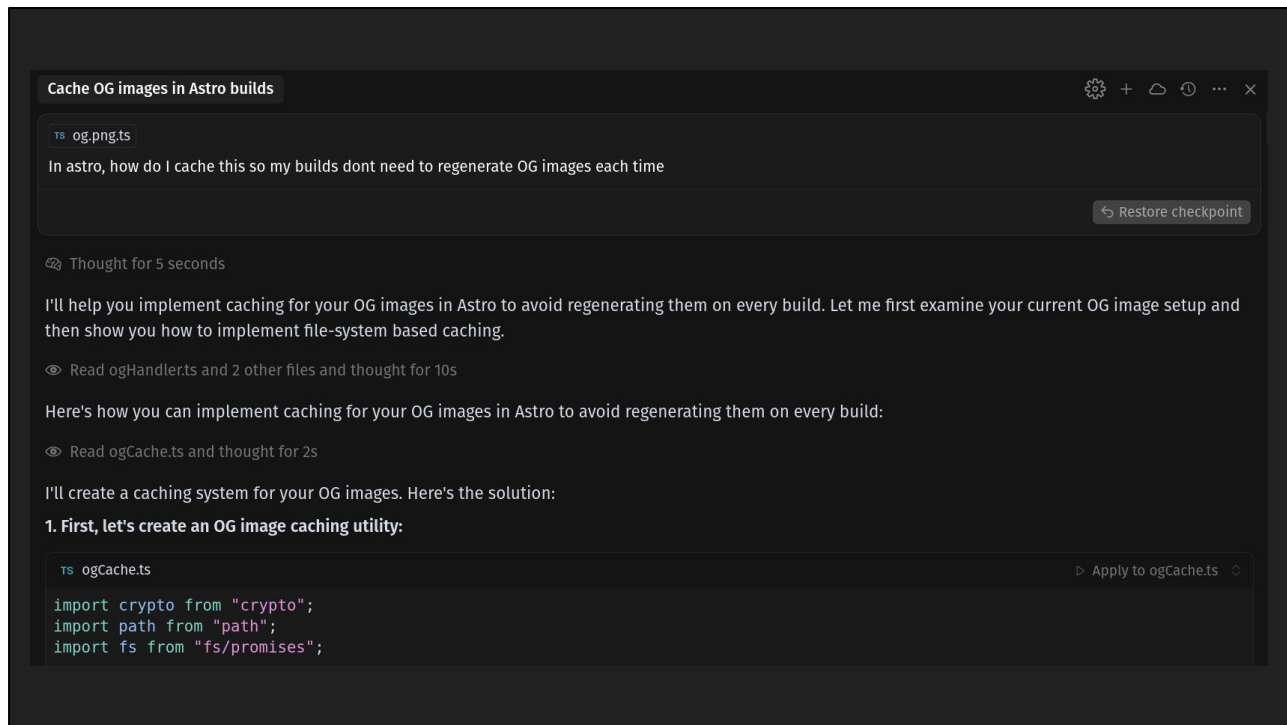
Output

Clever!

After the lint, it decided to also run a typecheck.

Now it's in a verification loop, fixing and checking types until there are no errors.

With Auto-Run mode, we can just fire and forget tasks, and only need to interact when the Agent thinks it's done.



This becomes very powerful when executing long running tasks with many steps.

In this example, I've asked an agent to implement a caching layer, so I don't need to regenerate images every time I run a build.

 Thought for 2 seconds

```
 cd /workspaces/oktech-web && npm run build
```

To properly test and debug this feature, it requires waiting for a number of long-running build steps.

In Supervised mode, this would require babysitting for an extended period.

**A FEW
MINUTES
LATER...**

But in YOLO mode, you can set it off and go make a coffee.

Let me also test the cache management scripts:

```
cd /workspaces/oktech-web && npm run og-cache:stats
```

```
<kspaces/oktech-web && npm run og-cache:stats
```

```
> project@0.0.1 og-cache:stats
```

```
> npx tsx scripts/manage-og-cache.ts stats
```

Cache statistics:

Images: 234

Total size: 56.75 MB

node → /workspaces/oktech-web (main) \$

Thought for 7s and searched 3 times

Perfect! 🎉 All updates have been successfully applied. Here's what I've implemented:

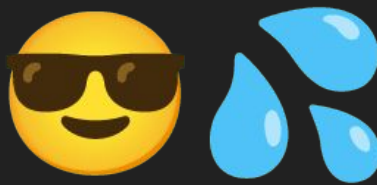
✅ **Complete OG Utilities Reorganization with Caching**

And come back in a few minutes to see that this feature has been implemented successfully with no intervention.

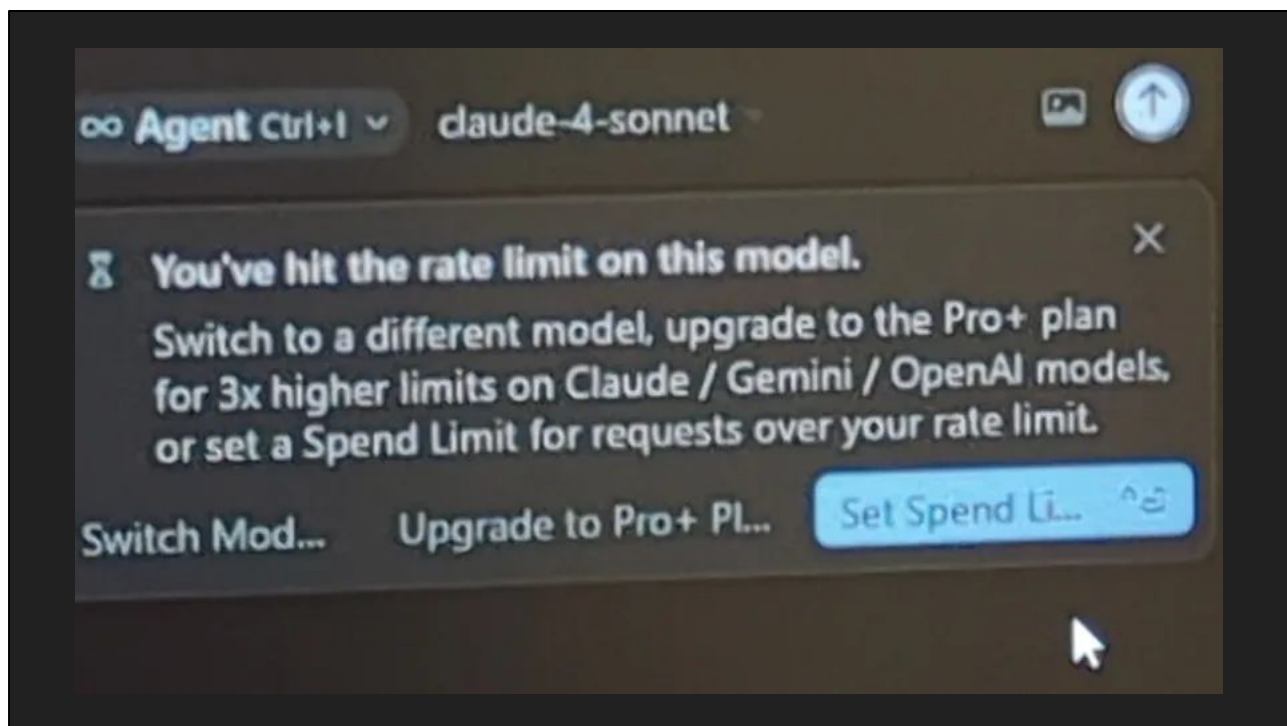
Along the way, it probably made mistakes - but the agent self-corrects as the verification loop iterates, and it fixes problems automatically.

These kinds of tasks are where Agentic Engineering really shines.

What goes from like, half an hour of manual work, now only takes a few seconds of your attention, and you can go work on other things.



But this Auto-Run mode has one little downside. It eats through tokens like nobody's business.



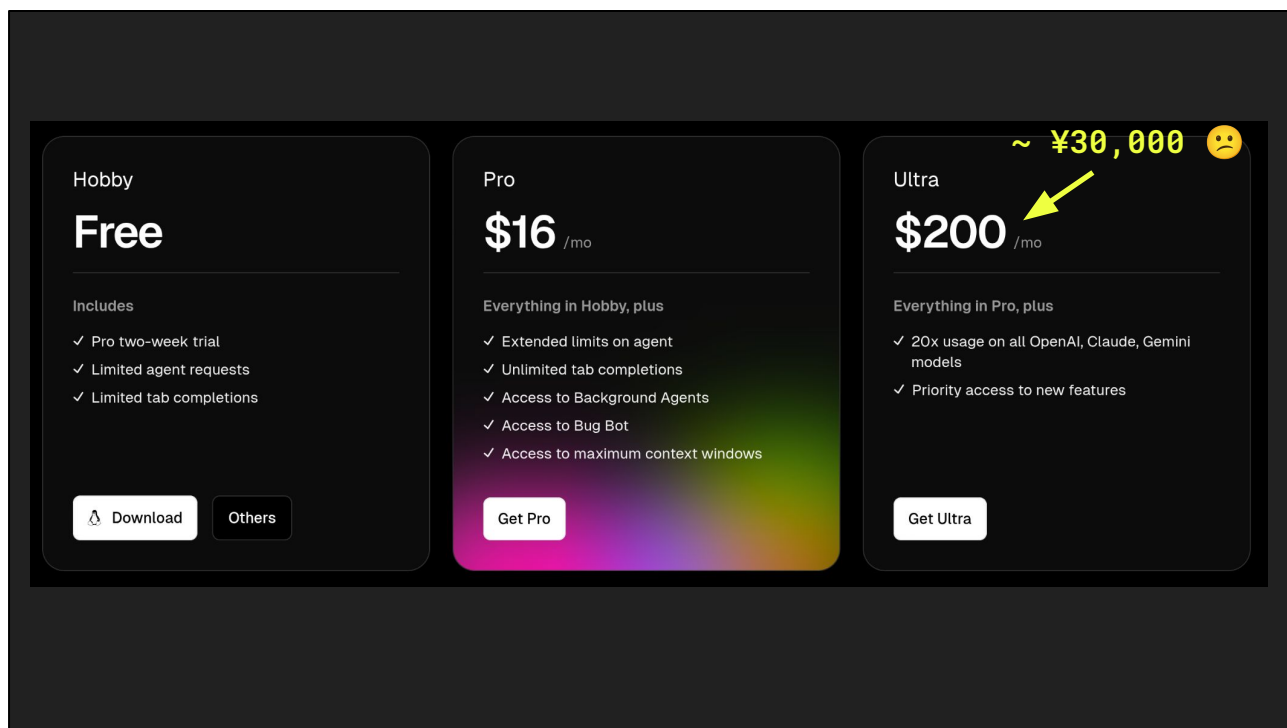
On the Cursor free tier, you will very quickly run into this message if using Auto-Run mode.

"You've hit the rate limit on this model"

Pricing

So now we need to talk about everyone's favorite topic.

Lining the pockets of the kleptocracy.



Here's Cursor's pricing model.

You can experiment for free, but that's pretty useless for anything serious.

Pro is reasonably priced, but you'll still hit your limits if you do extended coding sessions, especially with YOLO agents.

It's the 200 usd per month "Ultra" plan that give you the full power of agents.

Choose your Max plan

Max 5x

\$100
per month

Great for frequent users who work with Claude on a variety of tasks with 5x more usage than Pro.

Sign up

Max 20x

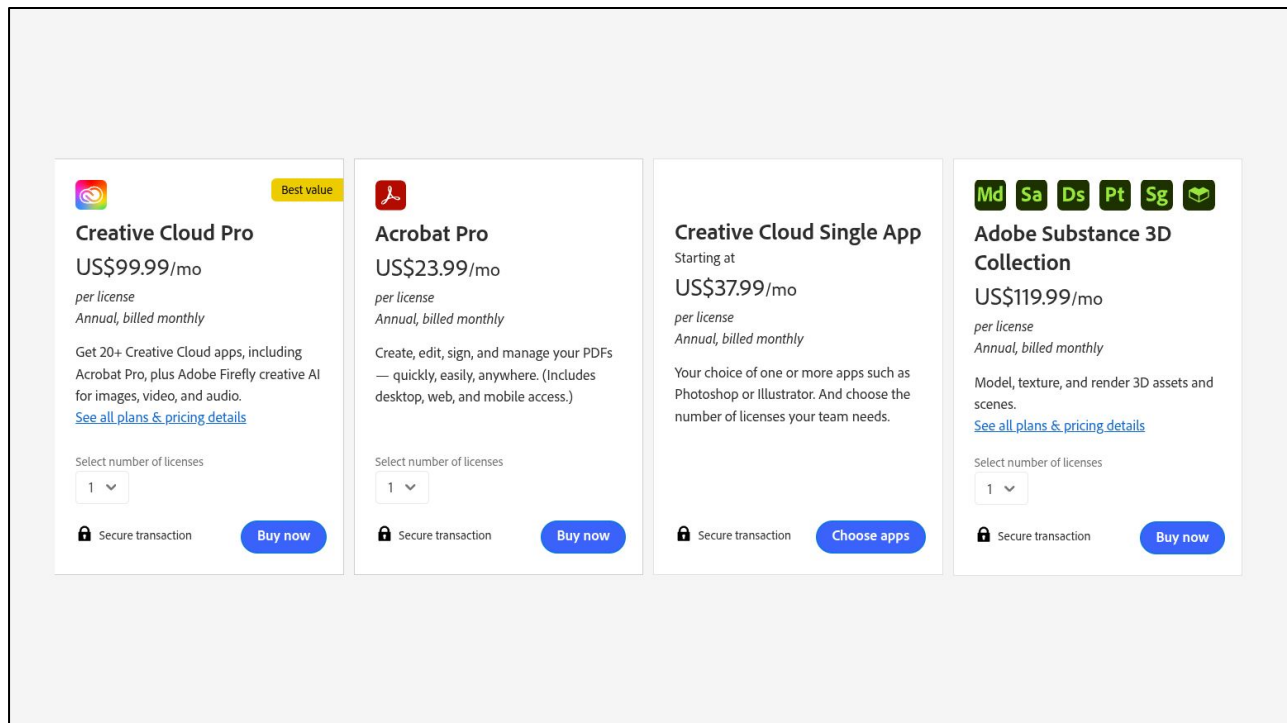
\$200
per month

Perfect for daily users who collaborate often with Claude for most tasks with 20x more usage than Pro.

Sign up

Claude Code's prices are suspiciously identical.

You're probably going to need at least the \$100 tier for YOLO Agents.



Other professional tools like Adobe Create Cloud are also in the 100 dollars per month range.

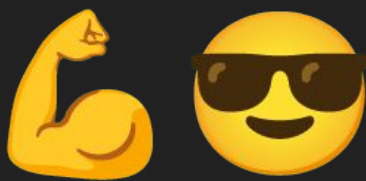
It's expensive, and only really only worth it if you can get value from it.

Prices will probably come down in the future as AI datacenters come online/

But personally, I am easily getting value for money, and I think of it like an early adopter premium.



I figure that if I want to continue Software Engineering Professionally, it's probably best to learn these techniques as soon as possible, while the knowledge gap is relatively shallow.



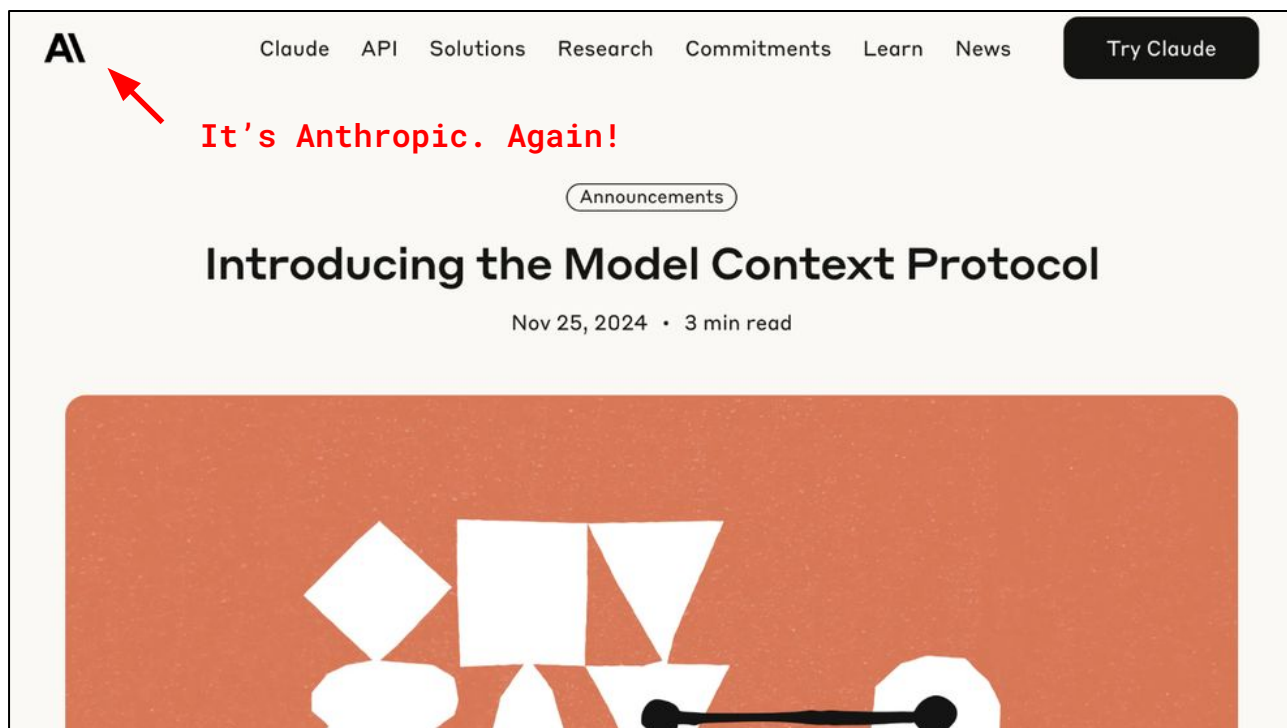
Anyway, we're now slightly poorer, but we're in "True" YOLO agent mode.

But we can go even further.

Part 4: MCP

Model Context Protocol

Let's talk about MCP: The Model Context Protocol.



Introduced by Anthropic in 2024, MCP has become a hot new protocol that has gained massive popularity.

Building MCP servers for deep research

Build an MCP server to use with deep research via API or ChatGPT.

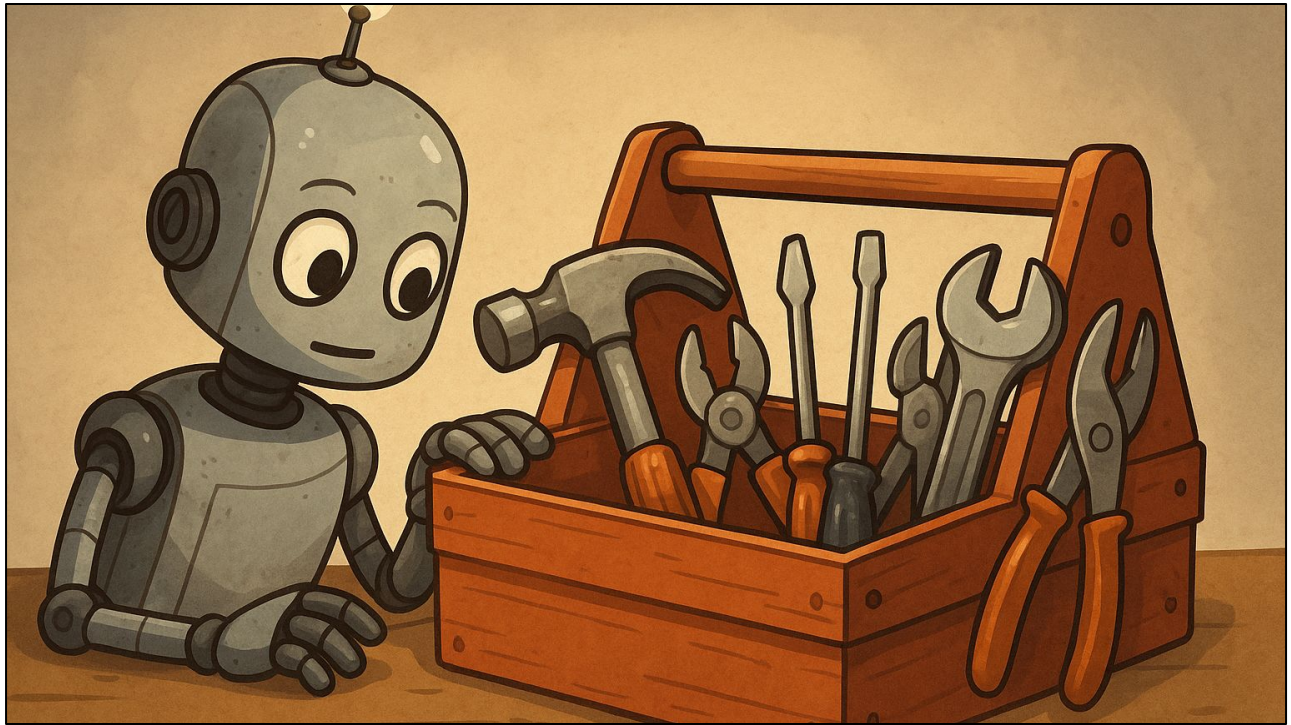
Model Context Protocol (MCP) is an open protocol that's becoming the industry standard for extending AI models with additional tools and knowledge. Remote MCP servers can be used to connect models over the Internet to new data sources and capabilities.

In this guide, we'll cover how to build a remote MCP server that reads data from a private data source (a **vector store**) and makes it available to deep research models in ChatGPT via connectors, and **via API**.

Even to the point where Anthropic's bitter rival OpenAI has adopted it.



A bit like HTTP, because MCP is a well designed open standard, network effects incentivize everyone to converge and use it.

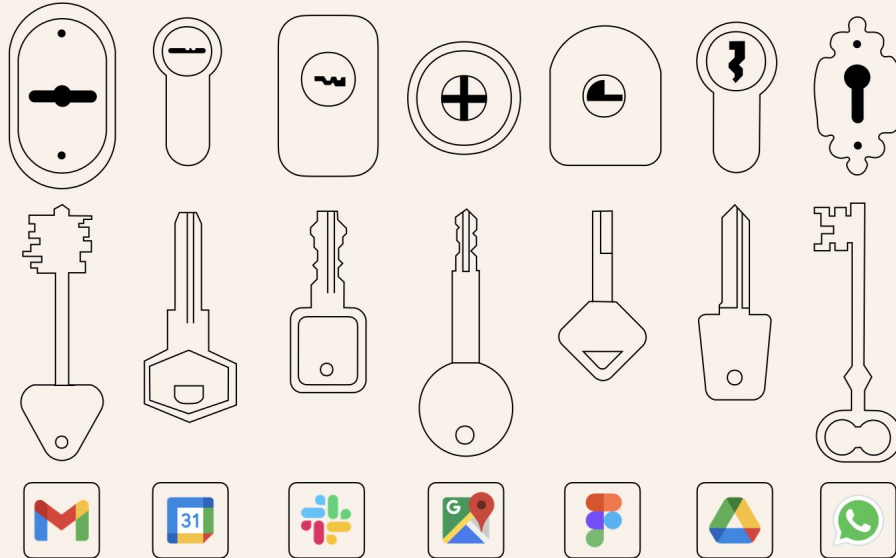


In simple terms, MCP is like a toolbox for Agents.

It allows agents call external services, to read some data or trigger some method.

APIs: Every tool needs its own key

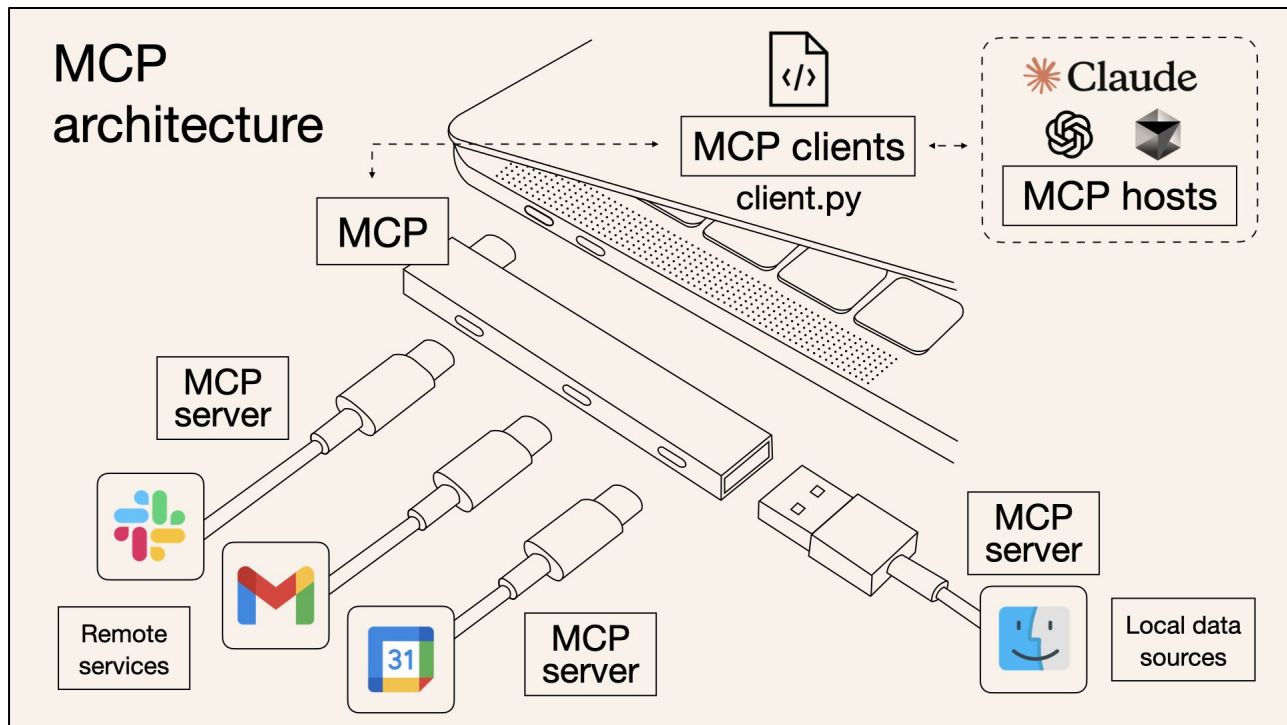
Traditional APIs require different authentication and integration for each service, like needing different keys for different locks



An agent *could* just call a service's traditional API directly.

But this requires extra context and the agent would need to work out how to use that API, with all its qwerks.

An API might spit out a huge response that floods the context window.

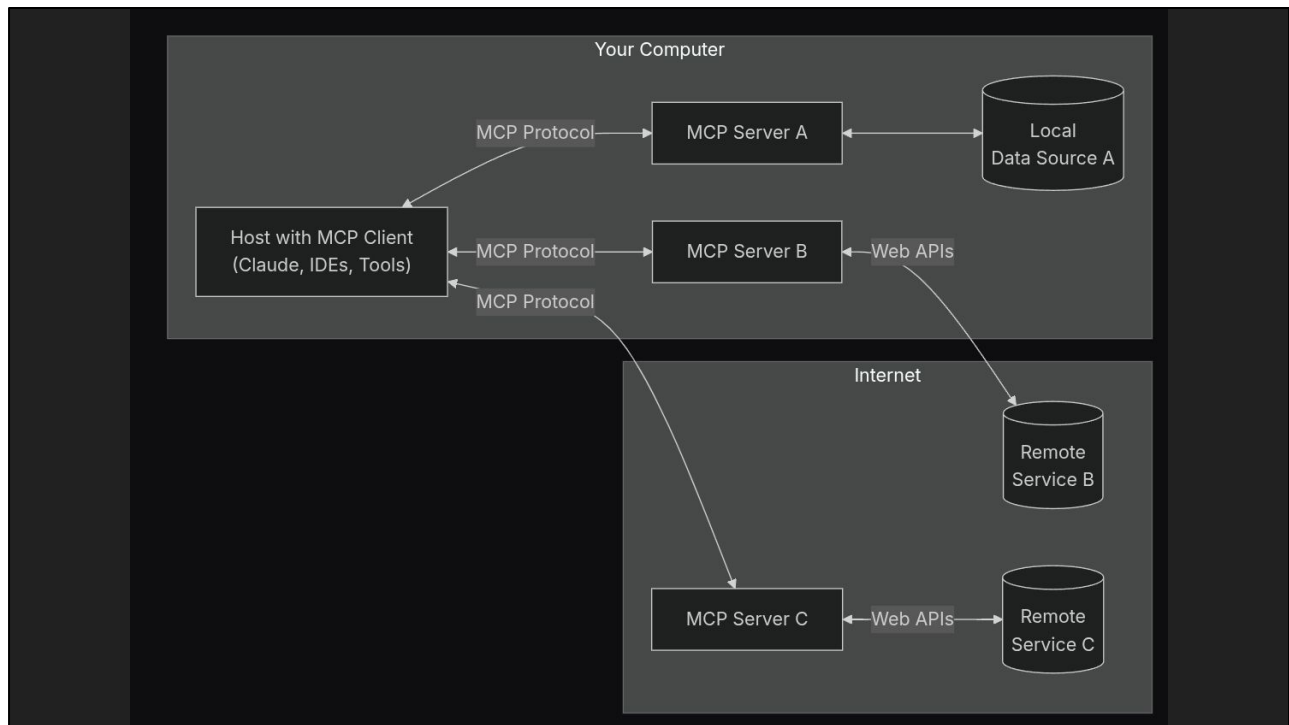


MCP is like a USB-C for APIs, which provides a standard interface for every possible service.

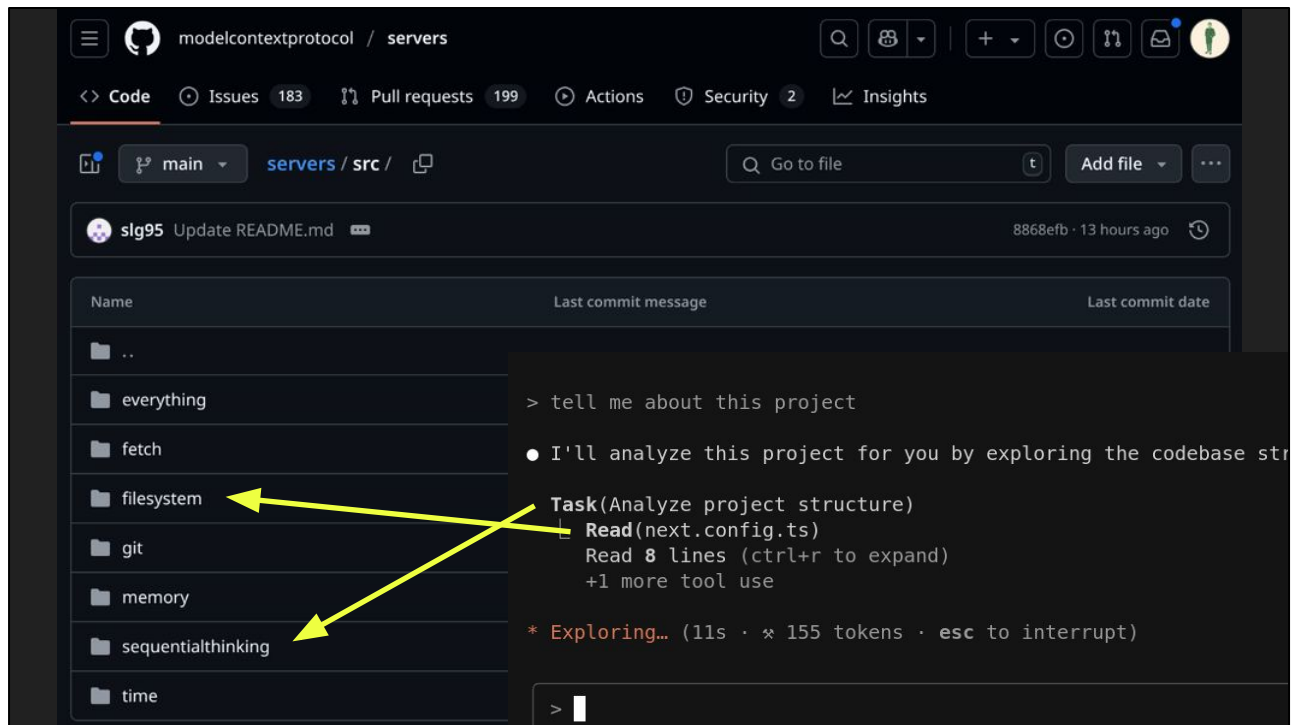
You don't need to worry about SDKs or API docs.

MCP servers describe what actions they are able to perform, and they manage responses specifically for LLMs so their context does not get overwhelmed.

MCP servers are also stateful, unlike most traditional REST APIs, so they can remember previous calls and respond more effectively.

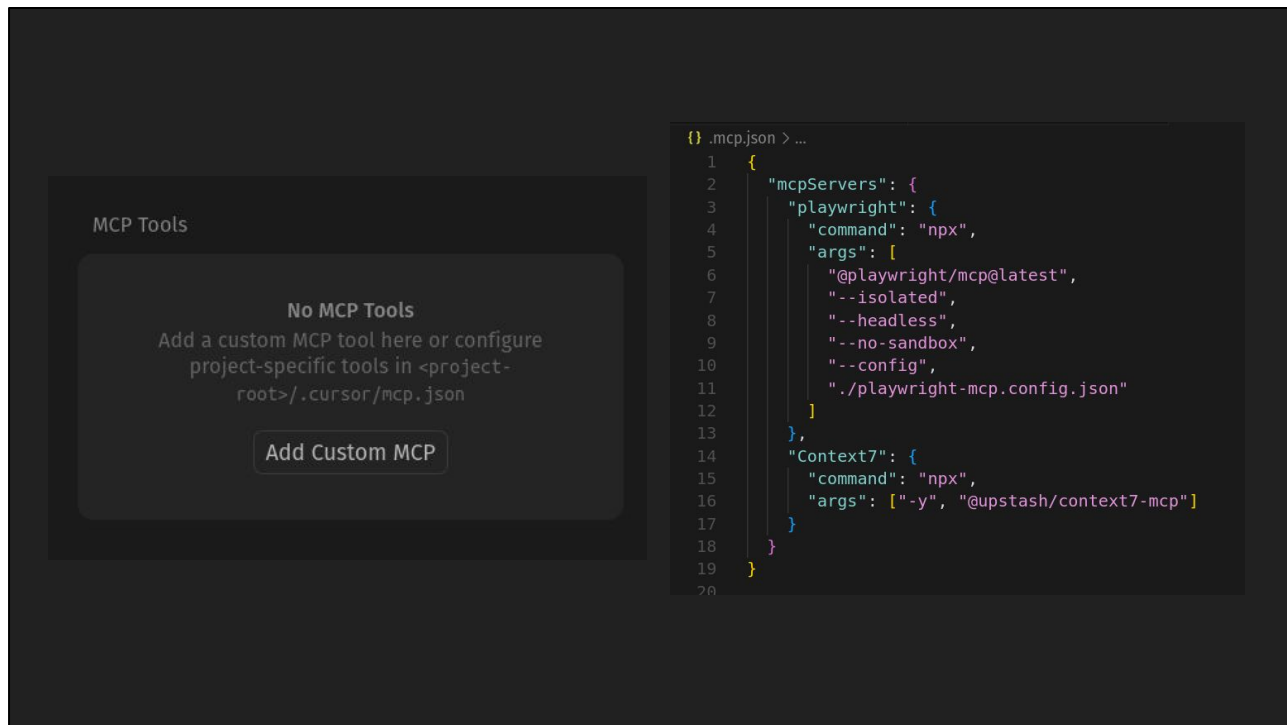


MCP servers can act as a proxy to APIs, and can be called locally or remotely.



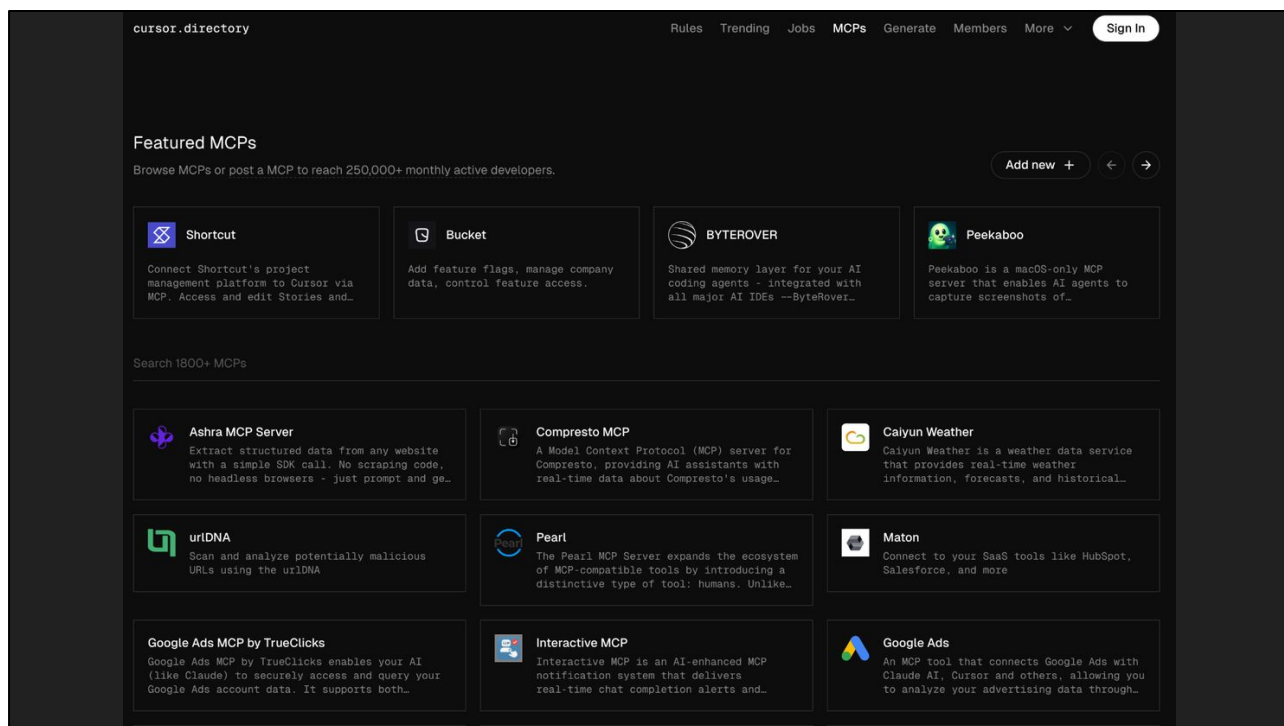
Under the hood, Claude uses MCP extensively for it's core functionality.

But this base functionality can be extended arbitrarily.



On the project or user level, you can configure which MCP servers you'd like to give your Agents access to.

Often they come as NPM packages that start a server locally.



There are thousands of servers to choose from, and the list is growing rapidly.

MCP servers you might find useful

Context7 🍌🍌🍌	Figma to HTML/TSX
Playwright 🍌🍌	Image Manipulation
Taskmaster 🧑🏻💻	Hosted Database
Emulation Automation	Internal Company Docs
Github Issues / PRs	Zapier, Webhooks
UI & Visual Design Feedback	Netlify / Vercel Deployment Checks
"Zen" (multi-model reasoning)	Home Assistant Control
Kubernetes	Nmap / scanning / pentesting
Notification when long task is done	Payments
Psychic Energy Readings 🌌	AI Voice Call Trello

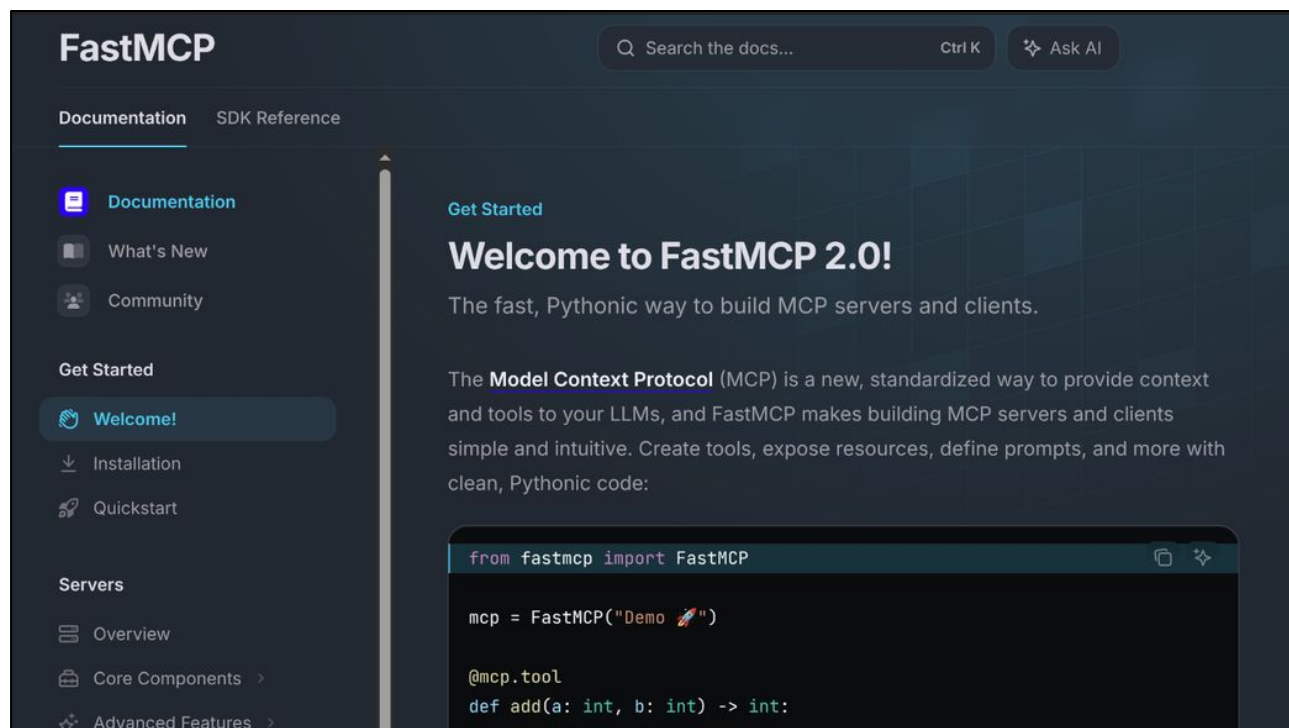
Just to name a few, some useful MCP servers I use daily are

Context7, as mentioned before, for grabbing 3rd party docs

Playwright to give Agents the ability to browse the DOM and take screenshots

And Database Adapter MCPs to check the state of your dev database for debugging

But there are many many more you to choose from



It's also super easy to make your own MCP server, especially with a Coding Agent!

FastMCP is a good choice of framework – it has both Python and Typescript options.

```
> Wait how about we ask the Psychic MCP server to  
"Tune into Lily's field and channel how does she  
feel about the UI right now, rated 0-10 each"
```

- **psychic-vibe:energy_reading** (MCP)(subject: "Lily's feelings about the SoulMirror UI", depth: "deep", timeframe: "current")

└ 🧘 ENERGY READING ACTIVATED 🧘

Subject: Lily's feelings about the SoulMirror UI
... +62 lines (ctrl+r to expand)

- Let me channel specific UI ratings through the psychic field:

- **psychic-vibe:intuitive_guidance** (MCP)(question: "What are

└ 🌙 INTUITIVE GUIDANCE CHanneled 🌙

Question: What are Lily's specific satisfaction r
... +81 lines (ctrl+r to expand)

- The psychic field reveals Lily's UI satisfaction ratings:

Overall Design: 8/10 - Clean but borderline sterile, needs more organic flow

Navigation Flow: 7.5/10 - Functional but stutters, users need clearer journey

Color Scheme: 6/10 - Too safe, needs deeper jewel tones or earth shades

Typography: 9/10 - Sharp and polished, could use handwritten flourish

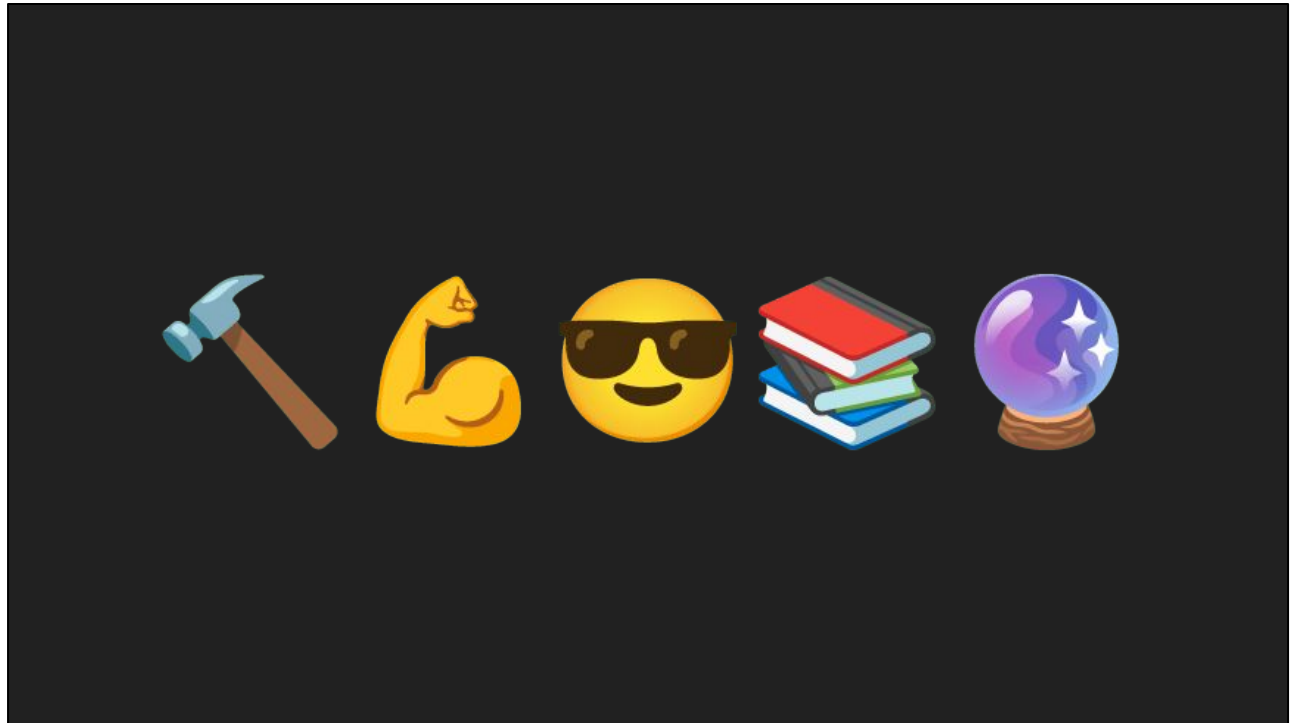
Mobile Responsiveness: 8.5/10 - Seamless but compressed energy on small screens

AI Chat Interface: 7/10 - Smooth but lacks texture and unpredictability

The cosmic counsel suggests the UI is competent but not yet enchanted - it needs more soul, organic flow, and daring to get weird. The field whispers: "Spiritual tools shouldn't feel corporate!" ✨

It's so easy that even a non-programmer friend was able to get her "Soul Mirror" MCP deployed with a Coding Agent.

Now she can be confident that her Vibe-Coded projects are psychically aligned.



So now our Coding agent is souped up with YOLO mode and a range of MCP tools.

With our verification loop in place, we can now effectively blast through tasks unattended.

But next, let's really get Cookin'

Part 5: Orchestration

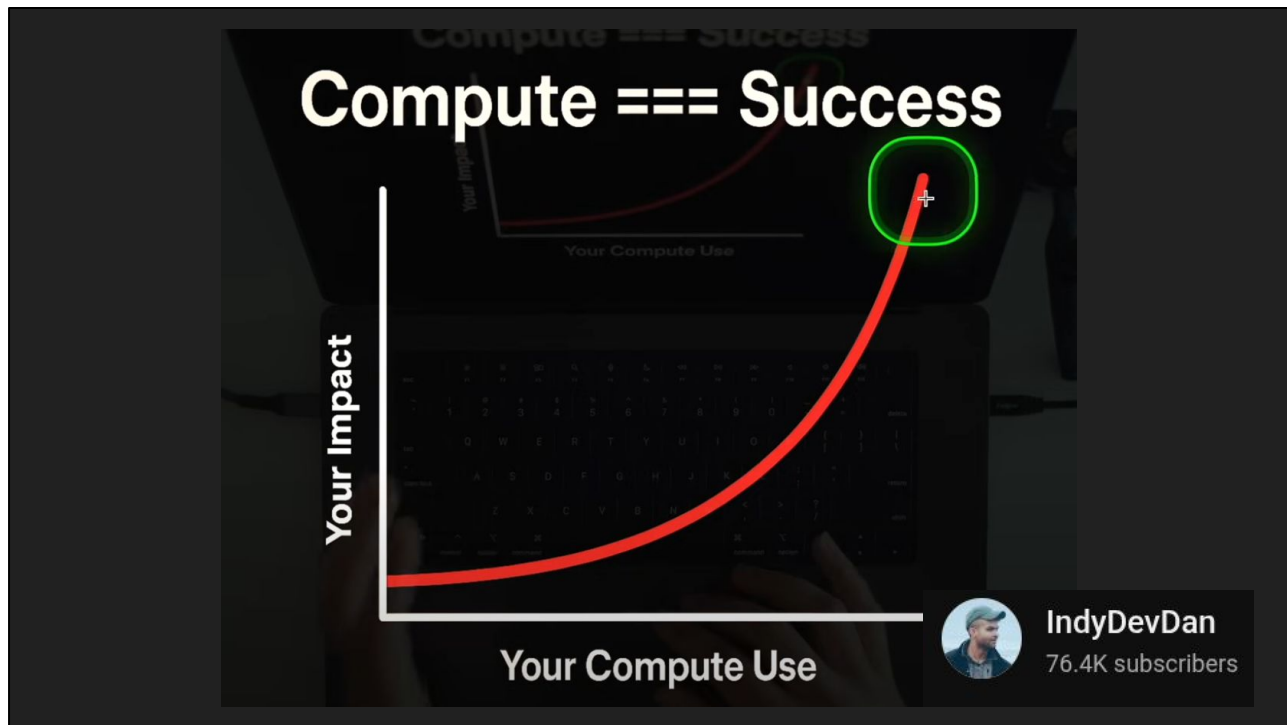
Unlocking Value with Parallelization

As we pondered, true unlock of Agentic coding is the ability to scale.

It's one thing having a single agent plod along, but a completely different story if a single developer can orchestrate many more simultaneously.

We have now reached the cutting edge of Agentic Engineering.

An exciting space to follow that's only emerged within the last month or so.



If we can effectively orchestrate, there's no upper bound to productivity gains.

To do this, we can:

Run multiple small tasks simultaneously in the foreground in subagents

Have Pull Requests automatically trigger Security Audits, Code Reviews, Documentation Generation, etc.

Split project tasks into isolated features that can be completed in parallel branches

Run multiple different feature tasks for *different projects* in the background

Aim for increasingly longer-running unattended tasks

Run the same task multiple times, and pick the best result

And eventually, compose all of the above into complex meta-tasks, with workflows where agents spin up other agents automatically

Subagents

Claude Micro Orchestration

But let's start small and quickly mention this currently undocumented SubAgents feature of Claude.

```
> run quickfix, the test suite, update my readme with a basic intro to the app, and create a CHANGELOG.md with daily updates based on my git history. use subagents for these tasks.
```

- I'll use subagents to handle all four tasks in parallel: running quickfix, running tests, updating the README, and creating a CHANGELOG.

- **Task**(Run quickfix)
 - └ **Bash**(npm run quickfix)
Waiting...

"use subagents"

- Task**(Run test suite)
 - └ **Bash**(npm run test)
Waiting...

- Task**(Update README intro)
 - └ **Read**(README.md)
Read 29 lines (ctrl+r to expand)

- **Task**(Create CHANGELOG.md)
 - └ **Bash**(git log --oneline --pretty=format:@"%h %ad %s" --date=short)
Waiting...

If you ask Claude to "use subagents", you can have it execute steps in parallel.

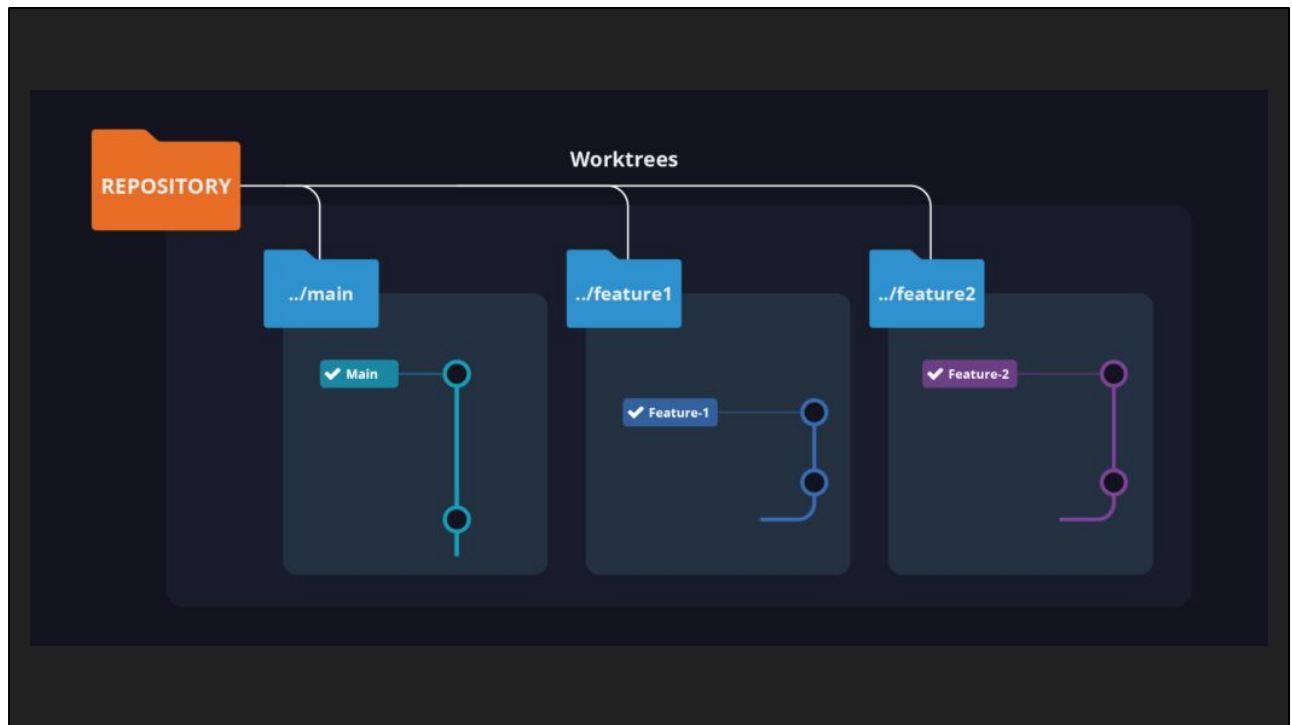
This is a simple way to speed up things that don't need to be done in series.

You can even have it spin up new instances of claude itself as Sub Agents.

Git Worktrees

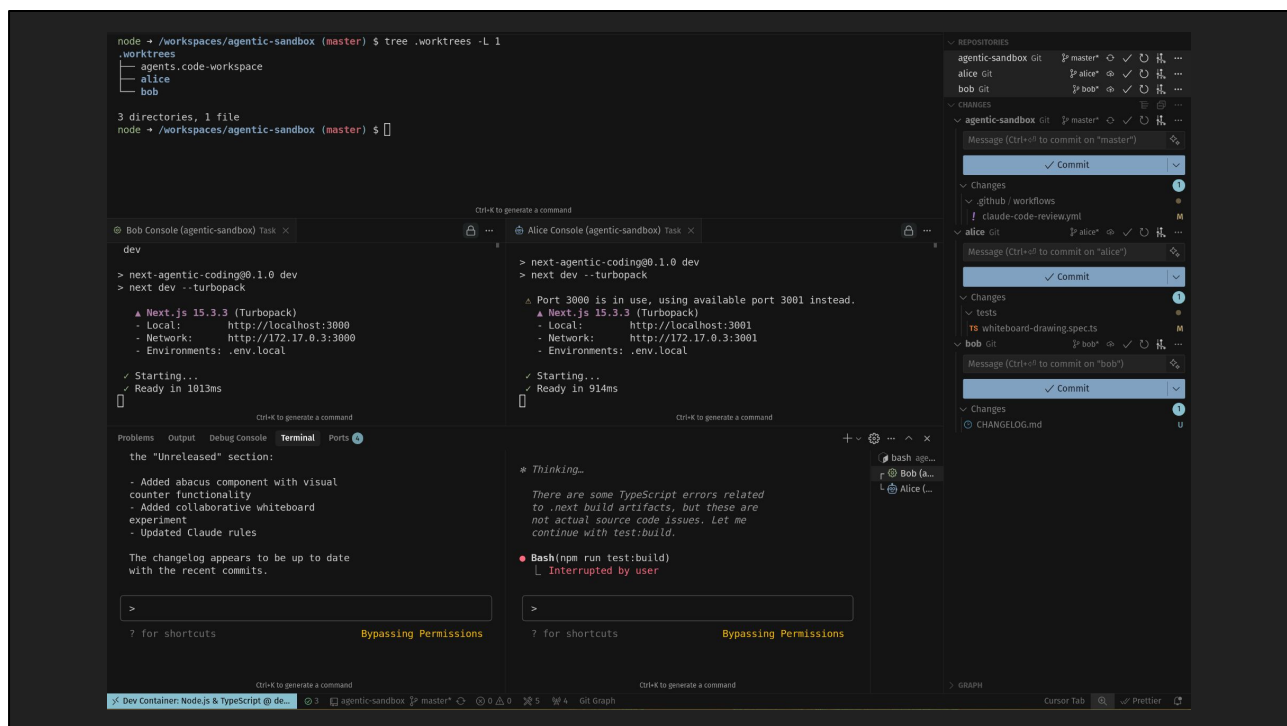
Orchestrating Claude

If you want more control for simultaneous long running tasks, you can utilize Git Worktrees.



Git worktrees let you have multiple copies of your repository in different directories, working on different branches.

With worktrees, you can easily manage multiple branches without having to jump between checkouts in the same directory.



You can then have two or more terminal windows with agents executing at the same time on different branches.

You can reserve a “main” worktree just for merging completed tasks.

But one problem with worktrees is that you need to organize your project in a way that manages conflicting ports and database operations, as worktrees share the same environment.

```
.devcontainer > worktree-alice > {} devcontainer.json > ...
1  {
2    "name": "Alice Worktree",
3    "build": {
4      "dockerfile": "../Dockerfile"
5    },
6    "mounts": [
7      "source=${localEnv:HOME}/.claude,target=/home/node/.claude,type=bind"
8    ],
9    "remoteEnv": {
10     "CLAUDE_CONFIG_DIR": "/home/node/.claude",
11     "APP_PORT": "3100"
12   },
13   "postCreateCommand": "npm install",
14   "customizations": {
15     "vscode": {
16       "extensions": [
17         "esbenp.prettier-vscode",
18         "dbaeumer.vscode-eslint",
19         "bradlc.vscode-tailwindcss",
20         "christian-kohler.path-intellisense"
21       ]
22     }
23   },
24   "workspaceFolder": "/workspaces/${localWorkspaceFolderBasename}/.worktrees/alice"
25 }
26
```

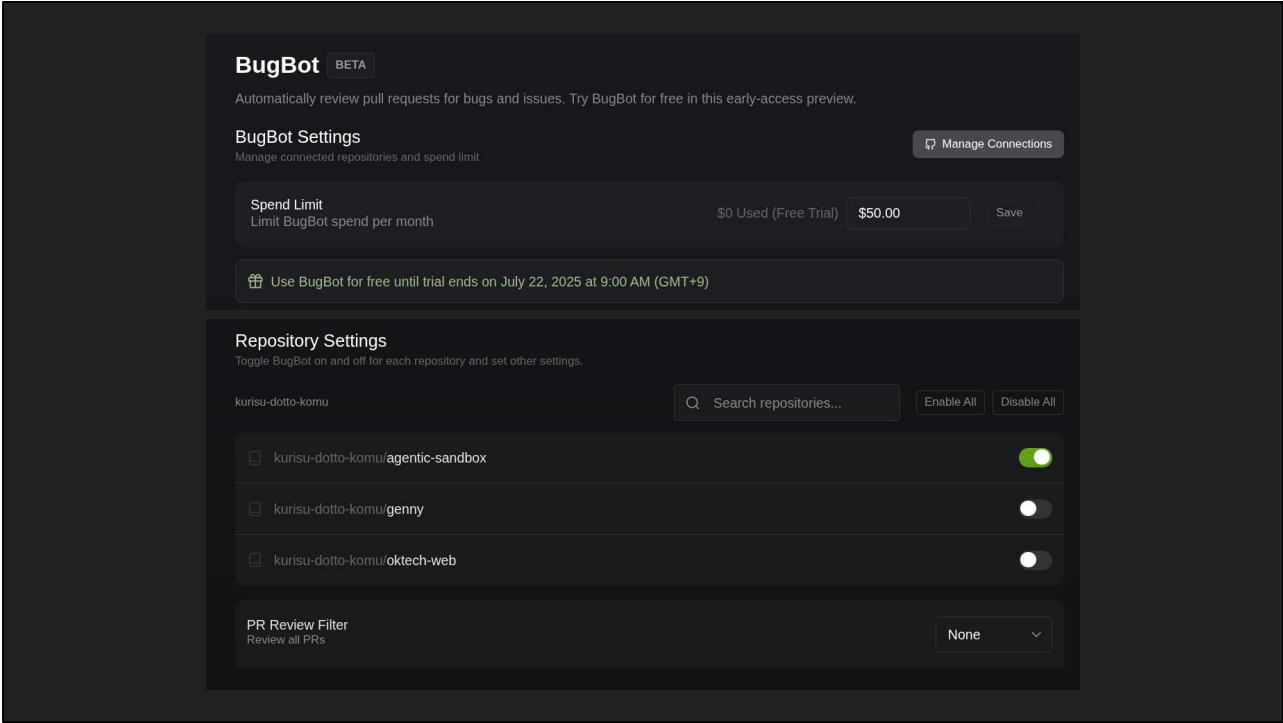
You can sidestep THAT limitation by combining worktrees with devctonainers.

Completely isolated dev environments means you don't need to worry about conflicting networking, while still sharing the local same git repository.

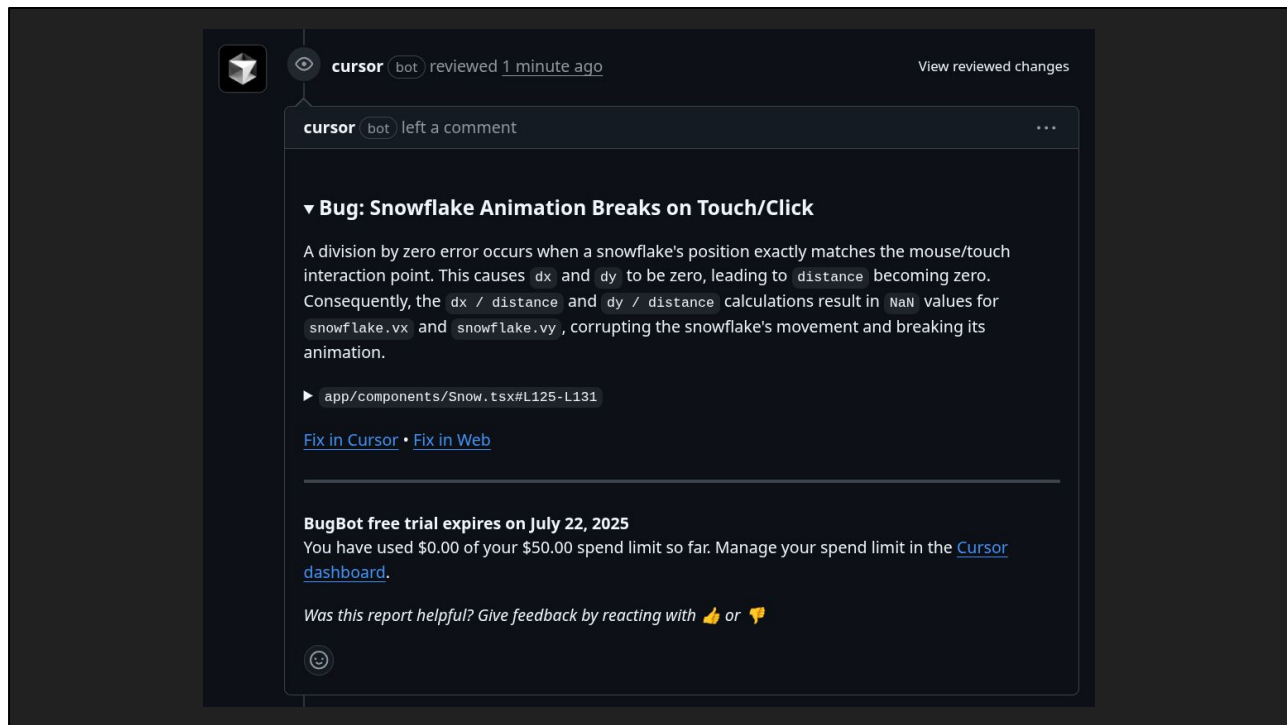
Triggers

Autonomous Orchestration

Another way to Orchestrate is to have agents do work for you automatically when certain events happen in your development lifecycle.



Cursor has a BugBot integration, which will trigger a pre-packaged bug scanning routine on every pull request.



It can spot nuances that might not be obvious in regular testing.

Why not have an additional, qualitative, safety layer that LLMs can uniquely provide.

The image shows a GitHub Actions workflow on the left and a Claude code review interface on the right. The workflow is a YAML file named 'Claude Code Review' that triggers on pull request events. It includes a job 'claude-review' that runs on 'ubuntu-latest' and uses the 'actions/checkout@v4' action. The main step is 'Run Claude Code Review', which uses the 'anthropics/claude-code-action@beta' action. The action is configured with an 'anthropic_api_key' from secrets, an 'env' block containing a 'direct_prompt' with a review request, and a 'Be constructive and helpful in your feedback.' instruction. Annotations with yellow arrows point to 'pull_request:' (labeled 'any github trigger'), 'claude-code-action' (labeled 'claude-code-action'), and the 'direct_prompt' (labeled 'any prompt'). The right side shows the Claude interface with a 'PR Review Complete' status, a 'Code Review Summary', and a list of 'Strengths' and 'Potential Bugs & Issues'.

```
name: Claude Code Review
on:
  pull_request:
    types: [opened, synchronize]
jobs:
  claude-review:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Run Claude Code Review
        id: claude-review
        uses: anthropics/claude-code-action@beta
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
        env:
          direct_prompt: |
            Please review this pull request and provide feedback on:
            - Code quality and best practices
            - Potential bugs or issues
            - Performance considerations
            - Security concerns
            - Test coverage
            Be constructive and helpful in your feedback.
```

any github trigger

claude-code-action

any prompt

claude · bot · commented 3 hours ago · edited · ...

Claude finished @kurisu-dotto-kenny's task — View job

PR Review Complete ✓

Code Review Summary

This PR successfully implements an interactive snow experiment with good overall structure and adherence to repository guidelines. The implementation demonstrates solid React/TypeScript patterns and effective Canvas API usage.

✓ Code Quality & Best Practices

Strengths:

- Excellent adherence to repo guidelines: All files follow the specified patterns (TypeScript, `export default function`, `@r imports`)
- Clean component structure: Good separation of concerns with page components as thin wrappers
- Proper TypeScript usage: Strong typing with the `Snowflake` interface and no type shortcuts
- Consistent code style: Matches existing patterns in the codebase
- Good file organization: Route restructuring creates logical grouping under experiments

Minor improvements:

- The `show.tsx` component is exactly 200 lines, approaching the 120-line guideline in `CLAUDE.md#8`. Consider splitting animation logic into custom hooks or utility functions.

⚠ Potential Bugs & Issues

Canvas initialization race condition (Show tsx:26-27):

```
const ctx = canvas.getContext("2d");
if (!ctx) return;
```

The Canvas API can return `null` in certain edge cases (e.g., context loss). While handled correctly, consider adding error boundaries for production robustness.

Animation cleanup dependency (Show tsx:189):

```
}, [isInteracting]);
```

The `useEffect` depends on `isInteracting` but the animation loop doesn't directly use this value. The dependency might cause unnecessary re-initialization. Consider using a ref for the interaction state within the animation loop.

🔧 Performance Considerations

Good practices:

- ✓ Proper cleanup of event listeners and animation frames
- ✓ Efficient snowflake recycling (reposition instead of recreate)
- ✓ Reasonable number of snowflakes (150)

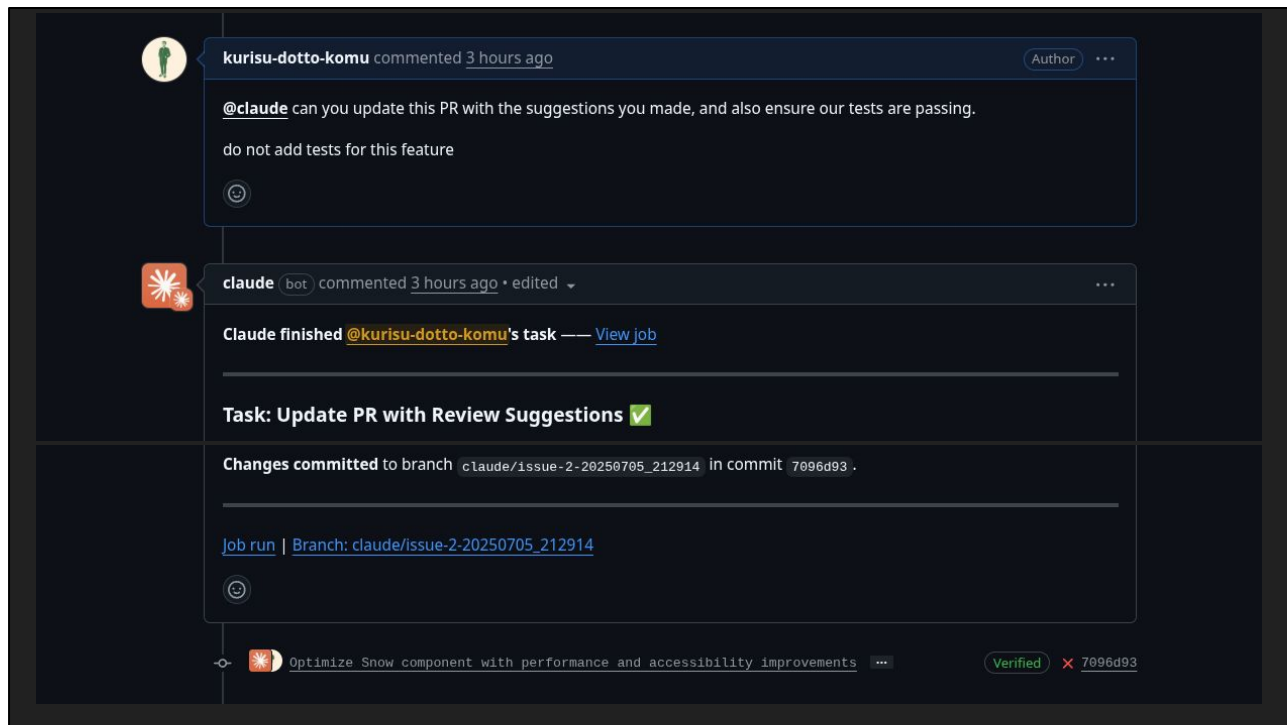
Optimization opportunities:

Claude is much more powerful than Cursor when it comes to triggers because of its CLI nature.

Here's an example of a similar code review trigger for pull requests, using Claude within a Github workflow.

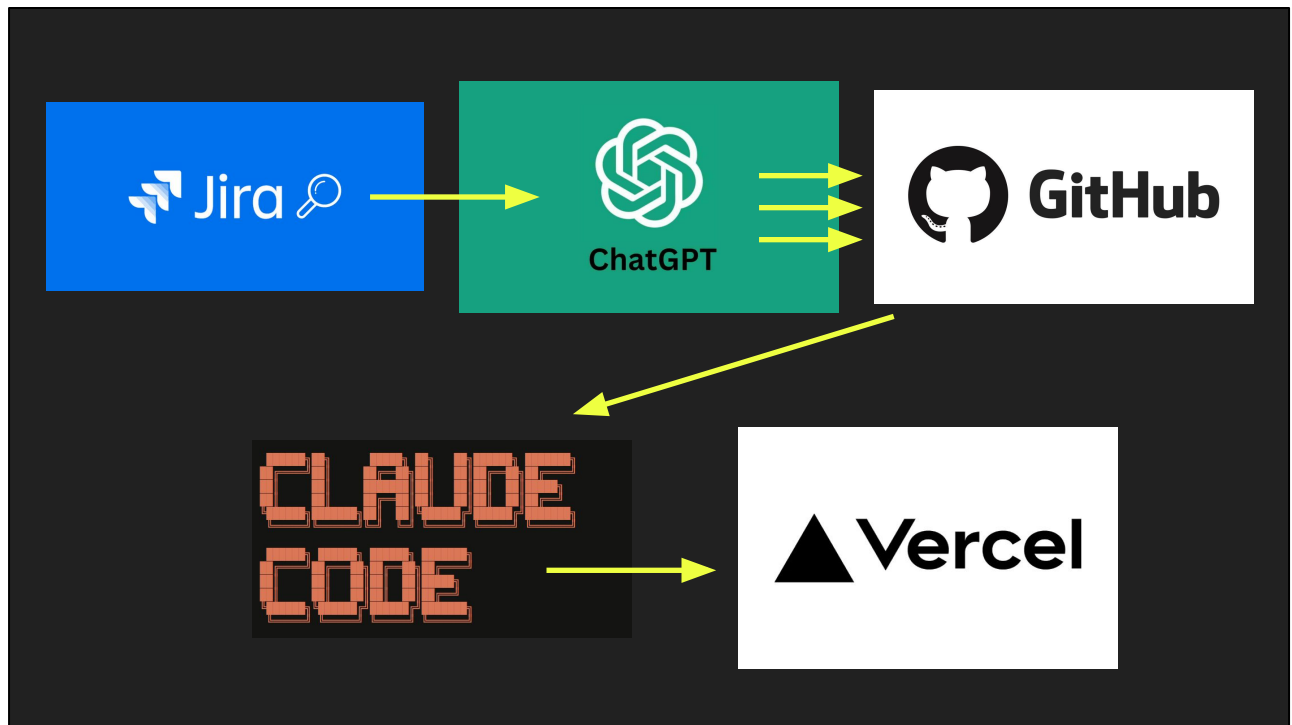
With this technique you can trigger Agentic tasks at any point in your development lifecycle.

Any trigger, any prompt, any scope. Super powerful.



You can even summon Claude within a github thread itself to have it implement changes, or do anything you want really.

But why limit to github?



It's 2am Japan time.

New issue arrives in Jira.

An LLM automatically triages it and determines that it's solvable.

An issue is generated on github, and three PRs with different solutions are generated.

Claude automatically implements the plans in Pull Requests.

Preview deploys are created automatically on Vercel.

You wake up at 6am, check things over, and manually approve the best Pull Request.

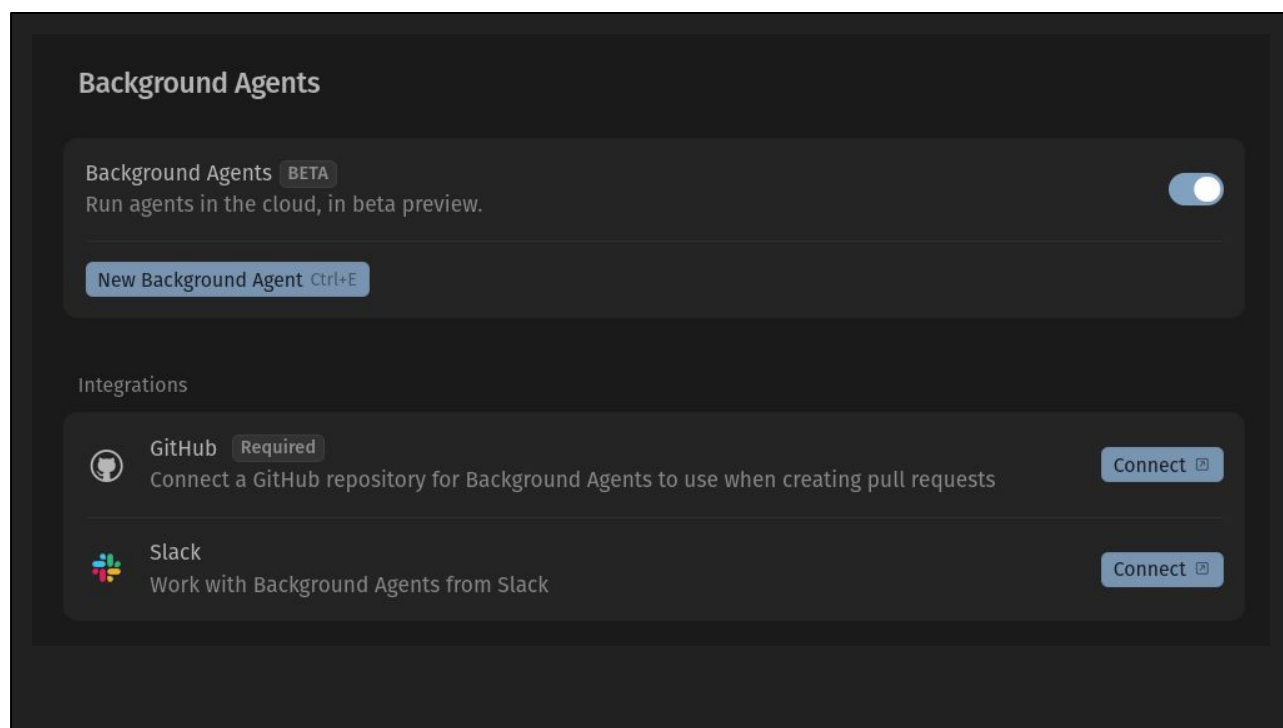
This new Agentic workflow means you can have solutions to problems ready before you are even aware of them.

Background Agents

Orchestrating Cursor

For Orchestrating big, long running tasks, Cursor's approach is called "Background Agents"

Basically, Auto-Run agents that can be spawned in the cloud.



You need to enable this feature specifically in Cursor settings.

Background Agents Setup

Background agents autonomously work on your codebase in the cloud. Let's setup the background agent's machine so it can get started. For more info, see [our docs](#).

STEPS

① Create a Machine Snapshot

Interactively set up all tools and dependencies needed, as if you were setting up a new computer to do development.

Once done, take a snapshot of the environment. All future background agents will start from this snapshot.

Take Snapshot

Rebuild Environment

Use Custom Dockerfile

② Configure Runtime Settings

...

ProblemsOutputDebug ConsoleTerminalPorts1bash +

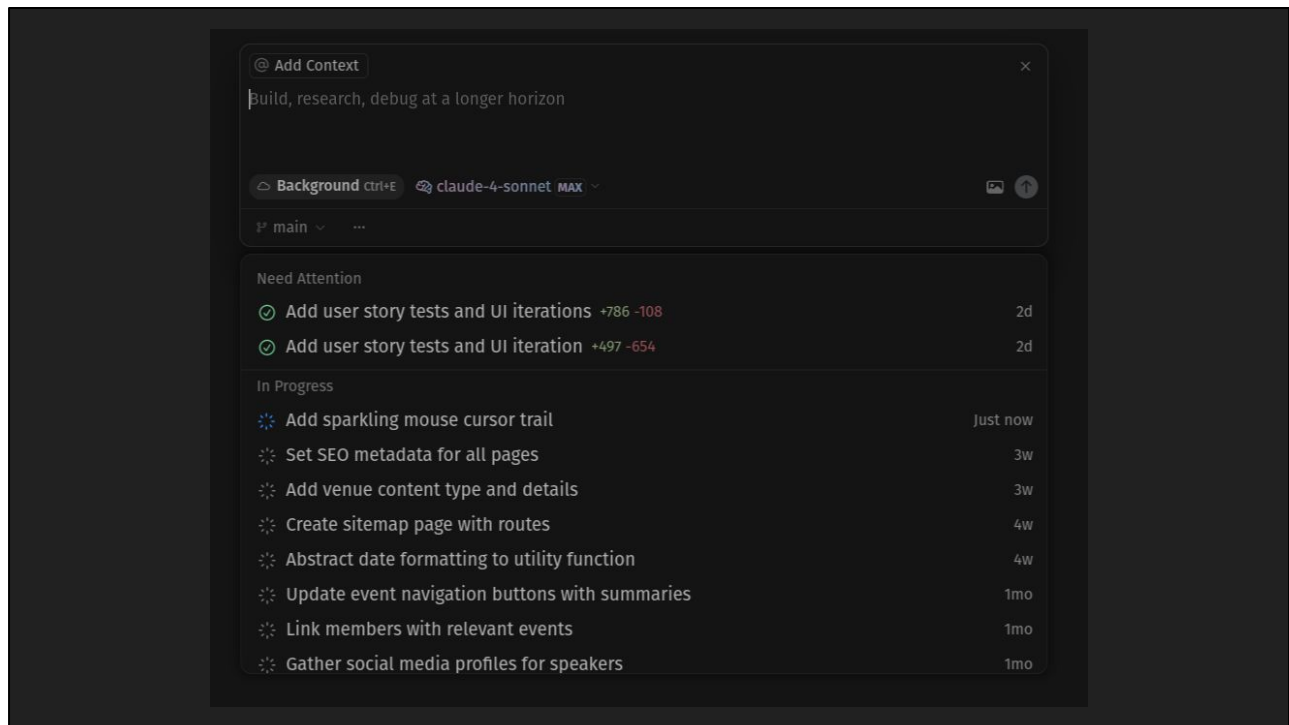
```
47/1054)
00:13:49 ▶ /_astro/knowledge-salon-logo.CYNwQa_k_17X4xS.webp (reused cache) (+2ms) (1048/1054)
00:13:49 ▶ /_astro/aiming-logo.IYNlIZrz_Z1CNrIS.webp (reused cache) (1049/1054)
00:13:49 ▶ /_astro/cybozu-logo.Ch1fNYi9_Z1cij1B.svg (reused cache) (1050/1054)
00:13:49 ▶ /_astro/r3-logo.B2knscTd_Z1cij1B.svg (reused cache) (1051/1054)
00:13:49 ▶ /_astro/galleria-logo.CfQeS-ru_1MauBu.webp (reused cache) (1052/1054)
00:13:49 ▶ /_astro/co-ba-logo.CtDNGveP_Z1cij1B.svg (reused cache) (1053/1054)
00:13:49 ▶ /_astro/helpfeel-logo.BxU0LBHU_Z1cij1B.svg (reused cache) (1054/1054)
00:13:49 ✓ Completed in 274ms.

00:13:49 [build] 381 page(s) built in 18.24s
00:13:49 [build] Complete!
o workspace $
```

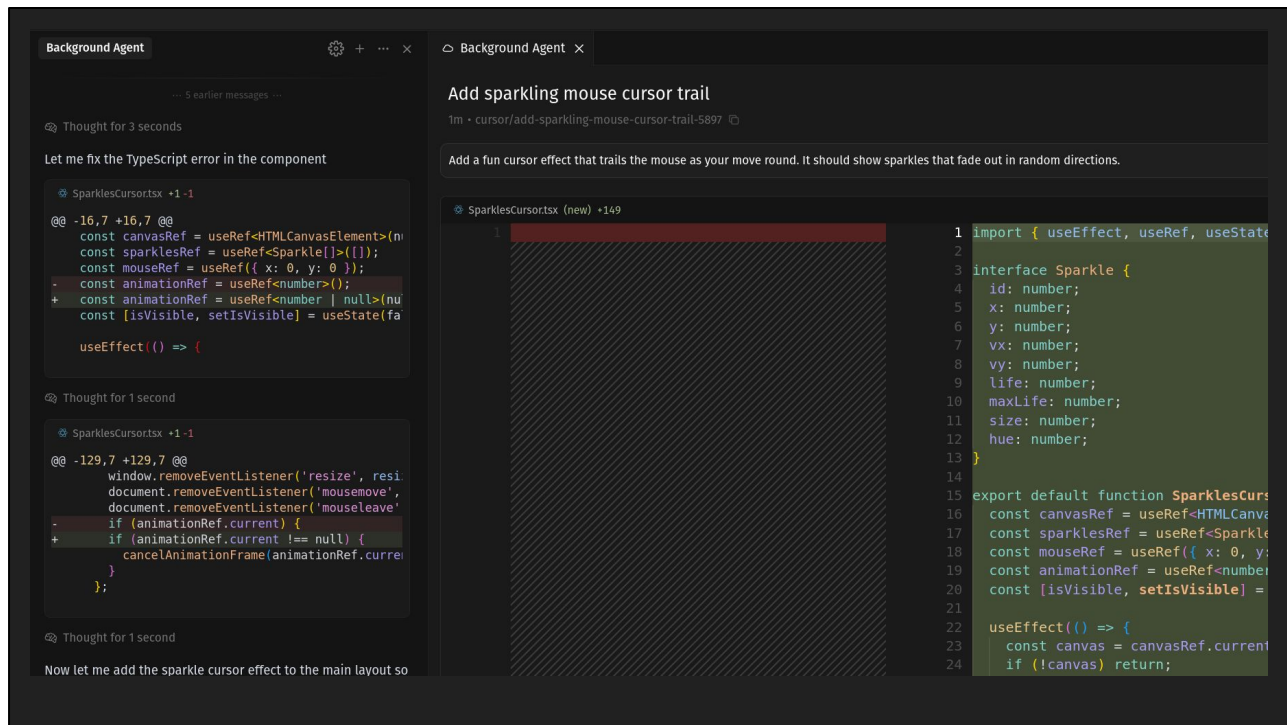
For each project, cursor will step you through the set up of a Virtual Machine snapshot.

You want to get to the point where you can build, run tests, and have primed any caches, ready for development, then create a snapshot.

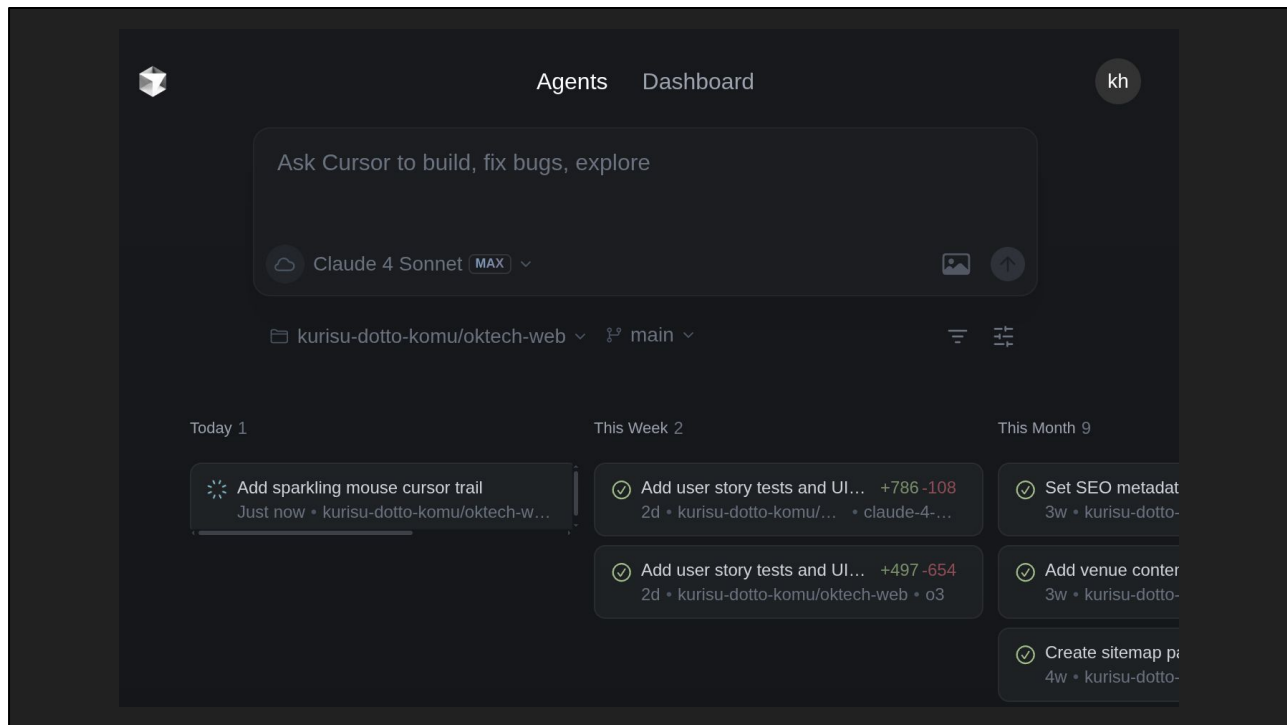
Background Agents connect to your project's github repo and will implement features on a new branch within this snapshotted environment.



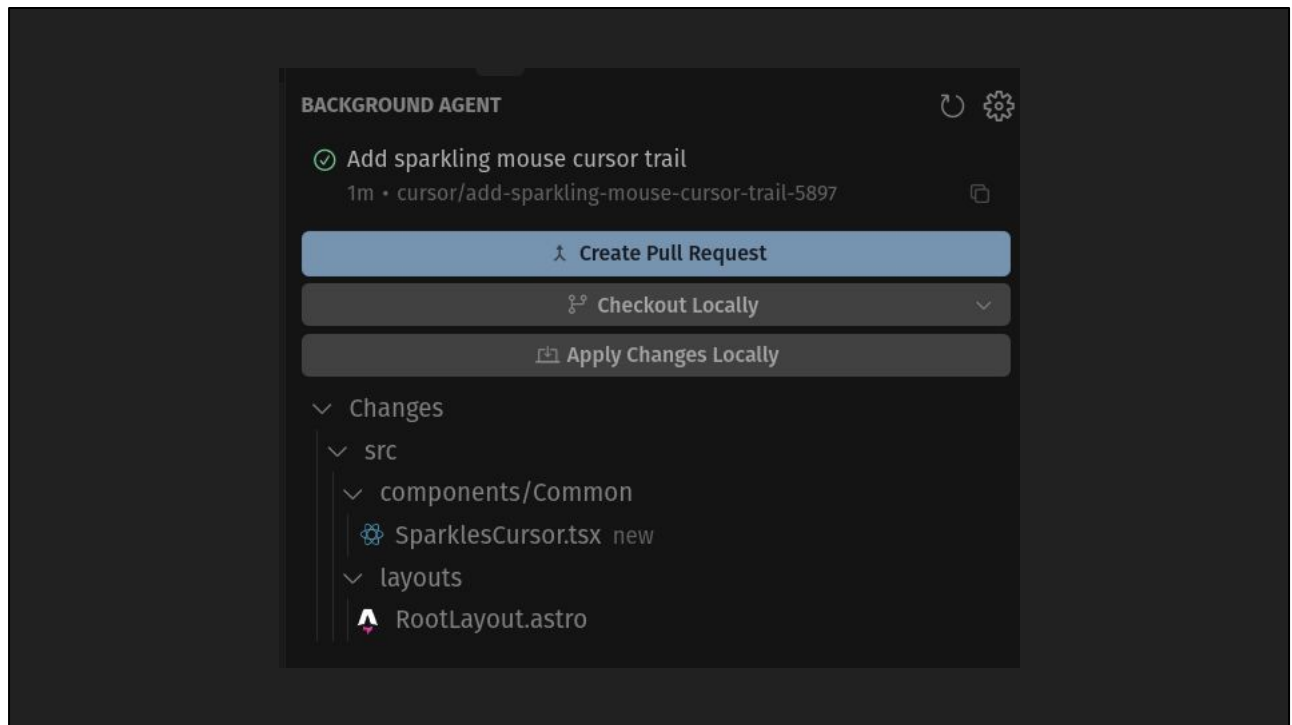
Once it's set up, you can run background agents just like you would with a regular agent.



The difference is you execute it in the cloud, and it will ping you with the changes when it's done.



There's even a web-based dashboard, which you can use to launch and monitor these background agent tasks.



Upon completion, you can checkout the changes locally in your IDE, or have the Agent make a github pull request automatically.

Update hero component with animated background #2

[Open](#) kurisu-dotto-ko... wants to merge 3 commits into `main` from `cursor/update-hero-component-with-animated-background-a588`

Conversation 1 Commits 3 Checks 1 Files changed 2 +400 -2

kurisu-dotto-komu commented 10 minutes ago Owner

This pull request contains changes generated by Cursor background composer.

`Changes from background composer bc-d06f07a2-2cf8-4a8c-8a7c-b18256ee07b2` ✓ `cd01261`

vercel bot commented 10 minutes ago • edited

The latest updates on your projects. Learn more about Vercel for Git

Name	Status	Preview	Comments	Updated (UTC)
oktech-web	Building (Inspect)	Visit Preview	Add feedback	Jul 4, 2025 0:50am

`Remove ShaderBackground component documentation` ✓ `b1213ac`

vercel bot deployed to **Preview** 7 minutes ago View deployment

Reviewers
No reviews
Still in progress? [Convert to draft](#)

Assignees
No one—[assign yourself](#)

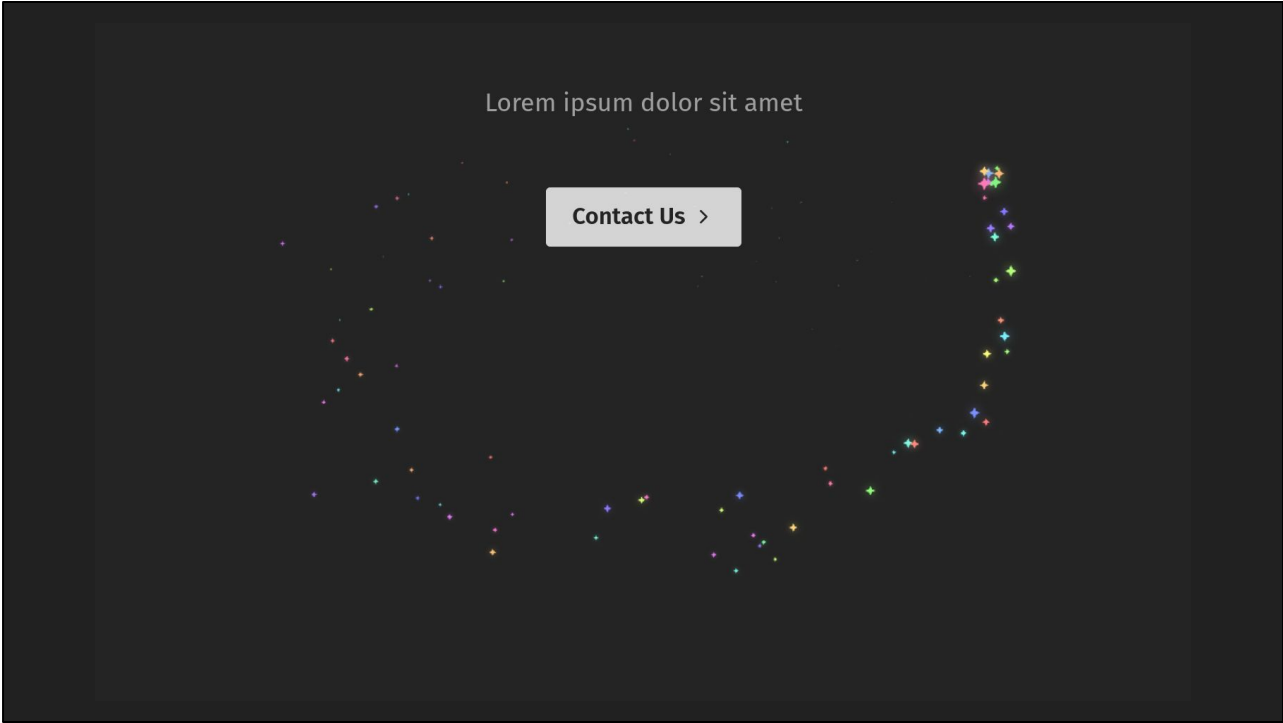
Labels
None yet

Projects
None yet

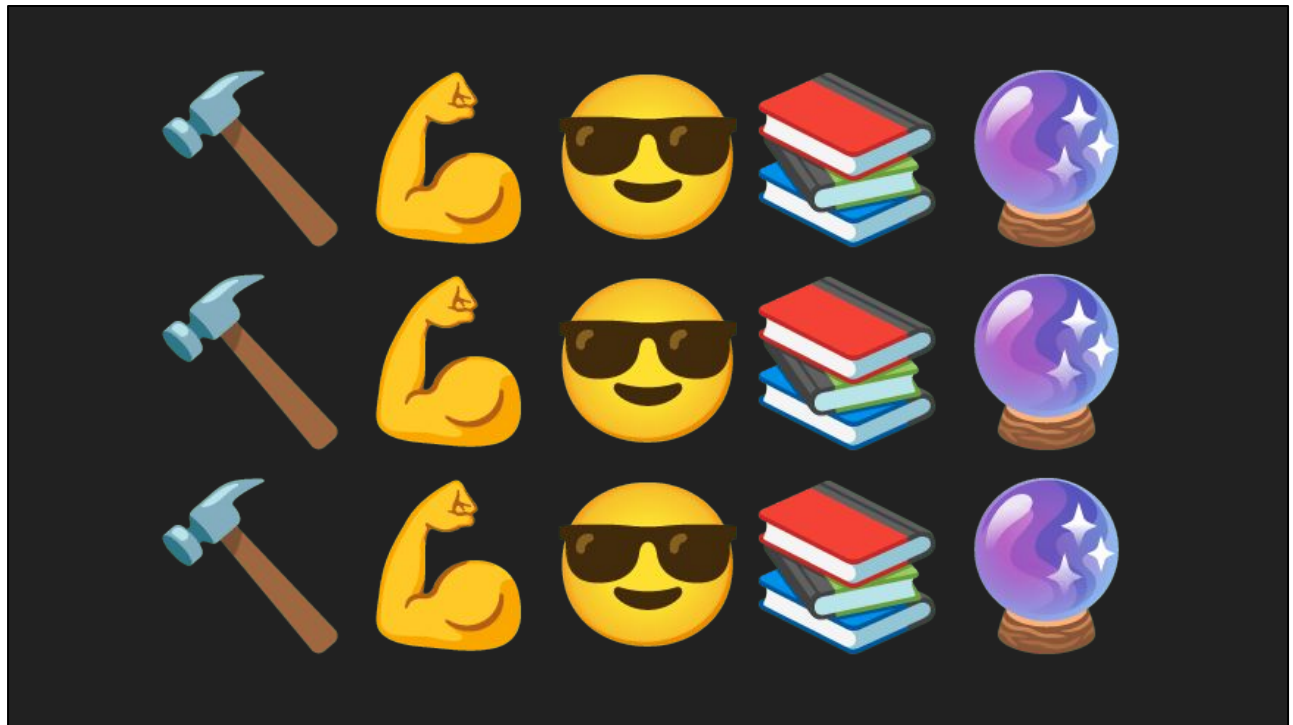
Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
None yet

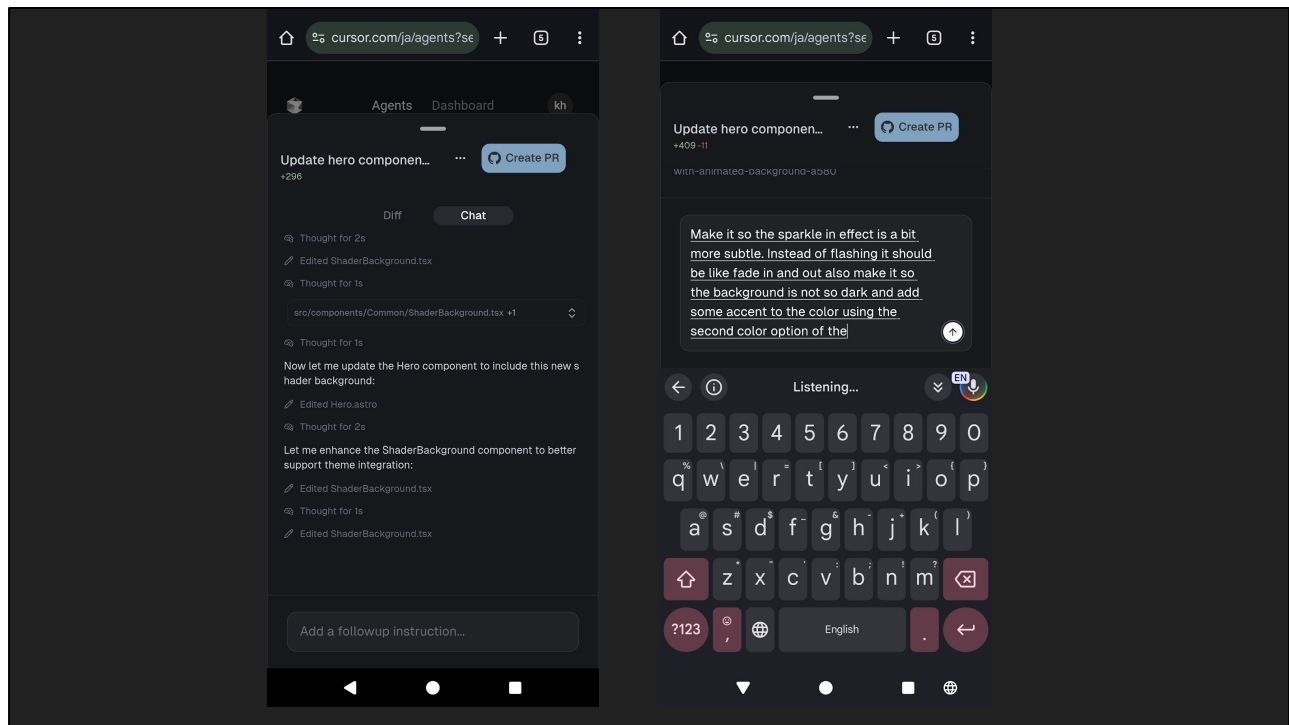
Combined with preview deployments, and you'll be able to view the changes directly from github.



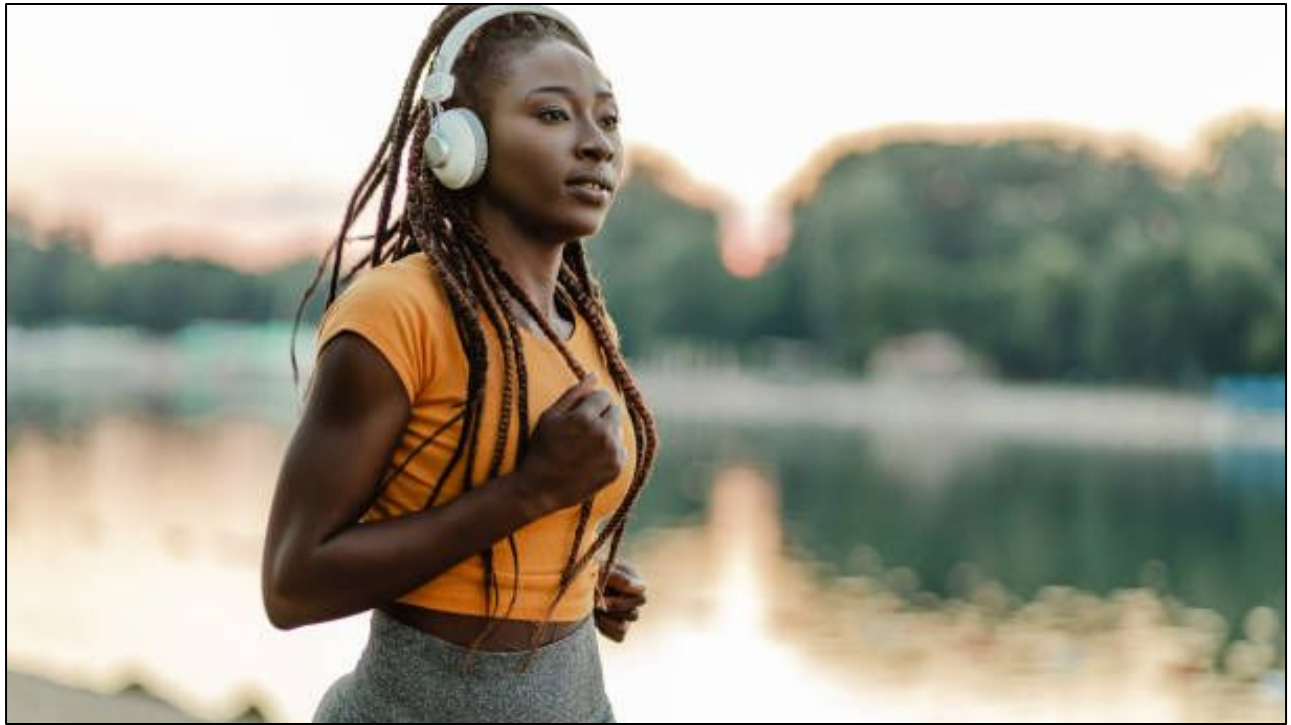
It's like magic!



But the real magic is being able to fire off multiple features at the same time.

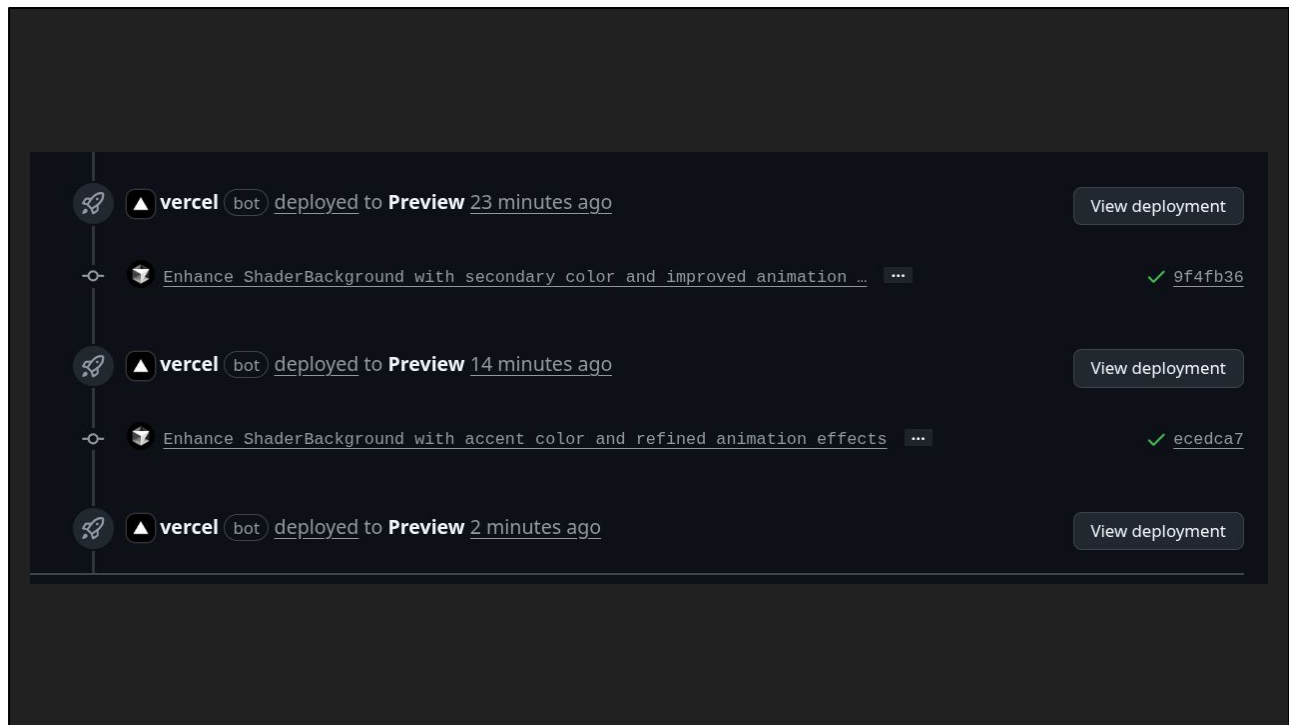


This new paradigm means you can code by just speaking to your phone.



Have a new idea for a feature while having a jog?

Don't waste a second and just tell your assistant to call the Background Agent MCP server.



By the time you get home, you'll be able to do the code review.

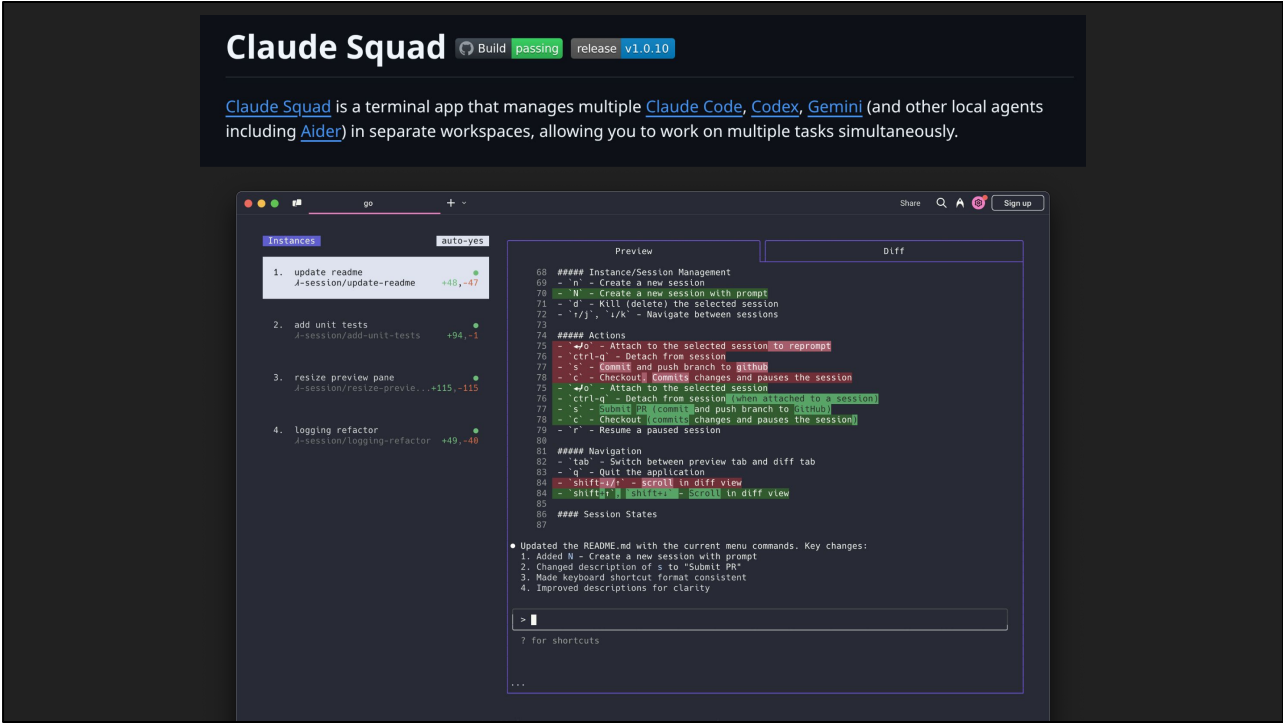
Agentic Engineering isn't just a workflow. It's a lifestyle.

Advanced Orchestration

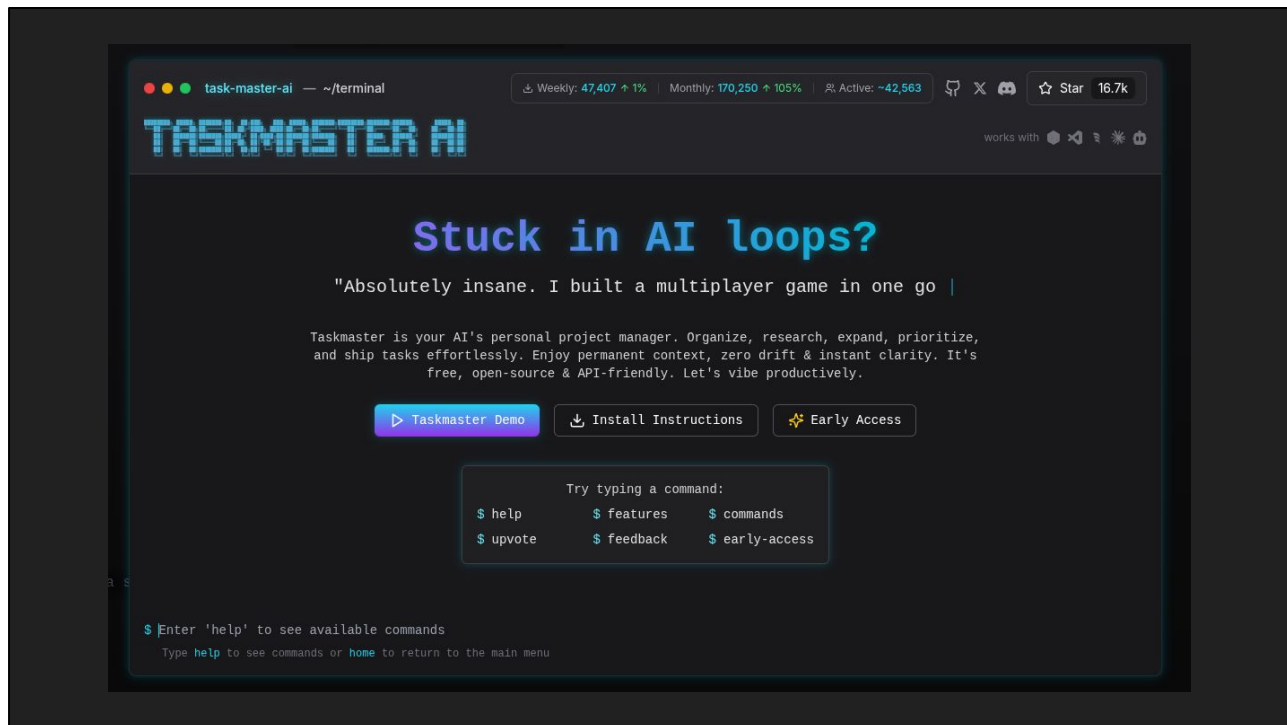
The Future of Agentic Engineering

If the last section was the cutting edge, we're now entering the bleeding edge.

I wanted to wrap things up by showcasing some experimental projects that aim to enhance Agentic Orchestration even further.



Claude Squad is simple multi-agent management tool that streamlines the process of managing Git Worktrees



Taskmaster is a framework for planning and implementing complex tasks and PRDs

Claude-Flow v1.0.72: Advanced AI Agent Orchestration Platform

 STARS **1.3K**  V1.0.72 **V1.0.72**  CLAUDE CODE **READY**

 RUNTIME **NODE | DENO**  TYPESCRIPT **FULL SUPPORT**  LICENSE **MIT**

Transform Your Development Workflow

Claude-Flow is the ultimate orchestration platform that revolutionizes how you work with Claude Code. Coordinate **multiple AI agents** simultaneously, manage complex workflows, and build sophisticated applications with AI-powered development.

 **One command to rule them all:** `npx claude-flow@latest init --sparc -`
Deploy a full AI agent coordination system in seconds!

Even more cutting edge, are a series of Context Engineering frameworks that handle both Task Management and Orchestration.

Claude-Flow unifies the two concepts into one tool that implements a multi-agent “SPARC” workflow.

BMad-Method: Universal AI Agent Framework

version [v4.27.6](#) License [MIT](#) node [>=20.0.0](#) [Discord](#) [Join Community](#)

Foundations in Agentic Agile Driven Development, known as the Breakthrough Method of Agile AI-Driven Development, but it is so much more. Transform any domain with specialized AI expertise: software development, entertainment, creative writing, business strategy to personal wellness just to name a few.

[Subscribe to BMadCode on YouTube](#)

[Join our Discord Community](#) - A growing community for AI enthusiasts! Get help, share ideas, explore AI agents & frameworks, collaborate on tech projects, enjoy hobbies, and help each other succeed. Whether you're stuck on BMad, building your own agents, or just want to chat about the latest in AI - we're here for you!

★ **If you find this project helpful or useful, please give it a star in the upper right hand corner!** It helps others discover BMad-Method and you will be notified of updates!

Some frameworks, like BMAD, implement tried and true traditional philosophies like Agile Development, replacing the humans with agents.

SuperClaude – Development Framework for Claude Code

License MIT version 2.0.1 issues 12 open PRs welcome

Persona	Focus Area	Tools	Use Cases
architect	System design	Sequential, Context7	Architecture planning
frontend	User experience	Magic, Puppeteer, Context7	UI development
backend	Server systems	Context7, Sequential	API development
security	Security analysis	Sequential, Context7	Security reviews
analyzer	Problem solving	All MCP tools	Debugging
qa	Quality assurance	Puppeteer, Context7	Testing
performance	Optimization	Puppeteer, Sequential	Performance tuning
refactorer	Code quality	Sequential, Context7	Code improvement

SuperClaude helps orchestrate multi-personality workflows that focus context on specific steps of the development process.

July 14, 2025 • 5 min read

Introducing Kiro

A new agentic IDE that works alongside you from prototype to production



Kiro's agent hooks act like an experienced developer catching the ball. Hooks are event-driven automations that execute when you create files—it's like delegating tasks to a collaborator. Set up a hook and Kiro handles the rest. Some examples:

- When you save a React component, hooks update the test files
- When you modify API endpoints, hooks refresh README files
- When you're ready to commit, security hooks scan for leaked credentials.

Finally, last week, a new IDE just dropped specifically for Agentic Engineering called Kiro.

Needless to say.

The space is hot.

And the hype is real.

Addendum

Extended Agentic Engineering

We've covered a lot of ground and reached the frontier of Agentic Engineering in 2025.

It's clear the next phase of Software Development is going to be all about effectively Orchestrating agents to handle increasingly lengthy and complex tasks.



If you squint a bit, things aren't that far away, and that explosion may have already been ignited.

Soon, Agentic Tooling and Techniques learned from Software Development will leak out into other industries.

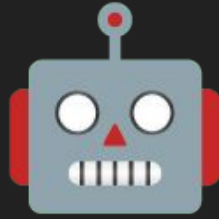
The skills learned today may compound, and turn out to be far more valuable than we can image.

Don't let the fear change allow you to miss out on a revolution.

At the very least, be prepared.



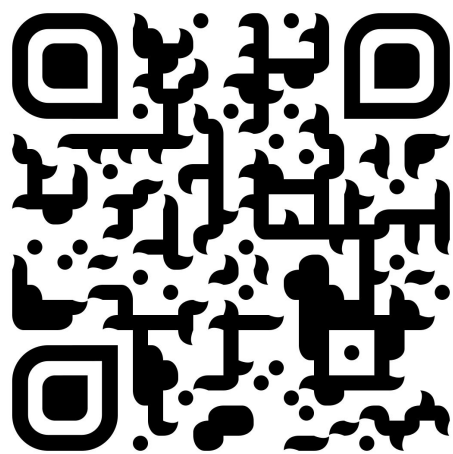
...and let's make sure the drone-sawm MCP requires KYC!



Thanks for your attention

and Happy Coding

Thanks for your attention, and Happy Coding.



<https://kris.com>